

**IMPLEMENTASI FINE-TUNING OPENAI WHISPER BASE
UNTUK AUTOMATIC SPEECH RECOGNITION BAHASA
MINANGKABAU**

TUGAS AKHIR

Fathan Andi Kartagama

122140055



**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
LAMPUNG SELATAN
2026**

LEMBAR PENGESAHAN

Saya menyatakan bahwa Tugas Akhir berjudul “Implementasi Fine-Tuning OpenAI Whisper Base untuk Automatic Speech Recognition Bahasa Minangkabau” merupakan hasil karya saya sendiri dan belum pernah diajukan, baik sebagian maupun seluruhnya, di Institut Teknologi Sumatera atau institusi pendidikan lain oleh saya maupun pihak lain.

Lampung Selatan, 30 April 2026

Penulis,

Fathan Andi Kartagama
NIM. 122140055

Foto 2x3

Diperiksa dan disetujui oleh,

Pembimbing

1. Martin Clinton Tosima Manullang, Ph.D.
NIP. 19930109 201903 1 017

.....

Penguji

1. I Wayan Wiprayoga Wisesa, S.Kom., M.Kom.
NIP. 19890322 201903 1 009
2. Radhinka Bagaskara, S.Si.Kom., M.Si., M.Sc.
NIP. 19941127 202012 1 018

.....

.....

Disahkan oleh,

Koordinator Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Andika Setiawan, S.Kom., M.Cs.
NIP. 19911127 2022 03 1 007

HALAMAN PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini

Menyatakan bahwa Tugas Akhir dengan judul : “Implementasi Fine-Tuning OpenAI Whisper Base untuk Automatic Speech Recognition Bahasa Minangkabau” adalah benar karya saya sendiri. Seluruh sumber baik yang dikutip maupun dirujuk dalam penulisan tugas akhir ini telah saya cantumkan secara benar sesuai dengan kaidah penulisan ilmiah.

Apabila di kemudian hari ditemukan adanya unsur plagiarisme dalam karya ini, maka saya bersedia menerima sanksi sesuai ketentuan yang berlaku di institusi saya.

Demikian pernyataan ini saya buat dengan sesungguhnya.

Lampung Selatan, 30 April 2026

Fathan Andi Kartagama
NIM. 122140055

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai civitas akademik Institut Teknologi Sumatera, saya yang bertanda tangan di bawah ini:

Nama : Fathan Andi Kartagama

NIM : 122140055

Program Studi : Teknik Informatika

Fakultas : Teknologi Industri

Jenis Karya : Tugas Akhir

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Institut Teknologi Sumatera **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

Implementasi Fine-Tuning OpenAI Whisper Base untuk Automatic Speech Recognition Bahasa Minangkabau

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Institut Teknologi Sumatera berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Lampung Selatan, 30 April 2026

Yang menyatakan

Fathan Andi Kartagama

NIM. 122140055

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan tugas akhir ini dengan baik.

Penyusunan tugas akhir ini tidak terlepas dari bimbingan dan dukungan berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Bapak Martin Clinton Tosima Manullang, Ph.D., selaku Dosen Pembimbing yang telah meluangkan waktu dan tenaga untuk mengarahkan penulis hingga tugas akhir ini selesai.
2. Bapak I Wayan Wiprayoga Wisesa, S.Kom., M.Kom., Radhinka Bagaskara, S.Si.Kom., M.Si., M.Sc., dan Prof. Sarwono Sutikno, Dr, Eng., selaku Dosen Penguji yang telah memberikan masukan, kritik, serta saran konstruktifnya.
3. Ibu tercinta yang senantiasa memberikan doa yang tak terputus, kasih sayang yang tulus, serta motivasi yang tiada henti kepada penulis dalam setiap langkah.
4. Kedua kakak dan kedua abang ipar atas dorongan semangat serta dukungannya selama proses penulisan.
5. Bapak Cahya Wirawan atas penyediaan dan pemeliharaan dataset LibriVox Indonesia di HuggingFace yang sangat membantu penelitian ini.
6. Seluruh pihak yang tidak dapat disebutkan satu per satu atas segala bantuan dan dukungannya.

Penulis menyadari bahwa tugas akhir ini masih jauh dari sempurna. Semoga karya ini dapat memberikan manfaat dan wawasan yang berguna bagi para pembaca.

*"BUILDING THE FUTURE AND KEEPING THE PAST ALIVE ARE ONE
AND THE SAME THING."*

- Solid Snake - Metal Gear Solid 2: Sons of Liberty

RINGKASAN

Implementasi Fine-Tuning OpenAI Whisper Base untuk Automatic Speech Recognition Bahasa Minangkabau

Fathan Andi Kartagama

Teknologi *Automatic Speech Recognition* (ASR) masih kurang mendukung bahasa bersumber daya rendah (*low-resource*) seperti Minangkabau akibat keterbatasan dataset dan tingginya variasi dialek. Oleh karena itu, penelitian ini bertujuan mengadaptasi model bahasa universal secara efisien ke bahasa daerah Minangkabau. Penelitian dilakukan melalui proses *fine-tuning* model OpenAI Whisper Base menggunakan arsitektur *Parameter-Efficient Fine-Tuning* via *Low-Rank Adaptation* (LoRA) untuk meminimalisasi beban komputasi. Optimasi *Weighted Cross-Entropy Loss* turut diintegrasikan guna mengatasi ketimpangan kemunculan fonem pada set pelatihan.

Evaluasi menggunakan *Word Error Rate* (WER) dan *Character Error Rate* (CER) membuktikan terwujudnya peningkatan akurasi yang signifikan dibandingkan mode *zero-shot* asalnya. Implementasi ganda antara metrik bobot kerugian dan LoRA sukses mereduksi kesalahan fonetik, walaupun sistem masih rentan terhadap diksi yang jarang terjadi. Kesimpulannya, metode eksperimental yang diuji ini merupakan kerangka yang efektif dan hemat biaya sumber daya untuk merancang ASR bahasa lokal. Disarankan untuk meluaskan representasi data wicara Minangkabau pada korpus masif masa depan untuk mempertinggi utilitas dunia nyata.

ABSTRAK

Teknologi pengenalan wicara atau *Automatic Speech Recognition* (ASR) saat ini mayoritas terpusat pada bahasa dengan sumber daya besar (*high-resource*), sementara bahasa daerah seperti Minangkabau yang berstatus *extremely low-resource* (1,3 jam data latihan) belum terfasilitasi dengan baik. Hal ini mengakibatkan rendahnya akurasi sistem ASR generik akibat kompleksitas ragam dialek lokal. Penelitian ini bertujuan untuk meningkatkan akurasi sistem ASR bahasa Minangkabau melalui implementasi *fine-tuning* pada model *pre-trained* OpenAI Whisper Base. Metodologi difokuskan pada teknik *Parameter-Efficient Fine-Tuning* (PEFT) menggunakan *Low-Rank Adaptation* (LoRA) untuk meminimalisasi beban komputasi, serta dibandingkan dengan strategi *Freeze Encoder*. Selain itu, fungsi *Weighted Cross-Entropy* (WCE) dengan pemisahan temporal diperkenalkan untuk menangani ketidakseimbangan sebaran fonem tanpa membocorkan data validasi. Performa model dievaluasi menggunakan skema *5-Fold Cross-Validation* dengan metrik *Word Error Rate* (WER) dan *Character Error Rate* (CER). Hasil pengujian mengonfirmasi bahwa LoRA secara konsisten mengungguli *Freeze Encoder*, mencapai rata-rata WER 15,67% dan CER 3,32% dengan hanya melatih 2,93% parameter model. Kesimpulannya, kombinasi LoRA dan pembobotan WCE terbukti efektif dan efisien dalam mentranskripsi bahasa Minangkabau, meskipun masih menyisakan sedikit tantangan pada tuturan fonetik logat spesifik. Di masa depan, perluasan korpus wicara melalui skema *crowdsourcing* menjadi rekomendasi untuk penelitian lanjutan.

Kata Kunci: Pengenalan Wicara, OpenAI Whisper, Minangkabau, *Fine-Tuning*, LoRA

ABSTRACT

Automatic Speech Recognition (ASR) technology is currently predominantly centered on high-resource languages, while regional languages such as Minangkabau, which are extremely low-resource (1.3 hours of training data), remain inadequately facilitated. This results in the low accuracy of generic ASR systems due to the complexity of local dialect variations. This study aims to improve the accuracy of Minangkabau language ASR systems through the implementation of fine-tuning on the pre-trained OpenAI Whisper Base model. The methodology focuses on the Parameter-Efficient Fine-Tuning (PEFT) technique using Low-Rank Adaptation (LoRA) to minimize computational load, and is compared against the Freeze Encoder strategy. Furthermore, a Weighted Cross-Entropy (WCE) function with temporal separation is introduced to address the imbalance of phoneme distribution without leaking validation data. Model performance is evaluated using a 5-Fold Cross-Validation scheme with Word Error Rate (WER) and Character Error Rate (CER) metrics. The experimental results confirm that LoRA consistently outperforms the Freeze Encoder, achieving an average WER of 15.67% and CER of 3.32% while only training 2.93% of the model parameters. In conclusion, the combination of LoRA and WCE weighting proves to be effective and efficient in transcribing the Minangkabau language, although minor challenges remain with specific dialectal phonetic utterances. In the future, expanding the speech corpus through a crowdsourcing scheme is recommended for further research.

Keywords: Speech Recognition, OpenAI Whisper, Minangkabau, Fine-Tuning, LoRA

DAFTAR ISI

LEMBAR PENGESAHAN	ii
HALAMAN PERNYATAAN ORISINALITAS	iii
HALAMAN PERSETUJUAN PUBLIKASI	v
KATA PENGANTAR	vii
MOTTO	viii
RINGKASAN	ix
ABSTRAK	xi
ABSTRACT	xii
DAFTAR ISI	xiii
DAFTAR TABEL	xviii
DAFTAR GAMBAR	xx
DAFTAR RUMUS	xxiii
DAFTAR KODE	xxv
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	5
1.4 Batasan Masalah	6
1.5 Manfaat Penelitian	8
1.6 Sistematika Penulisan	9
BAB II TINJAUAN PUSTAKA	13

2.1	Tinjauan Pustaka	13
2.1.1	Model Whisper dan Fondasi Teoritis	17
2.1.2	Strategi Fine-Tuning untuk Bahasa Low-Resource	17
2.1.3	Adaptasi untuk Aplikasi Dunia Nyata	18
2.1.4	Optimisasi Proses Fine-Tuning	19
2.1.5	Transfer Learning Multibahasa	20
2.1.6	ASR untuk Bahasa dengan Data Sangat Terbatas	20
2.1.7	Metrik Evaluasi ASR	21
2.1.8	Data Augmentation untuk ASR	21
2.1.9	Cross-Lingual dan Transfer Learning untuk ASR	22
2.1.10	Sintesis dan Posisi Penelitian	23
2.2	Dasar Teori	24
2.2.1	Bahasa Minangkabau	24
2.2.2	Automatic Speech Recognition (ASR)	25
2.2.2.1	Bahasa Low-Resource dan Tantangannya	25
2.2.3	Transfer Learning dan Model Pre-Trained	26
2.2.3.1	Model Pre-Trained untuk ASR	26
2.2.4	Whisper dan Arsitekturnya	27
2.2.4.1	Varian Model dan Parameter	28
2.2.4.2	Encoder	29
2.2.4.3	Decoder	29
2.2.4.4	Whisper Base	30
2.2.5	Fine-Tuning: Konsep dan Strategi	30
2.2.5.1	Full Fine-Tuning	30
2.2.5.2	Freeze Encoder Fine-Tuning	30
2.2.5.3	Parameter-Efficient Fine-Tuning (PEFT)	32
2.2.5.4	LoRA (Low-Rank Adaptation)	32
2.2.6	Metrik Evaluasi ASR	34
2.2.6.1	Word Error Rate (WER)	34

2.2.6.2	Character Error Rate (CER)	34
2.2.7	Data Preprocessing untuk ASR	35
2.2.7.1	Audio Preprocessing	35
2.2.7.2	Text Preprocessing	36
2.2.7.3	Data Augmentation	36
2.2.8	Regularisasi dan Optimisasi	37
2.2.8.1	Regularisasi	37
2.2.8.2	Optimisasi	37
2.2.9	Weighted Cross-Entropy Loss	38
2.2.10	K-Fold Cross-Validation	38
BAB III	METODE PENELITIAN	39
3.1	Alur Penelitian	39
3.2	Penjabaran Langkah Penelitian	41
3.2.1	Identifikasi Masalah	41
3.2.2	Studi Literatur	42
3.2.3	Pengumpulan Dataset	44
3.2.4	Preprocessing Dataset	46
3.2.4.1	Audio Preprocessing	47
3.2.4.2	Text Preprocessing	50
3.2.4.3	5-Fold Cross-Validation	51
3.2.5	Konfigurasi Model Whisper Base	52
3.2.6	Implementasi Fine-Tuning	53
3.2.6.1	Freeze Encoder Fine-Tuning	54
3.2.6.2	LoRA Fine-Tuning	58
3.2.6.3	Regularization	60
3.2.7	Training dan Monitoring	61
3.2.8	Evaluasi Model	63
3.2.8.1	Kategorisasi Error untuk Analisis Kualitatif	64

3.2.9	Analisis Hasil dan Perbandingan	66
3.3	Alat dan Bahan Tugas Akhir	67
3.3.1	Alat	67
3.3.1.1	Perangkat Keras	67
3.3.1.2	Perangkat Lunak	68
3.3.2	Bahan	71
3.4	Metode Pengembangan	72
3.4.1	Pendekatan Penelitian	72
3.4.2	Alur Pengembangan	73
3.4.3	Metodologi Fine-Tuning	75
3.4.3.1	Freeze Encoder Fine-Tuning Methodology	75
3.4.3.2	LoRA Fine-Tuning Methodology	76
3.4.3.3	Aggregation dan Statistical Testing	76
3.4.4	Metode Pengujian	77
3.4.4.1	Functional Testing	77
3.4.4.2	Performance Testing	77
3.4.4.3	Comparative Testing	77
3.5	Ilustrasi Perhitungan Metode	78
3.5.1	Perhitungan Word Error Rate (WER)	78
3.5.1.1	Formula WER	78
3.5.1.2	Contoh Perhitungan	78
3.5.2	Perhitungan Character Error Rate (CER)	79
3.5.2.1	Formula CER	79
3.5.2.2	Contoh Perhitungan	79
3.5.3	Perhitungan LoRA Parameters	80
3.5.3.1	Formula LoRA Parameters	80
3.5.3.2	Contoh Perhitungan untuk Whisper Base	80
BAB IV HASIL DAN PEMBAHASAN		83

4.1	Pendahuluan	83
4.2	Hasil Strategi Freeze Encoder	84
4.2.1	Perbandingan Sepuluh Percobaan Freeze Encoder	85
4.2.2	Hasil Cross-Validation Freeze Encoder (FE-10)	88
4.2.3	Konvergensi Training Freeze Encoder	89
4.2.4	Metrik Evaluasi Freeze Encoder	90
4.2.5	Prediksi Model Freeze Encoder	91
4.3	Hasil Strategi LoRA	92
4.3.1	Perbandingan Sepuluh Percobaan LoRA	92
4.3.2	Konfigurasi LoRA Terbaik (L-10)	95
4.3.3	Hasil Cross-Validation LoRA (L-10)	96
4.3.4	Konvergensi Training LoRA	98
4.3.5	Metrik Evaluasi LoRA	99
4.3.6	Prediksi Model LoRA	100
4.4	Perbandingan Strategi Fine-Tuning	104
4.4.1	Ringkasan Seluruh Dua Puluh Percobaan	104
4.4.2	Dampak Weighted Cross-Entropy dan Temporal Separation	105
4.4.3	Perbandingan Run Terbaik: FE-9 vs L-10	106
4.5	Pembahasan	111
4.5.1	Efektivitas LoRA pada Skenario Extremely Low-Resource	112
4.5.2	Kondisi Keuntungan Penggunaan Freeze Encoder	113
4.5.3	Dampak dan Mekanisme Weighted Cross-Entropy	114
4.5.4	Pengaruh Pengurangan Regularisasi	116
4.5.5	Analisis Trade-Off Performa vs Konsistensi	116
4.5.6	Dampak Augmentasi Data	117
4.5.7	Transfer Learning dan Kedekatan Tipologis	118
4.5.8	Pemetaan Kesalahan Berdasarkan Kelas Fonetik dan Linguistik Minangkabau	119
4.5.9	Limitasi dan Tantangan	120

BAB V KESIMPULAN DAN SARAN 123

5.1 Kesimpulan 123

5.2 Saran 128

DAFTAR PUSTAKA 131

LAMPIRAN 139

DAFTAR TABEL

Tabel 2.1	Ringkasan Penelitian Terdahulu (2022–2025).....	13
Tabel 2.2	Spesifikasi dan Kebutuhan Sumber Daya Varian Model Whisper	29
Tabel 3.1	Ruang Pencarian Hyperparameter — Freeze Encoder	56
Tabel 3.2	Ruang Pencarian Hyperparameter — LoRA	59
Tabel 3.3	Alignment untuk perhitungan WER (C=Correct, S=Substitution, D=Deletion).....	78
Tabel 4.1	Konfigurasi Hyperparameter Sepuluh Percobaan Freeze Encoder	85
Tabel 4.2	Perbandingan Hasil Sepuluh Percobaan Freeze Encoder ...	85
Tabel 4.3	Hasil 5-Fold Cross-Validation Strategi Freeze Encoder (FE-10).....	88
Tabel 4.4	Ringkasan Training Freeze Encoder FE-10 per Fold	89
Tabel 4.5	Contoh Prediksi Freeze Encoder FE-10 pada Fold 2 (WER Terbaik: 14,78%)	91
Tabel 4.6	Konfigurasi Hyperparameter Sepuluh Percobaan LoRA ...	93
Tabel 4.7	Perbandingan Hasil Sepuluh Percobaan LoRA.....	93
Tabel 4.8	Konfigurasi LoRA Terbaik (L-10).....	96
Tabel 4.9	Hasil 5-Fold Cross-Validation Strategi LoRA (L-10)	96
Tabel 4.10	Ringkasan Training LoRA L-10 per Fold	98
Tabel 4.11	Contoh Prediksi LoRA L-10 pada Fold 2 (WER dan CER Terbaik: 14,49% / 2,90%)	101
Tabel 4.12	Contoh Prediksi LoRA L-10 pada Fold 1 (WER: 14,63%) .	102
Tabel 4.13	Ringkasan Seluruh 20 Percobaan Training	104
Tabel 4.14	Perbandingan Strategi Freeze Encoder (FE-9) vs LoRA (L-10)	110

Tabel 4.15 Pemetaan Kesalahan Kata Berdasarkan Kelas Fonetik
Spesifik Minangkabau 120

DAFTAR GAMBAR

Gambar 2.1	Arsitektur dan <i>Multitask Training Format</i> Whisper [1] ..	28
Gambar 2.2	Visualisasi Strategi <i>Freeze Encoder</i> [1], [2]	31
Gambar 2.3	Visualisasi Mekanisme <i>Low-Rank Adaptation</i> (LoRA) [42].....	33
Gambar 3.1	Alur Penelitian Fine-Tuning Whisper untuk ASR Bahasa Minangkabau	40
Gambar 3.2	Alur Pengembangan Sistem	74
Gambar 4.1	Kurva <i>Training Loss</i> Strategi <i>Freeze Encoder</i> (FE-10)...	89
Gambar 4.2	Perkembangan WER dan CER Strategi <i>Freeze Encoder</i> (FE-10)	90
Gambar 4.3	Kurva <i>Training Loss</i> Strategi LoRA (L-10)	98
Gambar 4.4	Perkembangan WER dan CER Strategi LoRA (L-10) ...	100
Gambar 4.5	Perbandingan Metrik WER dan CER: Zero-Shot vs <i>Freeze Encoder</i> vs LoRA.....	110
Gambar 4.6	Ringkasan Cross-Validation, Strategi <i>Freeze Encoder</i> (FE-9).....	111
Gambar 4.7	Ringkasan Cross-Validation, Strategi LoRA (L-10)	111

DAFTAR RUMUS

Rumus 2.1	Formula WER (Word Error Rate)	34
Rumus 3.1	Definisi WER (Word Error Rate)	78
Rumus 3.2	Contoh Perhitungan WER	79
Rumus 3.3	Definisi CER (Character Error Rate)	79
Rumus 3.4	Contoh Perhitungan CER	80
Rumus 3.5	Formula Parameter LoRA	80
Rumus 3.6	Parameter LoRA Per Projection	80
Rumus 3.7	Parameter LoRA Per Layer	80
Rumus 3.8	Total Parameter LoRA (Matriks A dan B)	81
Rumus 3.9	Persentase Parameter LoRA Terhadap Total	81
Rumus 4.1	Perhitungan Selisih Kinerja per Fold (d_i)	107
Rumus 4.2	Rata-Rata Penekanan Kesalahan ($\bar{d}$)	108
Rumus 4.3	Perhitungan Standar Deviasi Uji-t (S_d)	108
Rumus 4.4	Perhitungan Skor Kepastian Uji (t)	108

DAFTAR KODE

Kode 3.1 Implementasi Kelas Pemrosesan Data Augmentation	49
Kode 3.2 Implementasi Fungsi Pembekuan Parameter Encoder (<i>Freeze Encoder</i>)	54
Kode 3.3 Implementasi Modifikasi <i>Weighted Cross-Entropy Loss</i> . . .	57
Kode 3.4 Implementasi Definisi Parameter LoRA Menggunakan Pustaka PEFT	60

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam beberapa dekade terakhir, kemajuan dalam pemrosesan suara dan pengenalan ucapan (*Automatic Speech Recognition / ASR*) telah menjadi salah satu bidang riset yang sangat penting, terutama karena potensinya besar di banyak aspek kehidupan manusia, misalnya asisten suara (*Siri, Google Assistant*), sistem transkripsi otomatis, aksesibilitas bagi penyandang disabilitas, layanan publik, pendidikan, serta interaksi manusia-komputer. Model-model berbasis pembelajaran mendalam (*deep learning*) yang menggunakan arsitektur *Transformer*, atau model "end-to-end", semakin banyak digunakan untuk menggantikan pendekatan tradisional seperti *Hidden Markov Model* dan *Gaussian Mixture Model*.

Namun, performa ASR sangat bergantung pada ketersediaan data berbicara (*speech*) dan transkripsi yang berkualitas dan representatif dari bahasa atau dialek target. Untuk bahasa mayor seperti Inggris, Mandarin, Spanyol, data sangat melimpah; sementara untuk bahasa lokal atau dialek (termasuk bahasa daerah di Indonesia) data seringkali sangat terbatas (*low-resource*). Model yang telah dilatih pada data multibahasa besar seperti Whisper kadang masih kurang optimal ketika menghadapi dialek atau bahasa yang kurang terwakili dalam data latihannya.

OpenAI memperkenalkan model Whisper sebagai model *speech-to-text* multibahasa dengan pelatihan lemah-supervisi skala besar (*Large-Scale Weak Supervision*) (sekitar 680.000 jam data audio transkrip) untuk generalisasi ke berbagai kondisi dan bahasa [1]. Meskipun demikian, model dasar ("base *Whisper*") seringkali tidak menghasilkan akurasi memadai ketika dihadapkan pada bahasa atau dialek yang sangat spesifik atau dengan variasi fonetik tinggi

yang belum banyak direpresentasikan. Oleh karena itu, teknik *fine-tuning* terhadap model dasar menjadi sangat penting agar model "teradaptasi" ke karakteristik bahasa lokal tersebut.

Teknik *fine-tuning* Whisper telah dieksplorasi dalam sejumlah studi, terutama untuk bahasa *low-resource*. Misalnya, penelitian *Exploration of Whisper fine-tuning strategies for low-resource ASR* menunjukkan bahwa berbagai strategi *fine-tuning* dapat secara signifikan meningkatkan performa pengenalan suara untuk bahasa-bahasa dengan data terbatas [2]. Ada pula penelitian "*Fine-tuning Whisper on Low-Resource Languages for Real-World Applications*" yang memperkenalkan metode transformasi dataset kalimat-ke-audio panjang untuk menjaga kemampuan segmentasi dan bagus bagi aplikasi nyata [3].

Studi lain mengusulkan metode *fine-tuning* efisien, seperti *LoRA-Whisper*, yang bertujuan agar adaptasi ke bahasa baru bisa dilakukan tanpa menurunkan performa bahasa lain dan dengan *overhead* parameter yang lebih kecil [4]. Penelitian terbaru juga menunjukkan efektivitas pendekatan *parameter-efficient fine-tuning* (PEFT) dengan berbagai strategi seperti *structured SVD* [5] dan *mixture of LoRA experts* [6] untuk meningkatkan adaptasi multibahasa. Juga ada penelitian *Multistage Fine-tuning Strategies for Automatic Speech Recognition in Low-resource Languages* yang menggunakan strategi *multistage* (bertahap) untuk mengadaptasi model melalui bahasa yang punya kemiripan linguistik [7].

Begitu juga, studi aplikasi domain khusus (misalnya dalam konteks medis) menekankan bahwa *fine-tuning* model Whisper pada domain spesifik (dengan kosakata dan pola suara khusus) mampu meningkatkan akurasi dibandingkan model standar [8]. Dengan demikian, adaptasi model ASR (melalui *fine-tuning*) telah menjadi pendekatan penting untuk menjembatani kesenjangan antara model umum dan kebutuhan lokal atau domain khusus.

Di Indonesia, keragaman bahasa dan dialek sangat tinggi, dan banyak komunitas menggunakan bahasa daerah dalam kehidupan sehari-hari. Bahasa

Minangkabau adalah salah satu bahasa daerah yang memiliki nilai budaya, sosial, dan historis tinggi, khususnya di wilayah Sumatera Barat dan daerah-daerah diaspora Minangkabau. Namun, dalam literatur riset ASR Indonesia, fokus utama lebih banyak pada bahasa Indonesia (standar) atau beberapa dialek besar (Sunda, Jawa, dsb.), sedangkan kontribusi penelitian terhadap bahasa Minangkabau relatif sedikit [9], [10].

Karena bahasa Minangkabau memiliki fonetik, kosakata, dan intonasi yang khas, transkripsi otomatis dengan model umum (bahasa Indonesia atau multibahasa) cenderung menghasilkan kesalahan (*error*) yang cukup tinggi. Hal ini dapat disebabkan oleh kekurangan data latih (audio + transkripsi) dalam bahasa Minangkabau, serta ketidakmampuan model awal menangani variasi dialek, aksen lokal, dan fenomena fonologi spesifik. Oleh karena itu, tugas untuk mengembangkan sistem ASR yang mampu mengenali ucapan bahasa Minangkabau dengan baik memerlukan adaptasi model melalui *fine-tuning*, agar model "belajar" karakteristik suara, kosakata, dan pola ucapan lokal tersebut.

Urgensi pengembangan sistem ASR untuk bahasa Minangkabau semakin meningkat seiring dengan kebutuhan nyata di berbagai sektor. Di bidang pelestarian budaya, terdapat banyak arsip audio sastra lisan Minangkabau (seperti *kaba*, *pantun*, dan cerita rakyat) yang tersimpan dalam format analog atau rekaman digital tanpa transkripsi, sehingga sulit diakses dan dianalisis oleh generasi muda maupun peneliti. Sistem ASR yang akurat dapat mempercepat digitalisasi dan dokumentasi warisan budaya ini. Di sektor media dan konten digital, para *content creator* lokal yang memproduksi konten video atau podcast berbahasa Minangkabau memerlukan alat transkripsi otomatis untuk membuat subtitle atau transkrip guna meningkatkan aksesibilitas dan jangkauan audiens. Selain itu, di bidang pariwisata dan layanan publik, sistem ASR dapat diintegrasikan ke dalam aplikasi pemandu wisata interaktif atau layanan informasi berbasis suara untuk wisatawan yang ingin memahami budaya Minangkabau. Dengan demikian, keberadaan sistem ASR bahasa

Minangkabau bukan hanya bersifat "*nice to have*" untuk riset akademis, tetapi merupakan kebutuhan *essential* yang memiliki dampak praktis langsung terhadap pelestarian budaya, ekonomi kreatif, dan aksesibilitas informasi bagi masyarakat Minangkabau.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka permasalahan utama dalam penelitian ini dapat dirumuskan sebagai berikut:

1. Bagaimana mengadaptasi (*fine-tuning*) model OpenAI Whisper Base agar mampu mengenali dan mentranskripsi ucapan dalam bahasa Minangkabau secara akurat, mengingat keterbatasan data suara dan teks (*low-resource*) yang tersedia?
2. Bagaimana proses perancangan dan implementasi sistem Automatic Speech Recognition (ASR) berbasis hasil *fine-tuning* tersebut dengan menggunakan bahasa pemrograman Python dan pustaka *machine learning* seperti PyTorch dan Hugging Face Transformers?
3. Bagaimana pengaruh penerapan strategi *fine-tuning* yang berbeda Freeze Encoder (Decoder-Only Fine-Tuning) sebagai pendekatan standar versus *parameter-efficient fine-tuning* dengan LoRA terhadap kinerja model Whisper Base dalam mengenali ucapan bahasa Minangkabau?
4. Bagaimana cara mengukur dan mengevaluasi performa model hasil *fine-tuning* menggunakan metrik kuantitatif seperti *Word Error Rate (WER)* dan *Character Error Rate (CER)*, serta melakukan analisis kualitatif terhadap kesalahan transkripsi dengan mengkategorikan jenis kesalahan (fonetik, leksikal, kontekstual, dan halusinasi) pada kondisi rekaman?
5. Bagaimana hasil implementasi dan pengujian sistem ASR yang dikembangkan dapat memberikan kontribusi nyata terhadap pelestarian bahasa daerah dan pengembangan teknologi pengenalan suara untuk bahasa lokal di Indonesia?

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah dikemukakan, penelitian ini memiliki beberapa tujuan sebagai berikut:

1. Mengadaptasi model OpenAI Whisper Base melalui proses *fine-tuning* agar mampu mengenali dan mentranskripsi ucapan dalam bahasa Minangkabau dengan tingkat akurasi yang lebih baik dibandingkan model dasar (*pretrained model*).
2. Merancang dan mengimplementasikan sistem Automatic Speech Recognition (ASR) berbasis hasil *fine-tuning* model Whisper menggunakan bahasa pemrograman Python dan pustaka *deep learning* seperti PyTorch serta Hugging Face Transformers.
3. Menerapkan dan membandingkan dua strategi fine-tuning, yaitu Freeze Encoder (Decoder-Only) dan LoRA (*parameter-efficient fine-tuning*), untuk mengevaluasi efektivitasnya pada skenario *extremely low-resource* (dataset 1,3 jam) berdasarkan *trade-off* akurasi, waktu pelatihan, kebutuhan memori, dan risiko *overfitting*.
4. Mengevaluasi performa model hasil fine-tuning menggunakan metrik kuantitatif seperti *Word Error Rate (WER)* dan *Character Error Rate (CER)*, serta melakukan analisis kualitatif dengan mengkategorikan kesalahan transkripsi berdasarkan tipe kesalahan (fonetik, leksikal, kontekstual, dan halusinasi) untuk mengidentifikasi pola kesalahan dominan dan memberikan rekomendasi perbaikan model.
5. Menghasilkan prototipe sistem ASR bahasa Minangkabau yang fungsional dan terverifikasi, serta memberikan kontribusi terhadap upaya pelestarian bahasa daerah melalui penerapan teknologi kecerdasan buatan di bidang pengenalan suara multibahasa.

1.4 Batasan Masalah

Agar penelitian ini memiliki ruang lingkup yang terarah dan dapat diselesaikan secara realistis dalam waktu yang terbatas, maka ditetapkan beberapa batasan masalah sebagai berikut:

1. Penelitian ini hanya berfokus pada implementasi *fine-tuning* model OpenAI Whisper Base untuk pengenalan ucapan (*Automatic Speech Recognition*) pada bahasa Minangkabau.
2. Dataset yang digunakan dalam penelitian ini bersumber dari LibriVox Indonesia, sebuah korpus audio publik yang menyediakan rekaman pembacaan *Universal Declaration of Human Rights* (UDHR) dalam bahasa Minangkabau beserta transkripsi teksnya. Dataset ini merupakan dataset *extremely low-resource* dengan total sekitar 1.3 jam audio (156 file audio dengan durasi rata-rata 30 detik). Untuk memaksimalkan penggunaan data yang sangat terbatas dan memperoleh evaluasi yang lebih robust, penelitian ini menggunakan pendekatan 5-Fold Cross-Validation, di mana seluruh 156 file dibagi menjadi 5 *folds* (distribusi: 4 folds berisi 31 audio, 1 fold berisi 32 audio karena $156 \div 5 = 31.2$), dengan setiap iterasi menggunakan 124-125 file untuk *training* dan 31-32 file untuk *testing*. Dataset terbatas pada domain pembacaan formal (*formal reading*) dari teks deklarasi hak asasi manusia, sehingga model yang dihasilkan merupakan *prototype* pada domain spesifik dan performa pada percakapan spontan atau konteks informal mungkin berbeda. Penelitian ini tidak mencakup proses pembuatan atau pengumpulan dataset baru dalam skala besar, melainkan fokus pada pemanfaatan dan *preprocessing* data yang sudah tersedia dengan pendekatan *transfer learning* yang efisien untuk mengatasi keterbatasan data.
3. Karakteristik audio pada dataset ini terbatas hanya pada suara penutur perempuan, sehingga model tidak dilatih maupun dievaluasi menggunakan

variasi suara laki-laki. Selain itu, kondisi rekaman audio yang digunakan bukanlah kualitas studio, melainkan mengandung banyak kebisingan latar (*noise*), gema ruangan, serta variasi kualitas mikrofon. Hal ini menjadi batasan intrinsik penelitian, di mana kemampuan model akan sangat terkalibrasi pada profil akustik (suara perempuan dan tipe *noise*) dalam dataset tersebut.

4. Model yang diujikan dibatasi pada varian Whisper Base dengan dua strategi *fine-tuning* yang dirancang untuk skenario *extremely low-resource*: (1) Freeze Encoder (Decoder-Only Fine-Tuning) sebagai pendekatan standar (membekukan seluruh lapisan *encoder* dan melatih seluruh *decoder* dengan 37 juta parameter, sekitar 50% dari total model), dan (2) LoRA (*Low-Rank Adaptation*) yang melatih $\sim 1,47\%$ – $2,93\%$ parameter ($\sim 1,08$ – $2,16$ juta parameter, tergantung konfigurasi *rank*) pada seluruh *linear layer* sebagai pendekatan *parameter-efficient*. Penelitian ini tidak membandingkan dengan varian Whisper lainnya (Tiny, Small, Medium, Large) atau strategi *full fine-tuning* (melatih seluruh model: *encoder* + *decoder*) karena pendekatan tersebut memerlukan dataset yang jauh lebih besar (>10 jam) untuk menghindari *overfitting*.
5. Evaluasi performa model dilakukan menggunakan metrik kuantitatif Word Error Rate (WER) dan Character Error Rate (CER). Selain itu, dilakukan analisis kualitatif terhadap kesalahan transkripsi dengan mengkategorikan jenis kesalahan ke dalam beberapa tipe, yaitu: (1) kesalahan fonetik (misalnya kesalahan pengenalan fonem vokal atau konsonan khas Minangkabau), (2) kesalahan leksikal (misalnya kesalahan pada kata serapan dari bahasa Indonesia atau bahasa asing yang diucapkan dalam bahasa Minangkabau), (3) kesalahan kontekstual (kesalahan pada kata yang bergantung pada konteks kalimat), dan (4) kesalahan halusinasi (pengulangan frasa atau teks acak saat periode hening, karakteristik intrinsik model Whisper). Analisis kualitatif ini akan dilakukan secara

manual terhadap sampel hasil transkripsi untuk mengidentifikasi pola kesalahan yang dominan dan memberikan rekomendasi perbaikan model. Evaluasi subjektif berbasis pengguna akhir (*user study*) tidak termasuk dalam ruang lingkup penelitian ini.

6. Implementasi sistem ASR dilakukan menggunakan bahasa pemrograman Python dengan pustaka PyTorch dan Hugging Face Transformers, serta dijalankan dalam lingkungan komputasi lokal atau GPU yang tersedia. Integrasi ke aplikasi lain (misalnya mobile atau web) tidak menjadi fokus utama penelitian ini.
7. Penelitian ini tidak membahas aspek keamanan data, privasi pengguna, maupun optimasi energi selama pelatihan model, melainkan berfokus pada adaptasi model dan analisis performa hasil fine-tuning.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat baik dari segi akademik maupun praktis, yaitu sebagai berikut:

1. Manfaat Akademik:

- (a) Memberikan kontribusi terhadap pengembangan ilmu pengetahuan di bidang kecerdasan buatan (Artificial Intelligence) dan pemrosesan suara (Speech Processing), khususnya dalam konteks *Automatic Speech Recognition* (ASR) untuk bahasa dengan sumber daya terbatas.
- (b) Menjadi referensi ilmiah dan bahan pembelajaran bagi mahasiswa atau peneliti lain yang tertarik untuk melakukan penelitian serupa dalam bidang *speech-to-text*, *low-resource language modeling*, dan *model adaptation*.
- (c) Menunjukkan penerapan teknik *fine-tuning* pada model OpenAI Whisper Base menggunakan pustaka PyTorch dan Hugging Face Transformers, sehingga dapat menjadi dasar penelitian lanjutan

terkait efisiensi model, optimasi pelatihan, dan adaptasi multibahasa.

2. Manfaat Praktis:

- (a) Menghasilkan prototipe sistem pengenalan suara bahasa Minangkabau yang dapat digunakan sebagai alat bantu transkripsi otomatis untuk konten audio formal, seperti pembacaan teks resmi, dokumentasi narasi budaya, atau rekaman arsip berbahasa Minangkabau yang bersifat formal. Perlu dicatat bahwa performa model pada percakapan spontan atau konteks informal mungkin memerlukan adaptasi lebih lanjut dengan data domain yang sesuai.
- (b) Mendukung pelestarian bahasa daerah melalui pemanfaatan teknologi AI yang dapat mempermudah digitalisasi dan dokumentasi arsip audio berbahasa Minangkabau, terutama untuk rekaman-rekaman historis atau konten edukatif yang bersifat formal.
- (c) Memberikan fondasi penelitian dan contoh implementasi untuk pengembangan sistem ASR bahasa daerah Indonesia, yang dapat menjadi *baseline* atau titik awal bagi penelitian lanjutan dengan dataset yang lebih beragam (termasuk percakapan spontan, podcast, atau konteks informal) untuk mencapai sistem ASR bahasa Minangkabau yang lebih general-purpose.

1.6 Sistematika Penulisan

Sistematika penulisan berisi penjelasan mengenai susunan bab dalam laporan tugas akhir ini. Setiap bab disusun secara sistematis agar pembaca dapat memahami alur penelitian yang dilakukan, mulai dari tahap perumusan masalah hingga kesimpulan akhir. Adapun sistematika penulisan tugas akhir ini adalah sebagai berikut:

Bab I Pendahuluan

Bab ini berisi penjelasan mengenai latar belakang penelitian, rumusan masalah, tujuan penelitian, batasan masalah, manfaat penelitian, serta sistematika penulisan. Bab ini memberikan gambaran umum mengenai alasan dan arah penelitian yang dilakukan, sehingga pembaca memahami konteks dan ruang lingkup penelitian ini.

Bab II Tinjauan Pustaka

Bab ini membahas landasan teori dan penelitian terdahulu yang relevan dengan topik tugas akhir, meliputi konsep dasar *Automatic Speech Recognition (ASR)*, model OpenAI Whisper, teknik *fine-tuning* dan *parameter-efficient fine-tuning* seperti LoRA, serta pembahasan mengenai bahasa Minangkabau sebagai bahasa low-resource. Bab ini menjadi dasar konseptual dan teoritis bagi metode yang digunakan dalam penelitian.

Bab III Metodologi Penelitian

Bab ini menjelaskan tahapan penelitian yang dilakukan, termasuk rancangan sistem, arsitektur model, dataset yang digunakan, tahapan *fine-tuning*, lingkungan pengujian, serta metode evaluasi performa. Selain itu, bab ini juga menjelaskan alur kerja penelitian mulai dari pengumpulan data, pelatihan model, hingga proses evaluasi hasil.

Bab IV Implementasi dan Hasil Penelitian

Bab ini membahas implementasi model dan sistem yang dikembangkan, hasil pengujian terhadap model *Whisper Base* yang telah di-*fine-tune*, serta analisis performa berdasarkan metrik *textitWord Error Rate (WER)*, *Character Error Rate (CER)*, dan evaluasi kualitatif terhadap hasil transkripsi. Pada bagian ini juga dijelaskan perbandingan antara model dasar dan model hasil adaptasi terhadap bahasa Minangkabau.

Bab V Kesimpulan dan Saran

Bab ini berisi kesimpulan yang diperoleh dari hasil penelitian dan saran untuk pengembangan lebih lanjut. Kesimpulan mencakup hasil utama penelitian, tingkat keberhasilan implementasi sistem ASR bahasa Minangkabau, serta potensi pengembangan model atau sistem di masa depan.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Pustaka

Bagian ini mengulas penelitian-penelitian terdahulu yang relevan dengan topik *fine-tuning* Whisper dan *Automatic Speech Recognition* (ASR) untuk bahasa *low-resource*. Setiap penelitian dianalisis secara kritis untuk mengidentifikasi kontribusi utama, keterbatasan, serta relevansinya dengan penelitian ini yang berfokus pada adaptasi Whisper Base untuk bahasa Minangkabau.

Tabel 2.1 Ringkasan Penelitian Terdahulu (2022–2025)

No.	Penelitian & Penulis	Fokus Masalah	Pendekatan & Kontribusi	Temuan Utama & Relevansi
1	<i>Robust Speech Recognition via Large-Scale Weak Supervision</i> Radford et al. [1] (2023)	Membangun ASR multibahasa yang <i>robust</i> dengan kemampuan generalisasi <i>zero-shot</i> tanpa <i>fine-tuning</i> spesifik	Pelatihan Whisper pada ~680k jam data audio multibahasa dengan paradigma <i>weak supervision</i> ; arsitektur <i>Transformer encoder-decoder</i>	Kinerja kuat di berbagai <i>benchmark</i> ASR; fondasi untuk adaptasi bahasa <i>low-resource</i> ; keterbatasan pada bahasa <i>under-represented</i> memotivasi <i>fine-tuning</i> untuk Minangkabau

No.	Penelitian & Penulis	Fokus Masalah	Pendekatan & Kontribusi	Temuan Utama & Relevansi
2	<i>Exploration of Whisper Fine-Tuning Strategies for Low-Resource ASR</i> Liu, Yang, & Qu [2] (2024)	Identifikasi strategi <i>fine-tuning</i> Whisper yang paling efektif untuk bahasa <i>low-resource</i>	Studi komparatif <i>full fine-tuning</i> , <i>partial fine-tuning</i> , <i>adapters</i> , dan PEFT (LoRA) pada berbagai bahasa <i>low-resource</i>	Semua strategi menurunkan WER; <i>trade-off</i> jelas antara akurasi dan efisiensi; panduan praktis untuk pemilihan strategi; basis untuk perbandingan <i>Freeze Encoder</i> vs LoRA pada Minangkabau
3	<i>Fine-tuning Whisper on Low-Resource Languages for Real-World Applications</i> Timmel et al. [3] (2024)	Model kehilangan kemampuan segmentasi dan <i>timestamping</i> saat dilatih hanya pada kalimat pendek	Rekonstruksi korpus <i>long-form</i> sintetis dari data kalimat pendek; evaluasi pada Swiss German	SOTA untuk Swiss German; segmentasi dan <i>timestamping</i> tetap terjaga; relevan untuk Minangkabau jika data <i>long-form</i> terbatas

No.	Penelitian & Penulis	Fokus Masalah	Pendekatan & Kontribusi	Temuan Utama & Relevansi
4	<i>Towards Efficiently Whisper Fine-tuning with Monotonic Alignments</i> Zhuang et al. [11] (2025)	<i>Fine-tuning</i> standar tidak mengoptimalkan keselarasan monotonic audio-teks secara eksplisit	Penambahan <i>soft monotonic alignment loss</i> tanpa modifikasi arsitektur model	WER turun konsisten pada ASR multibahasa dan <i>speech translation</i> ; overhead minimal; opsi pengembangan potensial untuk Minangkabau
5	<i>Indonesian Automatic Speech Recognition with XLSR-53</i> Arisaputra & Zahra [12] (2022)	Mencapai akurasi ASR kompetitif untuk bahasa Indonesia dengan data terbatas	<i>Fine-tuning</i> XLSR-53 (~24 jam) dikombinasikan dengan <i>language model</i> 4-gram (KenLM) pada <i>decoding</i>	WER turun drastis dari ~20% ke ~12% dengan LM; bukti <i>transfer learning</i> multibahasa efektif; mendukung hipotesis untuk bahasa daerah seperti Minangkabau

No.	Penelitian & Penulis	Fokus Masalah	Pendekatan & Kontribusi	Temuan Utama & Relevansi
6	<i>Breaking the Transcription Bottleneck: ASR for Extremely Low-Resource Languages</i> Liang & Levow [13] (2025)	Transkripsi bahasa dengan data sangat minimal (<1 jam) dalam konteks linguistik lapangan	<i>Benchmark</i> MMS vs. XLS-R pada 5 bahasa sangat <i>low-resource</i> dengan berbagai skala data	MMS unggul pada <1 jam data; paritas dengan XLS-R saat data bertambah; memberikan konteks batas bawah data untuk <i>fine-tuning</i> efektif
7	<i>From Speech to Summary: A Pipeline-Based Evaluation of Whisper and Transformer Models for Indonesian Dialogue Summarization</i> Martin et al. [14] (2026)	Evaluasi varian Whisper (tiny hingga turbo) untuk ASR bahasa Indonesia dan <i>benchmark</i> model <i>summarization</i> zero-shot	Evaluasi enam varian Whisper pada dataset percakapan Indonesia sintetis; <i>benchmark</i> 13 model <i>zero-shot summarization</i> berdasarkan metrik leksikal dan semantik	Whisper turbo (<i>distil-large-v2</i>) terbaik untuk bahasa Indonesia dengan WER 7,97% dan inferensi 1,25 detik; memberikan <i>baseline</i> perbandingan Whisper pada bahasa Indonesia sebagai acuan terhadap penelitian Minangkabau ini

2.1.1 Model Whisper dan Fondasi Teoritis

Radford et al. [1] memperkenalkan Whisper, model ASR multibahasa yang dilatih pada sekitar 680,000 jam data audio dengan paradigma *weak supervision*. Penelitian ini menjawab tantangan fundamental dalam membangun sistem ASR yang *robust* dan mampu melakukan generalisasi *zero-shot* pada berbagai bahasa dan kondisi audio tanpa memerlukan *fine-tuning* pada dataset spesifik. Whisper menggunakan arsitektur *Transformer encoder-decoder* yang dilatih pada tugas-tugas multibahasa, termasuk transkripsi dan translasi ucapan. Evaluasi komprehensif menunjukkan bahwa Whisper mencapai kinerja yang kuat di berbagai *benchmark* ASR dan translasi wicara dalam skenario *zero-shot*, membuktikan kemampuan generalisasi lintas bahasa yang luar biasa.

Meskipun Whisper menyediakan fondasi yang kokoh untuk adaptasi ke bahasa lokal, penelitian ini juga mengidentifikasi keterbatasan penting: bahasa dan dialek yang kurang terwakili dalam data pelatihan (*under-represented languages*) masih menghasilkan tingkat kesalahan yang relatif tinggi. Temuan ini memotivasi kebutuhan untuk *fine-tuning* model pada bahasa-bahasa *low-resource* seperti Minangkabau. Dalam konteks penelitian ini, Whisper menjadi model dasar yang akan diadaptasi melalui *fine-tuning* dengan strategi yang efisien untuk meningkatkan akurasi pada bahasa target.

2.1.2 Strategi Fine-Tuning untuk Bahasa Low-Resource

Liu, Yang, dan Qu [2] melakukan studi komparatif komprehensif terhadap berbagai strategi *fine-tuning* Whisper pada skenario data terbatas. Penelitian ini mengeksplorasi pendekatan yang beragam, termasuk *full fine-tuning* (melatih seluruh parameter model), *partial fine-tuning* dengan *freezing* sebagian lapisan, *re-initialization* lapisan atas, penggunaan *adapters*, serta metode Parameter-Efficient Fine-Tuning (PEFT) seperti LoRA. Evaluasi dilakukan pada berbagai bahasa *low-resource* untuk mengidentifikasi pola umum yang dapat dijadikan panduan praktis.

Hasil penelitian menunjukkan bahwa semua strategi yang diuji mampu menurunkan *Word Error Rate* (WER) dibandingkan model dasar, namun terdapat *trade-off* yang jelas antara akurasi dan efisiensi komputasi. *Full fine-tuning* cenderung menghasilkan akurasi tertinggi tetapi memerlukan memori GPU yang besar dan waktu pelatihan yang lama, sementara metode PEFT seperti LoRA menawarkan efisiensi signifikan dengan penurunan akurasi yang minimal. Keunggulan penelitian ini terletak pada panduan praktis untuk memilih strategi berdasarkan keterbatasan data dan sumber daya komputasi yang tersedia. Namun, keterbatasannya adalah fokus yang luas tanpa analisis mendalam pada satu bahasa atau dialek spesifik secara longitudinal.

Penelitian ini memberikan kontribusi penting sebagai dasar pemilihan strategi untuk bahasa Minangkabau. Dalam skripsi ini, perbandingan langsung antara *Freeze Encoder* dan LoRA pada Whisper Base akan dilakukan untuk mengidentifikasi strategi yang paling sesuai dalam konteks bahasa Minangkabau, dengan mempertimbangkan keterbatasan data dan infrastruktur komputasi yang tersedia.

2.1.3 Adaptasi untuk Aplikasi Dunia Nyata

Timmel et al. [3] mengangkat isu praktis penting dalam *fine-tuning* Whisper untuk bahasa *low-resource*, khususnya Swiss German. Penelitian ini mengidentifikasi bahwa kekurangan korpus audio *long-form* (rekaman panjang kontinu) menyebabkan model kehilangan kemampuan segmentasi dan *timestamping* ketika hanya dilatih pada kalimat-kalimat pendek yang terisolasi. Untuk mengatasi masalah ini, peneliti mengembangkan metode untuk menyusun ulang data kalimat pendek menjadi korpus audio *long-form* sintesis, kemudian melakukan *fine-tuning* Whisper pada data yang telah direkonstruksi tersebut.

Hasil evaluasi menunjukkan peningkatan signifikan pada aplikasi dunia nyata, terutama dalam menangani percakapan panjang dan kontinu, sementara kemampuan segmentasi dan *timestamping* tetap terjaga dengan baik. Pendekatan

ini mencapai *state-of-the-art* untuk Swiss German dan memberikan solusi praktis ketika data *long-form* asli sangat terbatas. Namun, validasi lintas bahasa daerah lain masih diperlukan untuk memastikan generalisasi metode ini.

Relevansi penelitian ini terhadap bahasa Minangkabau sangat tinggi, terutama jika dataset yang tersedia dari LibriVox Indonesia memiliki keterbatasan dalam bentuk *long-form*. Strategi penyusunan korpus *long-form* sintetis dapat diadopsi sebagai bagian dari *preprocessing* data untuk meningkatkan kemampuan model dalam menangani konteks yang lebih panjang dan kompleks.

2.1.4 Optimisasi Proses Fine-Tuning

Zhuang et al. [11] mengusulkan pendekatan inovatif untuk meningkatkan efisiensi *fine-tuning* Whisper dengan menambahkan *soft monotonic alignment loss* selama proses pelatihan. Penelitian ini didasarkan pada observasi bahwa *fine-tuning* standar tidak secara eksplisit mengoptimalkan keselarasan monotonic antara sekuens audio dan teks, yang seharusnya merupakan karakteristik alamiah dalam transkripsi ucapan. Dengan menambahkan *loss function* tambahan yang mendorong alignment monotonic tanpa mengubah arsitektur model, peneliti mencapai penurunan WER yang konsisten pada berbagai tugas ASR multibahasa, serta perbaikan pada tugas *speech translation*.

Keunggulan pendekatan ini adalah peningkatan akurasi dengan *computational overhead* yang minimal, karena tidak memerlukan modifikasi arsitektur atau penambahan parameter yang signifikan. Namun, metode ini memerlukan penyetelan *weight* untuk *loss function* tambahan, dan dampaknya pada bahasa yang sangat *low-resource* (dengan data sangat terbatas) masih memerlukan studi lanjutan. Dalam konteks penelitian ini, metode *monotonic alignment* menjadi opsi pengembangan potensial jika performa *baseline fine-tuning* untuk bahasa Minangkabau belum mencapai tingkat yang memadai.

2.1.5 Transfer Learning Multibahasa

Arisaputra dan Zahra [12] mendemonstrasikan efektivitas *transfer learning* untuk ASR bahasa Indonesia dengan data terbatas menggunakan model XLSR-53. Penelitian ini melakukan *fine-tuning* XLSR-53 pada sekitar 24 jam data audio bahasa Indonesia, dikombinasikan dengan *language model* berbasis KenLM 4-gram pada tahap *decoding* untuk meningkatkan akurasi transkrip. Hasil eksperimen menunjukkan penurunan WER yang dramatis dari sekitar 20% menjadi 12% dengan penambahan *language model*, membuktikan bahwa pendekatan *transfer learning* dari model *pre-trained* multibahasa sangat efektif untuk bahasa Indonesia.

Meskipun penelitian ini berfokus pada bahasa Indonesia baku dan tidak membahas dialek atau bahasa daerah, temuan ini memberikan bukti kuat bahwa representasi multibahasa yang dipelajari oleh model seperti XLSR dapat ditransfer ke bahasa Indonesia dengan efektif. Sebagai perbandingan non-Whisper, penelitian ini mendukung hipotesis bahwa model *pre-trained* multibahasa yang dikombinasikan dengan *fine-tuning* merupakan pendekatan yang viabel untuk bahasa Indonesia dan bahasa daerah seperti Minangkabau.

2.1.6 ASR untuk Bahasa dengan Data Sangat Terbatas

Liang dan Levow [13] mengeksplorasi tantangan transkripsi untuk bahasa-bahasa yang memiliki data sangat minimal, bahkan kurang dari 1 jam audio, dalam konteks penelitian linguistik lapangan (*fieldwork linguistics*). Penelitian ini membandingkan performa model MMS (*Massively Multilingual Speech*) dan XLS-R pada 5 bahasa yang sangat *low-resource* dengan berbagai skala data. Hasil evaluasi menunjukkan bahwa MMS unggul ketika data pelatihan sangat terbatas (kurang dari 1 jam), sementara performa kedua model mencapai paritas ketika data bertambah (lebih dari 1 jam).

Temuan ini memberikan panduan praktis untuk pemilihan arsitektur model berdasarkan ketersediaan data, meskipun fokus penelitian ini adalah pada MMS

dan XLS-R, bukan Whisper. Meskipun demikian, penelitian ini memberikan konteks penting tentang batas bawah data yang diperlukan untuk *fine-tuning* yang efektif. Dalam penelitian ini, meskipun dataset Minangkabau dari LibriVox Indonesia relatif terbatas, fokus tetap pada Whisper Base yang telah terbukti memiliki kemampuan *transfer learning* yang kuat, dengan eksplorasi strategi *fine-tuning* yang efisien melalui perbandingan *Freeze Encoder* dan LoRA.

2.1.7 Metrik Evaluasi ASR

Evaluasi performa sistem ASR memerlukan metrik yang akurat dan dapat dibandingkan lintas penelitian. *Word Error Rate* (WER) adalah metrik standar industri yang mengukur proporsi kata yang salah ditranskripsikan, termasuk kesalahan substitusi, insersi, dan delesi [15]. Metrik WER didasarkan pada algoritma *Levenshtein distance* [16] yang menghitung jumlah operasi edit minimum untuk mengubah prediksi menjadi referensi. Morris et al. [15] mengusulkan perbaikan terhadap WER konvensional dengan mempertimbangkan konteks linguistik dalam penghitungan kesalahan.

Character Error Rate (CER) adalah metrik komplementer yang mengukur kesalahan pada tingkat karakter, lebih sensitif terhadap kesalahan ejaan dan berguna untuk bahasa dengan morfologi kompleks atau tanpa batasan kata yang jelas. Dalam konteks bahasa Minangkabau yang belum memiliki standar ortografi resmi, CER memberikan perspektif tambahan untuk mengevaluasi akurasi model pada tingkat fonem dan grafem.

2.1.8 Data Augmentation untuk ASR

Dalam skenario *low-resource*, teknik *data augmentation* menjadi krusial untuk meningkatkan keberagaman data training dan mencegah *overfitting*. Salamon et al. [17] memperkenalkan *Scaper*, sebuah *framework* untuk sintesis audio dengan augmentasi yang terkontrol, termasuk penambahan *background noise* dan modulasi pitch. Kim et al. [18] mendemonstrasikan bahwa kombinasi

augmentasi temporal (*time masking*, *time stretching*) dan spektral (*frequency masking*, *pitch shifting*) dapat meningkatkan performa ASR pada dataset terbatas secara signifikan.

Park et al. [19] memperkenalkan *SpecAugment*, metode augmentasi pada representasi spektrogram yang telah menjadi standar dalam pelatihan model ASR modern. Ko et al. [20] menunjukkan bahwa *speed perturbation* (mengubah kecepatan audio antara 0.9x-1.1x) dapat meningkatkan robustness model terhadap variasi kecepatan bicara tanpa mengubah konten linguistik.

2.1.9 Cross-Lingual dan Transfer Learning untuk ASR

Pendekatan *cross-lingual learning* telah terbukti efektif untuk meningkatkan performa ASR pada bahasa *low-resource*. Conneau et al. [21] mendemonstrasikan bahwa representasi lintas bahasa yang dipelajari secara *unsupervised* dari model seperti XLM-R dapat ditransfer dengan baik ke berbagai tugas pemrosesan bahasa. Dalam konteks ASR multibahasa, Pratap et al. [22] memperkenalkan dataset Multilingual LibriSpeech (MLS) yang mencakup 8 bahasa dan menunjukkan bahwa pelatihan multibahasa meningkatkan generalisasi model pada bahasa target yang memiliki kemiripan linguistik.

Bagat dan Kumar [6] mengusulkan pendekatan *Mixture of LoRA Experts* untuk adaptasi ASR multi-aksen yang dapat menangani variasi dialek dalam satu model, sementara Laakkonen et al. [23] memperkenalkan metode *meta-learned LoRA* yang dapat digeneralisasi untuk deteksi *speech deepfake* lintas bahasa. Lin et al. [24] mendemonstrasikan efektivitas *resource-efficient fine-tuning* untuk identifikasi dialek, menunjukkan bahwa pendekatan PEFT dapat mencapai akurasi tinggi dengan overhead komputasi minimal. Penelitian-penelitian ini menunjukkan bahwa pendekatan PEFT dengan arsitektur modular dapat meningkatkan efisiensi adaptasi ke domain atau bahasa baru tanpa *catastrophic forgetting*.

Dalam konteks bahasa Indonesia dan dialeknya, pendekatan *transfer learning* dari model multibahasa seperti Whisper yang telah dilatih pada bahasa Indonesia dapat memberikan representasi awal yang baik untuk bahasa Minangkabau, mengingat kemiripan linguistik antara keduanya [25].

2.1.10 Sintesis dan Posisi Penelitian

Tinjauan terhadap literatur terbaru mengungkapkan beberapa temuan penting yang menjadi landasan penelitian ini. Pertama, pelatihan skala besar dengan paradigma *weak supervision* seperti yang dilakukan pada Whisper [1] memberikan fondasi representasi multibahasa yang kuat untuk adaptasi bahasa *low-resource*. Kedua, pemilihan strategi *fine-tuning* harus mempertimbangkan *trade-off* antara akurasi dan efisiensi komputasi [2], di mana metode seperti LoRA menawarkan alternatif yang efisien dibandingkan melatih lebih banyak parameter. Ketiga, rekayasa korpus *long-form* dapat membantu mempertahankan kemampuan segmentasi dan *timestamping* untuk aplikasi dunia nyata [3]. Keempat, modifikasi fungsi objektif seperti *monotonic alignment loss* dapat meningkatkan akurasi tanpa mengubah arsitektur [11]. Kelima, pembelajaran multibahasa dapat ditransfer secara efektif ke bahasa lokal dan daerah [12]. Keenam, bahkan pada bahasa Indonesia baku, Whisper turbo (*distil-large-v2*) masih menghasilkan WER 7,97% tanpa *fine-tuning* khusus (*zero-shot benchmark*) [14], mengindikasikan bahwa bahasa daerah seperti Minangkabau yang jauh lebih *low-resource* membutuhkan adaptasi yang lebih intensif.

Berdasarkan temuan-temuan tersebut, penelitian ini mengidentifikasi beberapa peluang pengembangan. Pertama, perbandingan sistematis antara *Freeze Encoder* dan LoRA pada Whisper Base untuk bahasa Minangkabau akan memberikan *insights* praktis tentang efisiensi penggunaan memori GPU tanpa mengorbankan akurasi secara signifikan. Kedua, jika data *long-form* Minangkabau terbatas, penyusunan korpus *long-form* sintetis dapat diadopsi untuk meningkatkan kemampuan model dalam konteks yang lebih panjang.

Ketiga, eksplorasi *alignment-based loss* dan/atau integrasi *language model* pada tahap *decoding* dapat menjadi strategi lanjutan untuk penurunan WER lebih lanjut.

2.2 Dasar Teori

Bagian ini menjelaskan landasan teoritis yang mendukung metodologi penelitian pada Bab III. Pembahasan disusun secara hierarkis dari konsep umum hingga spesifik: (i) Bahasa Minangkabau secara linguistik, (ii) *Automatic Speech Recognition* (ASR) dan tantangan bahasa *low-resource*, (iii) *transfer learning* dan model *pre-trained*, (iv) arsitektur Whisper, (v) strategi *fine-tuning* (*Full* dan *LoRA*), (vi) metrik evaluasi ASR, serta (vii) *preprocessing* data audio dan teks.

2.2.1 Bahasa Minangkabau

Bahasa Minangkabau adalah bahasa dari rumpun Austronesia yang secara historis dan kultural melekat pada etnis Minangkabau, utamanya dituturkan di provinsi Sumatera Barat, Indonesia, serta daerah-daerah perantauannya. Secara klasifikasi linguistik, bahasa ini berkerabat sangat erat dengan bahasa Melayu dan Indonesia, hal ini tercermin dari tingginya koeksistensi kognat, kemiripan sintaksis, serta struktur afiksasi dasarnya. Meski memiliki kedekatan tersebut, bahasa Minangkabau memiliki identitas leksikal, tata fonologi khusus (seperti peluluhan fonem tertentu di akhir kata), serta variasi dialek geografis yang sangat kaya yang saling dapat dipahami sesama penutur asli [9].

Sayangnya, dalam kajian Pemrosesan Bahasa Alami (*Natural Language Processing* atau NLP) dan Teknologi Ucapan (*Speech Technology*), bahasa Minangkabau berstatus sebagai bahasa dengan sumber daya rendah (*low-resource language*). Terdapat jarak yang signifikan antara intensitas penuturan bahasa ini secara lisan dalam kehidupan sehari-hari (hingga jutaan penutur) dengan minimnya jumlah korpus teks paralel maupun dataset audio berteknologi tinggi yang tersedia secara publik [25]. Tantangan kesenjangan data digital ini menjadi

landasan mengapa penelitian kecerdasan buatan, seperti sistem pengenalan wicara (ASR), wajib menggunakan strategi adaptasi tingkat lanjut agar mesin mampu merawat dan memahami bahasa daerah di era modern [9].

2.2.2 Automatic Speech Recognition (ASR)

Automatic Speech Recognition (ASR) adalah teknologi yang mengkonversi sinyal audio ucapan manusia menjadi teks tertulis secara otomatis [1]. Sistem ASR modern menggunakan pendekatan *deep learning* yang memanfaatkan arsitektur jaringan saraf untuk mempelajari pola kompleks dalam data audio. Perkembangan ASR telah mengalami evolusi signifikan [26], dari sistem berbasis *Hidden Markov Model* (HMM) [27] dan *Deep Neural Networks* untuk pemodelan akustik [28], hingga arsitektur *end-to-end* berbasis *Recurrent Neural Networks* dengan mekanisme *Connectionist Temporal Classification* (CTC) [29], [30] dan *attention-based sequence-to-sequence models* seperti *Listen, Attend and Spell* [31], serta arsitektur *Transformer* modern [32] dan variannya seperti *Conformer* [33] yang mampu memodelkan hubungan jarak jauh dalam sekuens audio.

Tantangan utama dalam ASR adalah variabilitas tinggi dalam ucapan manusia yang dipengaruhi oleh faktor-faktor seperti aksen, dialek, kecepatan bicara, *background noise*, dan karakteristik *speaker* individual. Untuk bahasa dengan sumber daya tinggi seperti bahasa Inggris, ketersediaan dataset besar (ribuan jam audio teranotasi) memungkinkan model mencapai akurasi tinggi. Namun, untuk bahasa daerah *low-resource* seperti Minangkabau, minimnya dataset merupakan hambatan signifikan [2].

2.2.2.1 Bahasa Low-Resource dan Tantangannya

Bahasa *low-resource* didefinisikan sebagai bahasa dengan keterbatasan data teranotasi yang tersedia untuk pelatihan model ASR [13]. Tantangan spesifik bahasa *low-resource* meliputi: (i) kurangnya korpus audio-transkrip berkualitas tinggi, (ii) minimnya keberagaman *speaker* dan konteks penggunaan

dalam dataset, (iii) keterbatasan sumber daya komputasi untuk pengumpulan data skala besar, dan (iv) variasi dialek yang tidak tercakup dalam data yang tersedia [3], [34]. Penelitian terbaru menunjukkan bahwa pendekatan *weighted cross-entropy loss* dapat meningkatkan performa ASR multibahasa untuk bahasa *low-resource* dengan memberikan bobot lebih tinggi pada bahasa minoritas selama pelatihan [35].

Penelitian terbaru menunjukkan bahwa pendekatan *transfer learning* dari model *pre-trained* multibahasa dapat mengatasi keterbatasan data pada bahasa *low-resource* [12], [34], [36]. Model yang telah dilatih pada bahasa-bahasa dengan sumber daya tinggi dapat di-adaptasi ke bahasa target dengan jumlah data yang relatif sedikit, memanfaatkan representasi linguistik dan akustik yang telah dipelajari dari bahasa-bahasa lain [21].

2.2.3 Transfer Learning dan Model Pre-Trained

Transfer learning adalah paradigma *machine learning* di mana pengetahuan yang dipelajari dari satu domain (*source domain*) ditransfer ke domain lain (*target domain*) untuk meningkatkan performa dengan data pelatihan yang terbatas [37]. Dalam konteks ASR, *transfer learning* memungkinkan model yang telah dilatih pada bahasa-bahasa dengan sumber daya tinggi untuk diadaptasi ke bahasa *low-resource* melalui proses *fine-tuning*.

2.2.3.1 Model Pre-Trained untuk ASR

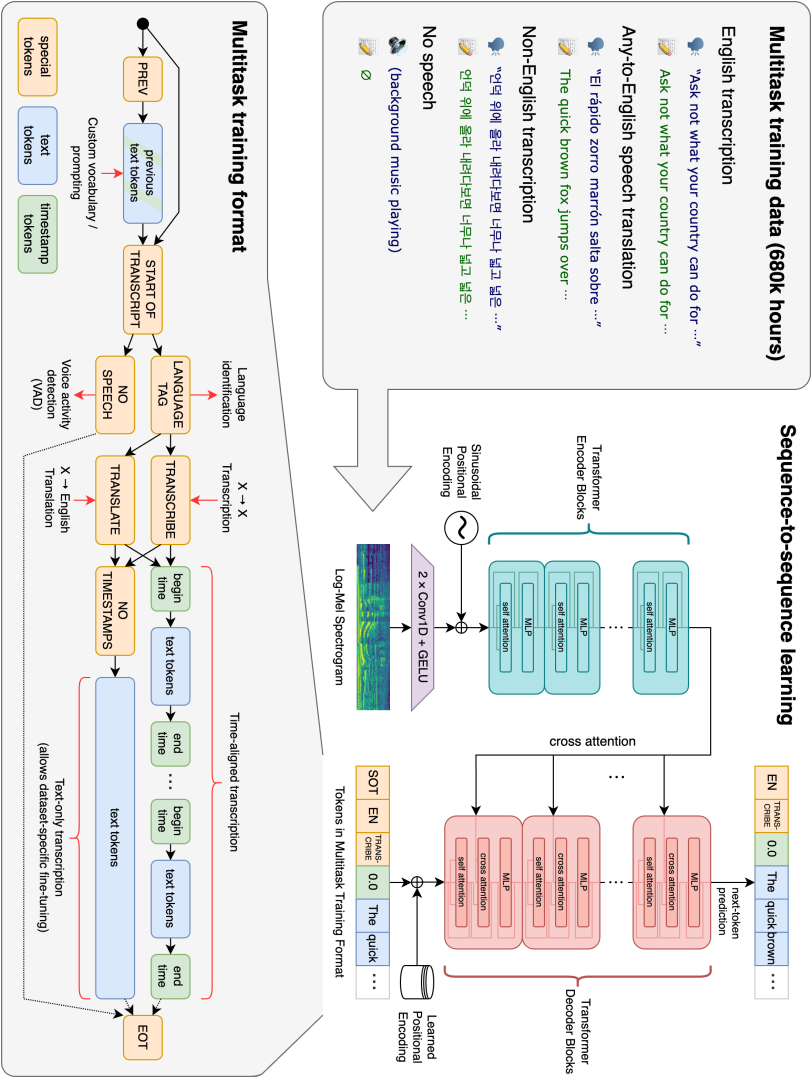
Model *pre-trained* multibahasa seperti wav2vec 2.0 [38], XLS-R [39], HuBERT [40], WavLM [41], dan Whisper [1] telah menunjukkan kemampuan luar biasa dalam *transfer learning* untuk ASR. Model-model ini dilatih pada ratusan ribu jam data audio multibahasa, memungkinkan mereka mempelajari representasi akustik dan linguistik yang generik dan dapat ditransfer ke bahasa-bahasa baru.

2.2.4 Whisper dan Arsitekturnya

Whisper adalah model *general-purpose speech recognition* yang dilatih pada dataset audio berskala besar yang beragam. Berbeda dengan model ASR konvensional yang umumnya dilatih pada dataset terkurasi, Whisper dilatih menggunakan pendekatan *weakly supervised* pada 680.000 jam data audio multibahasa dan multitugas yang dikumpulkan dari internet [1]. Skala data yang masif dan keragaman sumber data ini memberikan Whisper kemampuan *robustness* yang tinggi terhadap berbagai aksen, kebisingan latar belakang, dan bahasa teknis, serta memungkinkannya melakukan transkripsi dalam berbagai bahasa maupun terjemahan ke dalam bahasa Inggris.

Secara arsitektural, Whisper menggunakan model *Transformer* [32] yang sangat handal dalam menerjemahkan pola rumit. Analoginya, bayangkan Whisper sebagai seorang penerjemah poliglot yang sangat cerdas. Ia sudah mendengarkan dan mempelajari ribuan jam percakapan dari internet dalam berbagai bahasa.

Penerjemah ini bekerja dengan dua otak utama: 1. Telinga (Encoder): Bagian ini bertugas mendengarkan suara, menangkap nada, intonasi, dan membuang suara bising. 2. Mulut (Decoder): Bagian ini bertugas merangkai suara yang didengar menjadi teks tertulis kata demi kata.



Gambar 2.1 Arsitektur dan *Multitask Training Format* Whisper [1]

2.2.4.1 Varian Model dan Parameter

Whisper tersedia dalam berbagai ukuran ”kapasitas otak”. OpenAI merilis beberapa varian: *Tiny*, *Base*, *Small*, *Medium*, dan *Large*. Pemilihan ini ibarat

memilih seberapa besar kapasitas memori sang penerjemah; semakin besar maka semakin pintar, tapi butuh tenaga komputer yang jauh lebih berat.

Tabel 2.2 menyajikan perbandingan ukuran model Whisper.

Tabel 2.2 Spesifikasi dan Kebutuhan Sumber Daya Varian Model Whisper

Size	Parameters	English-only	Multilingual	VRAM	Relative Speed
Tiny	39 M	tiny.en	tiny	~1 GB	~10x
Base	74 M	base.en	base	~1 GB	~7x
Small	244 M	small.en	small	~2 GB	~4x
Medium	769 M	medium.en	medium	~5 GB	~2x
Large	1550 M	N/A	large	~10 GB	1x
Turbo	809 M	N/A	turbo	~6 GB	~8x

Dalam penelitian ini, model Whisper Base (74 juta parameter) dipilih. Kapasitas ini dianggap paling pas: cukup cerdas untuk menangkap logat Minangkabau, namun tidak terlalu besar sehingga tidak gampang "menghafal" (*overfitting*) data suara kita yang hanya berdurasi 1,3 jam.

2.2.4.2 Encoder

Encoder (Si Telinga) mengubah suara menjadi serangkaian sinyal frekuensi. Sebelumnya, sistem pengenalan suara menggunakan teknologi yang lebih tradisional, tetapi Whisper menggunakan *Transformer* yang memungkinkannya mengaitkan hubungan satu kata dengan kata lain meskipun posisinya berjauhan dalam satu kalimat pembicaraan.

2.2.4.3 Decoder

Decoder (Si Mulut) bertugas menebak dan menuliskan kata apa yang sedang diucapkan berdasarkan sinyal dari Encoder. Ia bekerja secara berurutan, menebak kata demi kata dari awal kalimat sampai kalimat tersebut selesai diucapkan.

2.2.4.4 Whisper Base

Whisper Base menawarkan keseimbangan yang sangat kuat antara kecerdasan dan efisiensi. Sangat cocok digunakan untuk situasi di mana data bahasa daerah kita sangatlah terbatas [2].

2.2.5 Fine-Tuning: Konsep dan Strategi

Meskipun Whisper sudah sangat pintar (*pre-trained*), ia belum pernah secara khusus diajari bahasa Minangkabau asli. *Fine-tuning* adalah proses menyekolahkan kembali model ini menggunakan data bahasa daerah kita agar ia terbiasa dengan logat dan kosakatanya [2], [3].

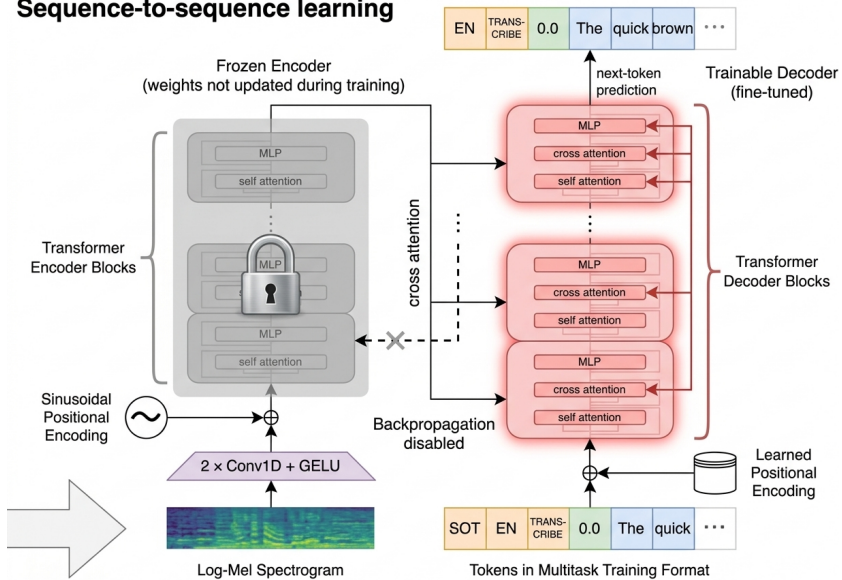
2.2.5.1 Full Fine-Tuning

Full fine-tuning ibarat merombak ulang seluruh otak sang penerjemah dari nol. Ia bebas belajar segalanya, tapi proses ini butuh waktu sangat lama dan komputer super canggih. Jika datanya sedikit, proses perombakan total ini justru berbahaya karena mesin cenderung hanya menghafal tanpa memahami polanya [2].

2.2.5.2 Freeze Encoder Fine-Tuning

Strategi ini membagi tugas: fungsi pendengaran (*Encoder*) dibekukan atau dikunci agar tidak berubah, karena mesin ini pada dasarnya sudah sangat pintar membedakan suara manusia dari suara bising. Kita hanya melatih ulang bagian "mulutnya" (*Decoder*) agar ia tahu bagaimana menuliskan suara tersebut ke dalam ejaan Minangkabau. Ini sangat menghemat waktu komputasi dan memori [2], [3].

Sequence-to-sequence learning



Gambar 2.2 Visualisasi Strategi *Freeze Encoder* [1], [2]

Visualisasi pada Gambar 2.2 mengilustrasikan cara kerja strategi ini secara sederhana. Sisi kiri model yang bertugas mendengarkan audio (Encoder) diberi label "Frozen" beserta ikon gembok. Hal ini menandakan bahwa memori pemahaman suara bawaan mesin sudah sangat mapan sehingga bobot di area tersebut dikunci dan tidak perlu dilatih ulang maupun diutak-atik. Di sisi lain, bagian yang bertugas memproduksi kalimat dari suara (Decoder) diberi label "Trainable" dengan instruksi terbuka, menandakan bahwa khusus hanya bagian ini yang secara aktif dipoles dan dilatih ulang kemampuannya supaya sanggup merangkai kata dalam dialek dan ejaan bahasa Minangkabau. Melalui diskriminasi tugas cerdas ini, kinerja pelatihan menjadi efisien dan cepat tanpa harus membebani kinerja komputer.

Catatan Terminologi: Istilah "Freeze Encoder" akan digunakan sebagai terminologi utama untuk strategi ini.

2.2.5.3 Parameter-Efficient Fine-Tuning (PEFT)

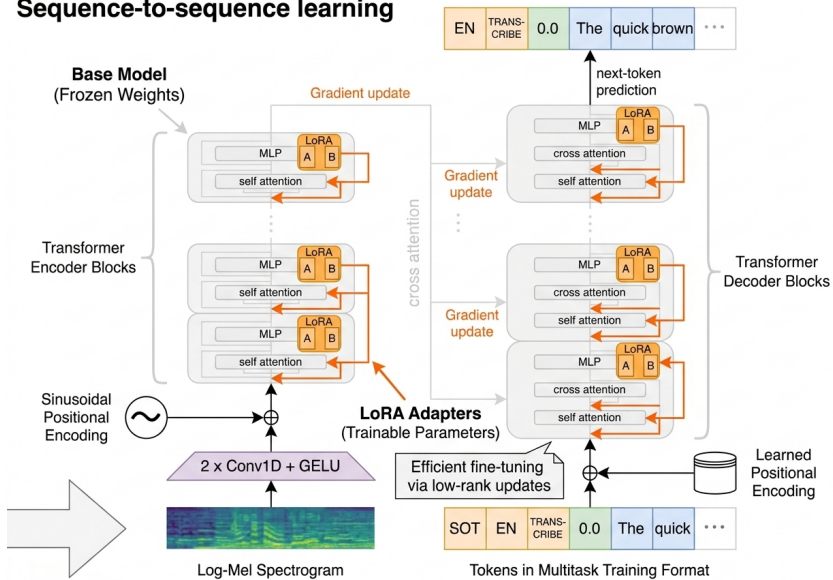
Parameter-Efficient Fine-Tuning (PEFT) adalah keluarga metode jenius yang melatih ulang mesin tanpa merombak otak utamanya.

2.2.5.4 LoRA (Low-Rank Adaptation)

LoRA (*Low-Rank Adaptation*) [42] adalah salah satu teknik PEFT paling populer. Alih-alih merombak ulang seluruh isi kepala sang penerjemah (yang butuh waktu dan tenaga masif), LoRA bekerja layaknya kita menyelipkan sebuah ”buku catatan kecil” di saku sang penerjemah. Buku catatan ini hanya berisi kosakata, logat halus, dan kebiasaan bahasa Minangkabau.

Selama penerjemah bekerja, ia menggunakan kepintaran aslinya, lalu merujuk pada buku catatan kecil tersebut saat ia menemui struktur bahasa Minang. Karena ”buku catatan” ini ukurannya sangat kecil (menghemat lebih dari 98% parameter yang harus dilatih), proses pelatihannya menjadi sangat ringan, hemat biaya komputer, namun tetap menghasilkan prediksi yang impresif dan tajam.

Sequence-to-sequence learning



Gambar 2.3 Visualisasi Mekanisme Low-Rank Adaptation (LoRA) [42]

Secara mekanis, proses ini diilustrasikan pada Gambar 2.3. Balok tebal di sebelah kiri (*Pretrained Weights*) adalah kumpulan pengetahuan asli model bahasa yang sengaja dipertahankan kuat dan dikunci (dilabeli *frozen*). Karena merombak balok besar ini menyita daya komputer yang brutal, strategi LoRA memasang jalur alternatif di sisi kanannya. Jalur ini terdiri dari perantara dua buah matriks ringan yaitu A dan B . Sepanjang masa pelatihan ke dalam bahasa Minangkabau, model cukup secara terfokus memoles dua kotak kecil tersebut. Matriks ini didesain berukuran sangat ramping dalam bentuk dimensi *rank*-nya (r), namun telah dipastikan cukup solid untuk mencatat kekhasan pola unik pengucapan dialek. Saat mesin akhirnya bertugas melakukan transkripsi wicara sesungguhnya, rumus luaran dari rute "buku saku" ($B \times A$) ini tinggal dijumlahkan (+) langsung dengan rute kecerdasan primernya. AI versi kolaborasi ini akhirnya piawai menerjemahkan bahasa daerah tanpa pernah terbebani untuk merombak total otak aslinya.

2.2.6 Metrik Evaluasi ASR

Evaluasi performa model ASR dilakukan menggunakan metrik yang mengukur akurasi transkrip yang dihasilkan dibandingkan dengan transkrip referensi. Dua metrik utama yang digunakan adalah *Word Error Rate* (WER) dan *Character Error Rate* (CER).

2.2.6.1 Word Error Rate (WER)

Word Error Rate (WER) adalah metrik standar untuk evaluasi ASR yang mengukur *edit distance* pada level kata antara transkrip hipotesis (*textithypothesis*, output model) dan transkrip referensi (*reference*, *ground truth*) [1]. WER dihitung sebagai:

$$\text{WER} = \frac{S + D + I}{N} \times 100\% \quad (\text{Rumus 2.1})$$

di mana S adalah jumlah substitusi (kata yang salah diganti), D adalah jumlah penghapusan (kata yang hilang dalam hipotesis), I adalah jumlah penyisipan (kata tambahan dalam hipotesis), dan N adalah total kata dalam referensi. WER dinyatakan dalam persentase, di mana nilai lebih rendah menunjukkan akurasi lebih tinggi (0% berarti sempurna, 100% atau lebih berarti kinerja sangat buruk).

2.2.6.2 Character Error Rate (CER)

Character Error Rate (CER) mengukur *edit distance* pada level karakter, dihitung dengan formula yang sama dengan WER tetapi operasi substitusi, penghapusan, dan penyisipan diterapkan pada karakter individual [15]. CER lebih sensitif terhadap kesalahan fonetik dan memberikan perspektif komplementer terhadap WER, terutama untuk bahasa dengan sistem penulisan yang kompleks atau ketika kesalahan parsial dalam kata perlu dipertimbangkan.

Kedua metrik ini penting karena memberikan informasi yang berbeda: WER mengukur akurasi semantik pada level kata (lebih penting untuk pemahaman makna), sementara CER memberikan granularitas lebih tinggi dan berguna untuk mengidentifikasi pola kesalahan fonetik spesifik.

2.2.7 Data Preprocessing untuk ASR

Preprocessing data adalah tahap kritis dalam pengembangan sistem ASR yang memastikan data input konsisten, berkualitas tinggi, dan kompatibel dengan persyaratan model [1], [19].

2.2.7.1 Audio Preprocessing

Audio *preprocessing* mencakup beberapa operasi penting:

1. **Resampling:** Konversi *sampling rate* audio ke frekuensi standar yang digunakan oleh model (16 kHz untuk Whisper). Resampling memastikan konsistensi temporal audio dan kompatibilitas dengan arsitektur model [1].
2. **Normalisasi Amplitudo:** Standarisasi volume audio di seluruh dataset untuk menghindari bias model terhadap audio dengan volume tertentu. Normalisasi dapat dilakukan dengan teknik seperti *peak normalization* atau *RMS normalization* [43].
3. **Segmentasi:** Pemotongan audio panjang menjadi segmen dengan durasi yang sesuai untuk pelatihan (biasanya 10-30 detik). Segmentasi meningkatkan efisiensi pelatihan dan mengurangi penggunaan memori [1].
4. **Silence Removal:** Penghapusan periode *silence* yang tidak informatif di awal dan akhir klip audio, sehingga model fokus pada bagian yang mengandung informasi linguistik [43].
5. **Quality Filtering:** Penyaringan klip dengan *Signal-to-Noise Ratio* (SNR) rendah atau distorsi signifikan untuk memastikan hanya data berkualitas

tinggi yang digunakan dalam pelatihan [1].

2.2.7.2 Text Preprocessing

Transkrip teks juga memerlukan standardisasi:

1. **Normalisasi Huruf:** Konversi teks ke huruf kecil atau format konsisten lainnya untuk mengurangi variabilitas yang tidak relevan [1].
2. **Normalisasi Tanda Baca:** Standarisasi penggunaan tanda baca di seluruh dataset [1].
3. **Normalisasi Whitespace:** Pembersihan spasi berlebih dan karakter *whitespace* lainnya [1].

Catatan: Berdasarkan pemeriksaan dataset UDHR Minangkabau yang digunakan dalam penelitian ini, transkrip tidak mengandung karakter numerik (0-9), sehingga normalisasi angka-ke-kata (*number-to-word normalization*) tidak diperlukan.

2.2.7.3 Data Augmentation

Data augmentation adalah teknik untuk meningkatkan keberagaman dan ukuran dataset pelatihan secara artifisial, yang sangat bermanfaat dalam skenario *low-resource*. Teknik augmentasi untuk ASR meliputi:

1. **SpecAugment** [19]: Augmentasi pada representasi spektrogram dengan menerapkan *masking* pada dimensi waktu dan frekuensi. Teknik ini memaksa model menjadi lebih *robust* terhadap variasi lokal dalam spektrum audio.
2. **Speed Perturbation** [20]: Variasi kecepatan audio dengan faktor tertentu (misalnya, 0.9, 1.0, 1.1) untuk mensimulasikan variasi kecepatan bicara alami.
3. **Noise Injection** [18]: Penambahan *background noise* dengan SNR tertentu untuk meningkatkan *robustness* model terhadap kondisi audio yang beragam.

4. **Pitch Shift** [18]: Modifikasi nada (*pitch*) audio dengan menaikkan atau menurunkan beberapa *semitone* tanpa mengubah durasi atau tempo aslinya. Teknik ini membantu model agar lebih tangguh terhadap variasi karakteristik frekuensi dasar suara (seperti suara rendah/tinggi) antar pembicara yang berbeda.

2.2.8 Regularisasi dan Optimisasi

Untuk mencegah *overfitting* pada dataset kecil dan memastikan konvergensi pelatihan yang stabil, berbagai teknik regularisasi dan optimisasi diterapkan.

2.2.8.1 Regularisasi

1. **Dropout** [44]: Teknik regularisasi yang secara acak men-*drop* neuron selama pelatihan untuk mengurangi *co-adaptation* dan meningkatkan generalisasi.
2. **Label Smoothing** [45]: Teknik yang menghaluskan distribusi target saat pelatihan untuk mencegah *overconfidence* pada prediksi dan meningkatkan kalibrasi model.
3. **Gradient Clipping** [46]: Pembatasan norma gradien untuk mencegah *exploding gradients* selama *backpropagation*.
4. **Early Stopping**: Menghentikan pelatihan ketika performa pada *validation set* tidak lagi meningkat, mencegah *overfitting* pada *training set*.

2.2.8.2 Optimisasi

AdamW [47] adalah algoritma optimisasi yang digunakan secara luas untuk *fine-tuning* model *Transformer*. AdamW adalah varian dari Adam yang memisahkan *weight decay* dari optimisasi gradien, menghasilkan regularisasi yang lebih efektif [48]. AdamW menggunakan *adaptive learning rates* per-parameter berdasarkan estimasi momen pertama dan kedua dari gradien, memungkinkan konvergensi cepat sambil mempertahankan stabilitas pelatihan.

2.2.9 Weighted Cross-Entropy Loss

Weighted Cross-Entropy (WCE) adalah ibarat sistem guru yang cerdas dalam menghukum kesalahan muridnya [35]. Pada sistem pembelajaran standar, mesin dihukum dengan bobot nilai yang sama untuk setiap kata yang salah ditebak (misalnya salah menebak kata "dan" dihukum sama beratnya dengan salah menebak kata spesifik Minang seperti "ancak").

Dengan WCE, sistem ini memberikan perhatian dan "hukuman" ekstra pada kata-kata Minang yang secara statistik paling sering salah ditebak oleh komputer di tes-tes sebelumnya. Hasilnya, alih-alih terus-terusan mengulangi kesalahan yang sama, mesin dipaksa untuk lebih serius belajar dari kelemahan terbesarnya. Teknik ini sangat berguna untuk bahasa daerah yang memiliki banyak kosakata unik yang jarang ditemui mesin sebelumnya.

2.2.10 K-Fold Cross-Validation

Dalam sistem AI, ada risiko mesin terlihat pintar bukan karena ia benar-benar paham bahasanya, tapi karena ia "menghafal" kunci jawaban dari data rekaman yang terbatas. Untuk mencegah hal ini, digunakanlah teknik *K-Fold Cross-Validation* [49].

Analoginya seperti ini: untuk memastikan AI benar-benar pintar, kita membagi total data ujian (rekaman suara) menjadi beberapa bagian (misalnya 5 bagian, disebut 5-Fold). 1. Mesin disuruh belajar menggunakan 4 bagian data. 2. Kemudian mesin dites menggunakan 1 bagian data yang belum pernah ia lihat sama sekali. 3. Proses ini diulang 5 kali dengan memutar bagian mana yang menjadi bahan belajar dan mana yang menjadi bahan ujian.

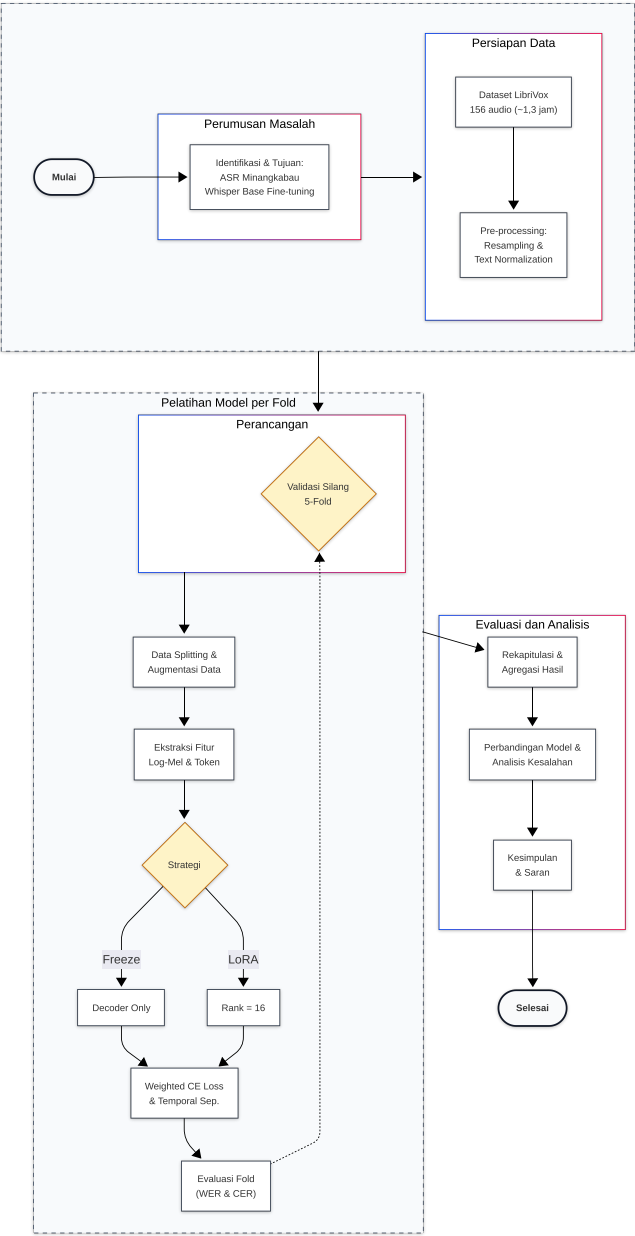
Jika hasil ujiannya rata-rata tetap bagus di tes mana pun, berarti mesin tersebut terbukti murni paham bahasa Minangkabau dan bukan cuma menghafal. Pendekatan ini sangat krusial agar AI yang dihasilkan benar-benar tangguh saat dipakai di dunia nyata [49].

BAB III

METODE PENELITIAN

3.1 Alur Penelitian

Pada penelitian ini, alur dirancang untuk memastikan setiap tahapan pemrosesan dilakukan secara sistematis dan efisien. Alur penelitian ini mencerminkan langkah-langkah utama terkait bagaimana proses yang dilakukan dalam penelitian, dari awal sampai dengan akhir. Gambar dalam bentuk diagram alir (*flowchart*) seperti yang terlihat pada Gambar 3.1.



Gambar 3.1 Alur Penelitian Fine-Tuning Whisper untuk ASR Bahasa Minangkabau

3.2 Penjabaran Langkah Penelitian

Penelitian ini dilakukan secara sistematis melalui beberapa tahapan utama yang saling terkait. Setiap tahapan dirancang untuk memastikan model Whisper Base dapat diadaptasi secara optimal untuk mengenali bahasa Minangkabau. Berikut adalah penjabaran detail dari setiap langkah penelitian yang telah digambarkan pada flowchart di Subbab 3.1.

3.2.1 Identifikasi Masalah

Tahap pertama dalam alur penelitian ini adalah identifikasi masalah, yang dilakukan untuk memetakan konteks serta urgensi dari penelitian yang akan dilakukan. Proses ini diawali dengan analisis terhadap permasalahan di dunia nyata, yaitu rendahnya ketersediaan sistem *Automatic Speech Recognition* (ASR) untuk bahasa daerah Indonesia, khususnya bahasa Minangkabau. Analisis ini divalidasi dengan meninjau statistik dan studi terkait yang menunjukkan bahwa sebagian besar sistem ASR komersial yang tersedia saat ini dirancang untuk bahasa-bahasa dengan sumber daya tinggi seperti bahasa Inggris, Mandarin, dan bahasa Eropa lainnya, sementara bahasa daerah Nusantara masih sangat terbatas representasinya.

Setelah urgensi masalah teridentifikasi, proses dilanjutkan dengan menganalisis tantangan spesifik yang dihadapi dalam pengembangan ASR untuk bahasa *low-resource* seperti Minangkabau. Hasil analisis menemukan bahwa tantangan utama terletak pada minimnya ketersediaan dataset audio-transkrip berkualitas tinggi yang diperlukan untuk melatih model ASR [2], [3]. Kondisi ini berbeda dengan bahasa-bahasa besar yang memiliki ribuan jam data audio teranotasi. Identifikasi tantangan *low-resource* ini menegaskan kebutuhan akan pendekatan yang efisien, sehingga penelitian ini difokuskan pada pemanfaatan model *pre-trained* seperti Whisper yang telah dilatih pada skala besar dan dapat diadaptasi untuk bahasa target dengan data yang relatif terbatas [1].

Berdasarkan analisis tersebut, penelitian ini merumuskan *research gap* yang jelas: belum ada penelitian yang secara komprehensif membandingkan strategi *fine-tuning* yang berbeda (*Freeze Encoder* vs *LoRA*) untuk adaptasi Whisper pada bahasa Minangkabau dalam skenario *extremely low-resource*. Identifikasi masalah ini mencakup beberapa aspek kunci berikut:

1. Analisis keterbatasan sistem ASR *existing* untuk bahasa daerah Indonesia, khususnya dari perspektif ketersediaan model dan dataset
2. Identifikasi tantangan bahasa *low-resource*: minimnya dataset audio-transkrip Minangkabau yang berkualitas dan teranotasi
3. Perumusan *research gap*: perlunya adaptasi model *pre-trained* (Whisper) untuk bahasa Minangkabau dengan pendekatan yang efisien secara komputasi
4. Penetapan objektif penelitian: membandingkan efektivitas strategi *fine-tuning* (*Freeze Encoder* vs *LoRA*) dalam hal akurasi dan efisiensi untuk dataset yang sangat terbatas
5. Penentuan metrik evaluasi utama: *Word Error Rate* (WER) dan *Character Error Rate* (CER) sebagai indikator kinerja model

3.2.2 Studi Literatur

Setelah masalah utama teridentifikasi, langkah selanjutnya dalam alur penelitian adalah melakukan studi literatur secara sistematis dan komprehensif. Proses ini bertujuan untuk memvalidasi *research gap* yang telah diidentifikasi pada tahap sebelumnya, mengumpulkan landasan teori yang relevan, serta memetakan metode *state-of-the-art* yang akan digunakan sebagai dasar perbandingan dalam penelitian ini. Studi literatur dilakukan dengan pendekatan yang terstruktur untuk memastikan bahwa semua aspek penting dari penelitian tercakup dengan baik.

Studi literatur difokuskan pada empat area utama yang saling berkaitan. Pertama, mendalami arsitektur dan kemampuan model Whisper dari OpenAI,

termasuk mekanisme *encoder-decoder Transformer*, proses *pre-training* pada 680,000 jam data multibahasa, dan karakteristik yang membuat Whisper unggul dalam skenario *zero-shot* maupun *few-shot* [1]. Kedua, mengidentifikasi dan menganalisis strategi *fine-tuning* yang telah terbukti efektif untuk skenario *extremely low-resource*, dengan fokus khusus pada perbandingan antara *Freeze Encoder* yang hanya melatih *decoder* versus teknik *Parameter-Efficient Fine-Tuning* (PEFT) seperti LoRA yang hanya melatih sebagian kecil parameter [2], [42]. Ketiga, mempelajari penelitian-penelitian terdahulu yang telah menerapkan Whisper atau model ASR lainnya pada bahasa Indonesia dan bahasa daerah Nusantara, untuk memahami tantangan spesifik dan solusi yang telah dicoba [12], [14]. Keempat, memahami karakteristik linguistik bahasa Minangkabau, termasuk fonologi, morfologi, dan pola ucapan yang dapat mempengaruhi performa model ASR [9], [25].

Proses studi literatur dilaksanakan melalui tahapan-tahapan berikut:

1. *Review paper ilmiah terkait Whisper dan strategi fine-tuning dari jurnal internasional bereputasi (e.g., ICASSP, Interspeech, ACL) dan conference proceedings di bidang speech processing*
2. *Analisis mendalam terhadap penelitian terdahulu tentang ASR untuk bahasa Indonesia dan bahasa daerah Nusantara, dengan fokus pada metodologi, dataset yang digunakan, dan hasil yang dicapai*
3. *Studi dokumentasi teknis OpenAI Whisper dan ekosistem Hugging Face Transformers library, termasuk API reference, model cards, dan best practices untuk fine-tuning*
4. *Eksplorasi metode fine-tuning yang tersedia, membandingkan karakteristik Freeze Encoder dengan pendekatan PEFT seperti LoRA, Adapter, dan Prefix-Tuning*
5. *Pemahaman mendalam tentang metrik evaluasi ASR, khususnya Word Error Rate (WER) dan Character Error Rate (CER), termasuk interpretasi dan keterbatasannya*

Hasil dari studi literatur ini menjadi landasan teoritis yang kokoh dalam memilih arsitektur model, menentukan strategi *fine-tuning* yang optimal, dan merancang metodologi evaluasi yang tepat untuk penelitian ini. Selain itu, studi literatur juga mengonfirmasi validitas *research gap* yang telah diidentifikasi dan memberikan justifikasi ilmiah untuk pendekatan yang diusulkan.

3.2.3 Pengumpulan Dataset

Dataset LibriVox Indonesia (<https://huggingface.co/datasets/indonesian-nlp/librivox-indonesia>) terdiri dari audio MP3 dan file teks yang sesuai yang dihasilkan dari buku audio domain publik LibriVox. Pengumpulan data dikhususkan pada bahasa-bahasa di Indonesia. Durasi buku audio atau file suara LibriVox asli bervariasi dari beberapa menit hingga beberapa jam, namun setiap file audio dalam dataset ucapan telah diproses menjadi berdurasi dari beberapa detik hingga maksimal 20 detik. Konversi buku audio menjadi dataset ucapan ini menggunakan perangkat lunak penyelarasan paksa (*forced alignment*) yang dikembangkan oleh pihak pembuat dataset. Perangkat lunak ini mendukung multibahasa, termasuk bahasa-bahasa dengan sumber daya terbatas, seperti Aceh, Bali, atau Minangkabau, dan dapat digunakan untuk bahasa lain tanpa pekerjaan tambahan untuk melatih model. Dataset saat ini secara keseluruhan terdiri dari 8 jam rekaman dalam 7 bahasa dari Indonesia, dan akan ditambahkan lebih banyak bahasa atau file audio seiring dengan pengumpulannya.

Penelitian ini menggunakan dataset publik LibriVox Indonesia versi 1.0 tersebut sebagai sumber data. Secara spesifik, dataset bahasa Minangkabau yang digunakan berisi rekaman pembacaan *Universal Declaration of Human Rights* (Pernyataan Umum tentang Hak-Hak Asasi Manusia) dalam bahasa Minangkabau. Dataset ini termasuk kategori *extremely low-resource* dengan total durasi sekitar 1.3 jam audio, terdiri dari 156 file audio yang sudah tersegmentasi (*pre-segmented*) dengan durasi rata-rata 30 detik per segmen.

Penting untuk dicatat: Dataset ini diterima dalam kondisi sudah tersegmentasi oleh *maintainer* LibriVox Indonesia, yang telah membagi rekaman pembacaan panjang menjadi klip-klip pendek berdasarkan jeda natural dalam pembacaan. Proses segmentasi ini dilakukan oleh *maintainer*, bukan sebagai bagian dari *preprocessing* penelitian ini. Sebagian besar segmen memiliki durasi yang mendekati batas maksimal input Whisper (30 detik), dengan distribusi durasi berkisar antara 15-35 detik dan median sekitar 28-30 detik.

Pemilihan dataset LibriVox Indonesia sebagai sumber data didasarkan pada beberapa pertimbangan penting. Pertama, LibriVox menyediakan audio dengan kualitas rekaman yang relatif baik karena mengikuti standar produksi *audiobook* dengan *sampling rate* 44.1 kHz. Kedua, setiap audio dilengkapi dengan transkrip teks yang akurat karena dibacakan dari naskah tertulis (teks UDHR), sehingga mengurangi beban anotasi manual dan menjamin akurasi transkrip. Ketiga, dataset ini tersedia secara publik dengan lisensi *Public Domain* (CC-0), memungkinkan penggunaan bebas untuk penelitian akademis. Namun, penting untuk dicatat bahwa dataset terbatas pada domain pembacaan formal (*formal reading*) dari teks legal/deklarasi, sehingga performa model pada percakapan spontan atau konteks informal mungkin berbeda [9].

Proses pengumpulan dataset dirancang secara sistematis dengan menggabungkan seluruh data yang tersedia dari folder *train* dan *test* yang disediakan oleh *maintainer* Hugging Face. Tahap awal dimulai dengan mengunduh dataset bahasa Minangkabau dari repositori LibriVox Indonesia melalui platform Hugging Face Datasets. Setelah dataset diunduh, dilakukan proses verifikasi kualitas untuk memastikan bahwa setiap segmen audio memiliki kejernihan ucapan yang baik (*clear speech*), minim *background noise*, dan tidak mengandung distorsi audio yang signifikan. Dataset LibriVox Indonesia telah melalui proses segmentasi otomatis yang membagi rekaman panjang menjadi klip-klip pendek dengan durasi beberapa detik hingga maksimal 30 detik, yang ideal untuk pelatihan model ASR [1].

Proses pengumpulan dan persiapan dataset mencakup langkah-langkah berikut:

1. Mengunduh dataset LibriVox Indonesia 1.0 dari Hugging Face Datasets, dengan fokus pada subset bahasa Minangkabau yang berisi pembacaan UDHR
2. Menggabungkan seluruh audio dari folder *train* dan *test* yang disediakan *maintainer* untuk memaksimalkan penggunaan data dalam skema *cross-validation*
3. Verifikasi kualitas audio secara teknis untuk setiap segmen: *clear speech*, minimal *background noise*, tidak ada distorsi atau *clipping*
4. Validasi transkrip teks yang sesuai dengan setiap segmen audio, memastikan akurasi dan kelengkapan transkrip berdasarkan teks UDHR dalam bahasa Minangkabau
5. Ekstraksi metadata *speaker (reader ID)* untuk dokumentasi keberagaman pembicara dalam dataset
6. Inventarisasi total durasi audio yang tersedia (156 file @ 30s = 1.3 jam total)

Dataset yang berhasil dikumpulkan mencakup rekaman pembacaan UDHR dalam bahasa Minangkabau dengan total 1.3 jam audio (156 file audio). Meskipun terbatas pada satu jenis teks (dokumen deklarasi HAM) dan durasi yang sangat kecil, dataset ini tetap merepresentasikan bahasa Minangkabau formal dan standar. Karakteristik *extremely low-resource* ini membuat penelitian ini mengadopsi pendekatan 5-Fold Cross-Validation untuk memaksimalkan penggunaan data dan memperoleh evaluasi yang lebih robust, dengan strategi *transfer learning* yang sangat efisien dari model *pre-trained* Whisper.

3.2.4 Preprocessing Dataset

Dataset mentah yang telah dikumpulkan akan melalui tahapan *preprocessing* yang sistematis dan komprehensif untuk memastikan

kompatibilitas dengan persyaratan teknis model Whisper serta mengoptimalkan kualitas data yang akan digunakan dalam proses *fine-tuning*. *Preprocessing* merupakan tahap kritis dalam penelitian ini karena kualitas dan konsistensi data input secara langsung mempengaruhi kinerja model ASR yang dilatih. Model Whisper memiliki spesifikasi teknis tertentu, terutama terkait format audio input, yang harus dipenuhi agar model dapat berfungsi secara optimal [1]. Selain itu, *preprocessing* juga bertujuan untuk standarisasi data sehingga mengurangi variabilitas yang tidak relevan dan memungkinkan model fokus pada pembelajaran pola linguistik yang penting [43].

Proses *preprocessing* dirancang dalam tiga tahap utama yang saling melengkapi: *audio preprocessing*, *text preprocessing*, dan *dataset splitting*. Setiap tahap memiliki tujuan spesifik dan menggunakan teknik-teknik yang telah terbukti efektif dalam penelitian ASR sebelumnya [26]. Pendekatan sistematis ini memastikan bahwa dataset yang dihasilkan memiliki kualitas yang konsisten, kompatibel dengan arsitektur Whisper, dan siap digunakan untuk proses *training*, *validation*, dan *testing* [49].

3.2.4.1 Audio Preprocessing

Tahap *audio preprocessing* berfokus pada transformasi sinyal audio mentah menjadi format yang sesuai dengan spesifikasi teknis Whisper dan mengoptimalkan kualitas sinyal untuk pembelajaran model. Proses ini mencakup beberapa operasi penting:

1. **Format Conversion:** Konversi format audio dari MP3 (format asli LibriVox Indonesia) ke WAV (*Waveform Audio File Format*). Konversi ini diperlukan karena format MP3 yang terkompresi dengan *lossy compression* memerlukan proses *decoding* tambahan yang dapat membebani komputasi Whisper. Format WAV yang *uncompressed* memungkinkan akses langsung ke data audio mentah (*raw audio samples*), sehingga mengurangi beban komputasi dan mempercepat proses *preprocessing* serta *training*

[43]

2. **Resampling:** Konversi *sampling rate* audio dari 44.1 kHz (format asli LibriVox Indonesia) ke 16 kHz, yang merupakan spesifikasi standar yang digunakan oleh model Whisper. Konversi ini penting karena model Whisper dilatih dengan data audio pada frekuensi 16 kHz dan akan menghasilkan performa optimal pada *sampling rate* yang sama [1]
3. **Duration Filtering and Validation:** Dataset LibriVox Indonesia telah menyediakan audio yang sudah tersegmentasi dengan durasi rata-rata 30 detik per segmen. Proses *preprocessing* tidak melakukan pemotongan ulang (*splitting*) terhadap audio, melainkan melakukan validasi durasi dan filtering untuk memastikan semua audio memenuhi batasan teknis Whisper:
 - **Maximum duration:** Whisper memiliki batasan maksimal 30 detik untuk input audio. Audio yang melebihi durasi ini (jika ada) akan dipotong (*truncated*) pada detik ke-30 atau dibuang jika informasi penting terpotong [1]
 - **Minimum duration:** Audio yang terlalu pendek (<2 detik) akan difilter karena kemungkinan tidak mengandung informasi linguistik yang cukup untuk pembelajaran
 - **No padding:** Audio pendek (2-20 detik) tidak di-*pad* secara artifisial, karena Whisper dapat menangani audio dengan panjang variabel melalui mekanisme *attention masking* [1], [32]. Selama *training*, batching diimplementasikan menggunakan *data collator* khusus yang secara dinamis mem-*pad* audio dalam *batch* ke panjang maksimal dalam *batch* tersebut (bukan panjang maksimal global), sehingga setiap *batch* dapat diproses oleh GPU secara efisien meski panjang audio berbeda-beda antar *batch* [50]. Pendekatan *dynamic padding per-batch* ini lebih efisien secara memori dibandingkan *global padding* yang akan membuang banyak komputasi pada bagian

yang di-*pad*. Dengan validasi ini, dataset final yang digunakan untuk pelatihan adalah subset dari 156 file audio asli yang memenuhi kriteria durasi, memastikan kompatibilitas dengan batasan teknis Whisper tanpa kehilangan informasi linguistik yang signifikan

4. **Normalization:** Normalisasi *amplitude* audio untuk memastikan konsistensi volume di seluruh dataset, menghindari bias model terhadap audio dengan volume tertentu
5. **Silence Removal:** Penghapusan periode *silence* yang tidak informatif di awal dan akhir setiap *clip*, sehingga model tidak perlu memproses bagian audio yang tidak mengandung informasi linguistik
6. **Quality Check:** Penyaringan *clips* dengan *Signal-to-Noise Ratio* (SNR) rendah atau mengandung distorsi signifikan, untuk memastikan hanya data berkualitas tinggi yang digunakan dalam *training*
7. **Augmentasi Data:** Karena rekaman suara bahasa Minang yang tersedia sangat sedikit (hanya sekitar 1,3 jam), kami menggunakan trik sulap digital. Suara tersebut kami percepat, perlambat, ubah nada dasar suaranya, atau kami beri suara bising latar belakang. Hal ini seolah-olah memaksa mesin untuk mendengarkan orang Minang berbicara di pasar yang ramai atau mendengarkan berbagai macam karakter suara manusia. Efeknya, mesin belajar menjadi jauh lebih tangguh dan tidak kaget saat mendengar suara baru di dunia nyata.

Teknik ini hanya diterapkan pada data pelatihan, dan tidak pernah diterapkan pada data tes.

Potongan Kode 3.1 menampilkan cuplikan dari kelas pipeline augmentasi menggunakan pustaka `torchaudio` yang diimplementasikan pada file `augmentation.py`.

Kode 3.1 Implementasi Kelas Pemrosesan Data Augmentation

```
1 import torch
2 import torchaudio.transforms as T
```

```

3 import random
4
5 class SpecAugment:
6     """Apply SpecAugment to mel spectrogram."""
7
8     def __init__(
9         self,
10         time_mask_param: int = None,
11         freq_mask_param: int = None
12     ):
13         """
14         Args:
15             time_mask_param: Maximum time mask length (T parameter)
16             freq_mask_param: Maximum frequency mask length (F
17             parameter)
18             """
19         self.time_mask_param = time_mask_param or
20         AUGMENTATION_CONFIG["specaugment_time_mask"]
21         self.freq_mask_param = freq_mask_param or
22         AUGMENTATION_CONFIG["specaugment_freq_mask"]
23
24         self.time_masking =
25         T.TimeMasking(time_mask_param=self.time_mask_param)
26         self.freq_masking =
27         T.FrequencyMasking(freq_mask_param=self.freq_mask_param)
28
29     def __call__(self, mel_spectrogram: torch.Tensor) ->
30         torch.Tensor:
31         """Apply time and frequency masking to mel spectrogram."""
32         # Apply time masking
33         augmented = self.time_masking(mel_spectrogram)
34
35         # Apply frequency masking
36         augmented = self.freq_masking(augmented)
37
38         return augmented

```

3.2.4.2 Text Preprocessing

Tahap *text preprocessing* bertujuan untuk menstandarkan transkrip teks dan menghilangkan variasi yang tidak relevan untuk tugas ASR. Standarisasi teks penting untuk memastikan konsistensi dalam pelatihan model dan evaluasi hasil. Proses ini mencakup:

1. **Lowercase Conversion:** Konversi semua teks ke huruf kecil untuk mengurangi ukuran *vocabulary* dan menghilangkan perbedaan yang tidak relevan antara kapitalisasi [1]
2. **Punctuation Normalization:** Standarisasi tanda baca untuk konsistensi format, termasuk normalisasi tanda kutip, tanda hubung, dan simbol-simbol khusus [1]
3. **Number Handling:** Dataset UDHR bahasa Minangkabau tidak mengandung angka numerik (0-9) dalam transkrip, sehingga tidak

diperlukan proses *number-to-word normalization*. Transkrip asli sudah bersih dan sesuai dengan audio yang direkam

4. **Whitespace Normalization:** Pembersihan spasi berlebih, tab, dan karakter *whitespace* lainnya untuk memastikan format teks yang konsisten
5. **Special Character Handling:** Penanganan khusus untuk karakter-karakter yang spesifik dalam bahasa Minangkabau, termasuk diakritik atau simbol yang mungkin muncul dalam transkrip [9], [25]

3.2.4.3 5-Fold Cross-Validation

Untuk memastikan AI ini tidak sekadar menghafal rekaman yang sudah ada, kami mengujinya dengan metode layaknya ”soal ujian yang diacak berulang kali”. Seluruh rekaman suara dibagi menjadi 5 kelompok ujian (*5-Fold Cross-Validation*).

Proses belajarnya berlangsung secara bergilir:

1. AI belajar menggunakan 4 kelompok soal, lalu diuji kemampuannya pada 1 kelompok soal yang sengaja belum pernah ia dengar sama sekali.
2. Proses ini diulang terus sebanyak 5 kali dengan memutar-mutar kelompok mana yang dijadikan bahan belajar dan mana yang dijadikan soal ujian.

Melalui pendekatan pengujian silang ini, konsistensi performa pada seluruh *fold* menjadi indikator objektif bahwa model berhasil menggeneralisasi pemahamannya. Model tersebut terbukti mempelajari pola akustik bahasa Minangkabau asli secara mendalam, bukan sekadar menghafal sampel data pelatihan yang kuantitasnya sangat terbatas (*overfitting*) [49].

Selain itu, prosedur evaluasi selama fase pelatihan didukung oleh mekanisme penghentian otomatis (*early stopping*). Siklus pelatihan model akan dihentikan secara prematur apabila performa pada data validasi tidak lagi menunjukkan perbaikan yang signifikan. Mekanisme ini mencegah model agar tidak hanya menghafal data (menghindari *overfitting*) dan menghemat sumber daya komputasi.

3.2.5 Konfigurasi Model Whisper Base

Model Whisper Base dipilih sebagai arsitektur *base model* dalam penelitian ini berdasarkan pertimbangan yang cermat terhadap *trade-off* antara performa, efisiensi komputasi, dan kelayakan untuk skenario *extremely low-resource* [1]. Whisper Base, dengan 74 juta parameter, merupakan varian yang optimal untuk dataset 1.3 jam: cukup kompleks untuk menangkap pola bahasa, namun tidak terlalu besar sehingga risiko overfitting dapat diminimalkan dibandingkan varian Small (244M) atau Medium (769M). Pemilihan ini juga didukung oleh studi-studi sebelumnya yang menunjukkan bahwa Whisper Base dapat mencapai performa yang kompetitif pada berbagai tugas ASR untuk skenario *low-resource*, terutama setelah di-*fine-tune* pada bahasa target [2], [3].

Arsitektur Whisper Base menggunakan *encoder-decoder Transformer* [1], [32] dengan konfigurasi yang telah dioptimalkan untuk tugas *speech recognition*. Model ini telah di-*pre-train* pada dataset multibahasa yang sangat besar (680,000 jam audio) sehingga memiliki representasi yang kuat tentang pola akustik dan linguistik lintas bahasa [1]. Karakteristik *pre-trained* ini memungkinkan model untuk di-*transfer* dengan efisien ke bahasa Minangkabau meskipun dengan data *fine-tuning* yang relatif terbatas (1.3 jam). Proses konfigurasi model dilakukan secara sistematis untuk memastikan semua komponen berfungsi dengan benar dan siap untuk tahap *fine-tuning*.

Tahap konfigurasi model Whisper Base mencakup beberapa langkah teknis penting:

1. *Loading pre-trained* Whisper Base (74M *parameters*) dari Hugging Face Model Hub menggunakan *library* Transformers, memastikan model dalam keadaan *pre-trained* yang telah terbukti efektif
2. Verifikasi arsitektur model: 6 *encoder layers* dan 6 *decoder layers*, dengan *hidden dimension (width)* 512 dan 8 *attention heads per layer*, sesuai dengan spesifikasi resmi Whisper Base

3. Inisialisasi *tokenizer* dengan *vocabulary* sebesar 51,864 *tokens*, yang mencakup representasi karakter dan subword untuk berbagai bahasa termasuk kemampuan untuk mengakomodasi bahasa baru
4. Konfigurasi *feature extractor* untuk menghasilkan representasi *log-Mel spectrogram* dengan 80 dimensi, menggunakan *window size* 25ms dan *hop length* 10ms, yang merupakan parameter standar untuk ekstraksi fitur audio dalam Whisper
5. *Setup device* (GPU/CPU) untuk *training*, dengan preferensi GPU (NVIDIA GeForce RTX 5060 Ti dengan 16 GB VRAM) untuk mempercepat komputasi matriks dalam proses *fine-tuning*

3.2.6 Implementasi Fine-Tuning

Penelitian ini menerapkan pendekatan komparatif eksperimental dengan membandingkan dua strategi *fine-tuning* yang berbeda: Freeze Encoder (Decoder-Only Fine-Tuning) sebagai pendekatan standar dan Low-Rank Adaptation (LoRA) sebagai pendekatan *parameter-efficient*. Kedua strategi ini dipilih untuk mengevaluasi *trade-off* antara jumlah parameter yang dilatih dan performa pada skenario *extremely low-resource* [2].

Freeze Encoder melatih seluruh *decoder* (37 juta parameter, sekitar 50% dari total model) sambil membekukan seluruh *encoder*. Pendekatan ini mempertahankan representasi akustik universal yang telah dipelajari *encoder* dari 680,000 jam data *pre-training* [1], sementara mengadaptasi *decoder* untuk pola linguistik bahasa Target [3].

Sebaliknya, LoRA merupakan teknik *Parameter-Efficient Fine-Tuning* (PEFT) yang hanya melatih $\sim 1,47\%$ dari total parameter (sekitar 1,08 juta parameter), menawarkan efisiensi komputasi yang maksimal dan regularisasi implisit melalui pembatasan kapasitas model [42]. Dengan membatasi jumlah parameter yang dilatih, LoRA mengurangi risiko *overfitting* pada dataset sangat terbatas [4].

Pendekatan komparatif ini memungkinkan analisis mendalam terhadap *trade-off* antara akurasi model dan efisiensi sumber daya. Dalam konteks bahasa Minangkabau dengan dataset *extremely low-resource* (1.3 jam), pertanyaan penting yang perlu dijawab adalah: apakah efisiensi LoRA (dengan hanya melatih $\sim 1,47\%$ parameter) memberikan hasil yang sebanding dengan *Freeze Encoder* (yang melatih 50% parameter, yaitu seluruh *decoder*)? Untuk menjawab pertanyaan ini, implementasi kedua metode dirancang dengan cermat melalui proses eksplorasi *hyperparameter* secara iteratif. Masing-masing strategi diuji dalam sepuluh kali percobaan (*run*) dengan variasi *hyperparameter* kunci untuk menemukan konfigurasi optimal. Subbab berikut menjelaskan kerangka konfigurasi dan ruang pencarian *hyperparameter* untuk setiap strategi, sedangkan konfigurasi spesifik dan hasil dari setiap percobaan dibahas secara rinci pada Bab IV.

3.2.6.1 Freeze Encoder Fine-Tuning

Pada metode *Freeze Encoder (Decoder-Only Fine-Tuning)*, seluruh parameter *encoder* Whisper Base (yang berfungsi memproses input audio) dibekukan (*frozen*), dan hanya parameter *decoder* (yang menghasilkan teks transkrip) yang di-*update* selama proses *training*. Pendekatan ini merupakan strategi standar untuk skenario *low-resource* karena mempertahankan representasi akustik universal yang telah dipelajari *encoder* dari 680,000 jam data *pre-training*, sementara hanya mengadaptasi *decoder* untuk pola linguistik bahasa Minangkabau. Dengan hanya melatih *decoder* (~ 37 juta dari 74 juta parameter total), pendekatan ini lebih efisien dibandingkan melatih seluruh model (*full fine-tuning*). Implementasi pembekuan *encoder* diilustrasikan pada Potongan Kode 3.2.

Kode 3.2 Implementasi Fungsi Pembekuan Parameter Encoder (*Freeze Encoder*)

```
1 def freeze_encoder(model: WhisperForConditionalGeneration) ->
2     WhisperForConditionalGeneration:
3     """
4     Freeze the encoder weights; only train the decoder.
```



```

4      Args:
5          model: Whisper model
6
7      Returns:
8          """ Model with frozen encoder
9
10     # Freeze encoder
11     model.model.encoder.requires_grad_(False)
12
13     # Ensure decoder is trainable
14     model.model.decoder.requires_grad_(True)
15
16     # Count trainable parameters
17     total_params = sum(p.numel() for p in model.parameters())
18     trainable_params = sum(p.numel() for p in model.parameters() if
19                             p.requires_grad)
20     print(f"Total parameters: {total_params:,}")
21     print(f"Trainable parameters: {trainable_params:,}")
22     print(f"Percentage trainable: {(trainable_params / total_params)
23         * 100:.2f}%")
24
25     return model

```

Komponen arsitektural dan parameter *training* yang bersifat tetap (tidak divariasikan) pada seluruh percobaan *Freeze Encoder* adalah:

1. **Frozen Components:** Seluruh 6 lapisan *encoder* Transformer dan *feature extractor* (konvolusi) dibekukan
2. **Trainable Components:** Seluruh 6 lapisan *decoder* Transformer (~37M parameter, 50% dari total)
3. **Optimizer:** AdamW (*Adam with Weight Decay*) [47] dengan *cosine annealing scheduler*
4. **Effective Batch Size:** Divariasikan antar percobaan; lihat Tabel 3.1
5. **Max Steps:** Divariasikan antar percobaan; lihat Tabel 3.1
6. **Mixed Precision:** BF16 (*Brain Floating Point 16-bit*) untuk efisiensi komputasi [51]
7. **Dropout:** *Standard dropout* 0,3; *attention dropout* 0,2; *activation dropout* 0,2 (agresif untuk mencegah *overfitting* pada 37M parameter trainable) [44]
8. **Early Stopping:** *Patience* 8 *evaluation steps* dengan evaluasi setiap 4 *training steps*
9. **Best Model Selection:** Berdasarkan *lowest evaluation WER*

10. **Data Augmentation** (diterapkan hanya pada training set):

- *Speed Perturbation*: Faktor {0.85, 0.9, 0.95, 1.0, 1.05, 1.1, 1.15}
- *Noise Injection*: SNR 10–30 dB
- *SpecAugment*: Time masking (80 frames), Frequency masking (40 bins)
- *Pitch Shifting*: ± 2 semitones

11. **Max Length**: 225 tokens pada saat generasi

12. **Loss Function**: *Cross-entropy loss* standar

Sementara itu, beberapa *hyperparameter* kunci divariasikan antar percobaan untuk menemukan konfigurasi optimal. Tabel 3.1 merangkum ruang pencarian *hyperparameter* yang dieksplorasi.

Tabel 3.1 Ruang Pencarian Hyperparameter — Freeze Encoder

Hyperparameter	Nilai yang Diuji	Justifikasi
Learning Rate	5×10^{-6} , 1×10^{-5} , 2×10^{-5}	Rentang konservatif–moderat untuk <i>fine-tuning</i> decoder
Effective Batch Size	16, 32, 64	Pengaruh frekuensi pembaruan bobot vs stabilitas estimasi gradien
Max Steps	60, 100, 120, 200, 400	Batas <i>training</i> ; diperbesar seiring eksplorasi
Weight Decay	0,01; 0,05; 0,1; 0,2	Kekuatan regularisasi L2 pada parameter decoder
Beam Width	1 (greedy), 5	Strategi dekoding: greedy vs <i>beam search</i>

Total dilakukan sepuluh percobaan dengan kombinasi *hyperparameter* yang berbeda, dilaksanakan dalam tiga fase: Fase 1 (FE-1 hingga FE-6, Januari–Februari 2026) berfokus pada eksplorasi *learning rate*, *weight decay*, augmentasi, dan strategi dekoding; Fase 2 (FE-7 hingga FE-9, 3 Maret 2026) memperkenalkan *Weighted Cross-Entropy* dikombinasikan dengan optimasi lebih lanjut; Fase 3 (FE-10, 5 Maret 2026) menambahkan *Temporal Separation* pada WCE, di mana bobot token dihitung hanya dari riwayat percobaan sebelumnya untuk menghindari kebocoran informasi validasi. Konfigurasi spesifik dan hasil dari setiap percobaan dijabarkan pada Bab IV (Tabel 4.1 dan Tabel 4.2).

Implementasi modifikasi fungsi *loss* menjadi *Weighted Cross-Entropy* beserta *Label Smoothing* menggunakan kerangka kelas bawaan dari pustaka *transformers* ditunjukkan pada Potongan Kode 3.3.

Kode 3.3 Implementasi Modifikasi *Weighted Cross-Entropy Loss*

```

1  class WeightedCEMixin:
2      """
3      Mixin that overrides compute_loss() to use weighted
4      cross-entropy.
5      Supports label_smoothing_factor from training args.
6      """
7
8      def __init__(self, *args, token_weights: Optional[torch.Tensor]
9          = None, **kwargs):
10         self._token_weights = token_weights
11         super().__init__(*args, **kwargs)
12
13     def compute_loss(self, model, inputs, num_items_in_batch=None,
14         **kwargs):
15         # If no token weights, fall back to parent (standard CE)
16         if self._token_weights is None:
17             return super().compute_loss(model, inputs,
18                 num_items_in_batch=num_items_in_batch, **kwargs)
19
20         # --- Weighted CE path ---
21         labels = inputs.pop("labels")
22         outputs = model(**inputs)
23         logits = outputs.logits # (batch, seq_len, vocab_size)
24
25         # Move weights to same device as logits, pad if vocab size
26         # changed
27         weights = self._token_weights.to(logits.device)
28         model_vocab = logits.size(-1)
29         if weights.size(0) < model_vocab:
30             pad = torch.full((model_vocab - weights.size(0),),
31                 weights[0].item(), device=weights.device, dtype=weights.dtype)
32             weights = torch.cat([weights, pad])
33
34         log_probs = F.log_softmax(logits, dim=-1)
35         loss = F.nll_loss(
36             log_probs.view(-1, model_vocab),
37             labels.view(-1),
38             weight=weights,

```

```

33         ignore_index=-100,
34     )
35     return (loss, outputs) if kwargs.get("return_outputs") else
        loss

```

3.2.6.2 LoRA Fine-Tuning

Metode LoRA (*Low-Rank Adaptation*) bekerja dengan menambahkan matriks *low-rank* sebagai parameter yang dapat dilatih pada seluruh *linear layer* di setiap blok Transformer, sementara seluruh parameter *pre-trained* dari Whisper Base tetap *frozen*. Pendekatan ini diperkenalkan oleh Hu et al. [42] dan telah terbukti efektif untuk adaptasi model besar dengan dataset terbatas [5], [52]. Bagat dan Kumar [6] menunjukkan bahwa LoRA dapat dikombinasikan dengan arsitektur *mixture of experts* untuk meningkatkan adaptasi multi-domain.

Komponen arsitektural dan parameter *training* yang bersifat tetap pada seluruh percobaan LoRA adalah:

1. **Target Modules:** Seluruh 6 *linear layer* pada setiap blok Transformer (q_proj, v_proj, k_proj, out_proj, fc1, fc2), mencakup *attention projections* dan *feed-forward network* pada encoder maupun decoder
2. **Scaling:** Rasio α/r dijaga konstan pada $2,0\times$ untuk seluruh percobaan
3. **Bias:** None (tidak melatih parameter *bias*)
4. **Optimizer:** AdamW [47] dengan *cosine annealing scheduler*
5. **Effective Batch Size:** Divariasikan antar percobaan; lihat Tabel 3.2
6. **Mixed Precision:** BF16 untuk efisiensi komputasi [51]
7. **Early Stopping:** *Patience* 8 *evaluation steps* dengan evaluasi setiap 4 *training steps*
8. **Best Model Selection:** Berdasarkan *lowest evaluation WER*
9. **Beam Width:** 5 (*beam search*) dengan max length 225 tokens
10. **Data Augmentation:** Identik dengan *Freeze Encoder* (diterapkan hanya pada *training set*)

Berbeda dengan *Freeze Encoder*, LoRA memiliki *hyperparameter* tambahan yang spesifik untuk konfigurasi *adapter*, sehingga ruang pencarian

lebih luas. Tabel 3.2 merangkum *hyperparameter* yang divariasikan.

Tabel 3.2 Ruang Pencarian Hyperparameter — LoRA

Hyperparameter	Nilai yang Diuji	Justifikasi
Rank (r)	8, 16	Kapasitas adaptasi vs risiko <i>overfitting</i>
Alpha (α)	16, 32	Disesuaikan dengan rank (rasio $\alpha/r = 2,0$)
LoRA Dropout	0,1; 0,15	Regularisasi pada <i>adapter</i>
Learning Rate	3×10^{-4} – $1,4 \times 10^{-3}$	Lebih tinggi dari FE karena parameter trainable sedikit
Effective Batch Size	16, 32, 64	Pengaruh frekuensi pembaruan bobot vs stabilitas gradien
Weight Decay	0,01; 0,03; 0,05	Kekuatan regularisasi L2
Label Smoothing	0,05; 0,1	Kalibrasi probabilistik dan anti- <i>overconfidence</i>
Max Steps	100, 150, 200, 250, 400	Batas <i>training</i>

Dengan variasi konfigurasi *rank* (8 atau 16), LoRA melatih $\sim 1,47\%$ – $2,93\%$ dari total parameter ($\sim 1,08$ – $2,16$ juta dari 74 juta parameter), yang secara drastis lebih efisien dibandingkan *Freeze Encoder* (50%). Total dilakukan sepuluh percobaan dengan variasi *hyperparameter* yang berbeda, dilaksanakan dalam tiga fase: Fase 1 (L-1 hingga L-6, Februari 2026) berfokus pada eksplorasi *rank*, *learning rate*, augmentasi, dan regularisasi; Fase 2 (L-7 hingga L-9, 3 Maret 2026) memperkenalkan *Weighted Cross-Entropy*, termasuk satu percobaan dengan *rank* yang lebih tinggi ($r=16$, $\sim 2,93\%$ parameter trainable); Fase 3 (L-10, 5 Maret 2026) menambahkan *Temporal Separation* pada WCE, di mana bobot token dihitung hanya dari riwayat percobaan sebelumnya untuk menghindari kebocoran informasi validasi. Konfigurasi spesifik dan hasil dari

setiap percobaan dijabarkan pada Bab IV (Tabel 4.6 dan Tabel 4.7).

Potongan Kode 3.4 menunjukkan implementasi dari konfigurasi LoRA menggunakan pustaka PEFT (*Parameter-Efficient Fine-Tuning*) dari Hugging Face yang digunakan dalam pengembangan model ini.

Kode 3.4 Implementasi Definisi Parameter LoRA Menggunakan Pustaka PEFT

```

1  from peft import LoraConfig, get_peft_model
2
3  # Definisi parameter LoRA (dari LORA_CONFIG)
4  lora_config = LoraConfig(
5      r=16,                                     # Rank dimensi matriks
6      lora_alpha=32,                             # Faktor skala (alpha/r = 2x
7      scaling)                                   # Probabilitas dropout
8      lora_dropout=0.15,                         # All attention + FFN linear
9      target_modules=[                           layers
10         "q_proj", "v_proj", "k_proj",
11         "out_proj", "fc1", "fc2"
12     ],
13     bias="none",                               # Don't train bias parameters
14     task_type=None,
15     modules_to_save=[]
16 )
17
18 # Integrasi LoRA pada model Whisper pre-trained
19 model = get_peft_model(model, lora_config)
20 model.print_trainable_parameters()

```

3.2.6.3 Regularization

Selain *data augmentation*, beberapa teknik *regularization* diterapkan untuk mencegah *overfitting* dan meningkatkan generalisasi model [48]. Strategi regularisasi dirancang berbeda untuk kedua metode, disesuaikan dengan jumlah parameter yang dilatih:

1. *Dropout* pada berbagai level untuk mengurangi *co-adaptation* antar *neurons* [44]. *Freeze Encoder* menggunakan *dropout* yang lebih agresif (hingga 0,3) karena melatih 37M parameter pada dataset kecil, sedangkan LoRA menggunakan *dropout* yang lebih ringan (0,05–0,15) karena *low-rank constraint* sudah berfungsi sebagai regularisasi struktural. Nilai spesifik *dropout* yang digunakan pada tiap percobaan dirinci pada Bab IV
2. *Weight Decay*: Divariasikan antar percobaan (lihat Tabel 3.1 dan Tabel 3.2) untuk menemukan keseimbangan antara regularisasi L2 dan kapasitas belajar

3. *Label Smoothing*: Diterapkan pada strategi LoRA ($\epsilon = 0,05-0,1$) untuk mencegah *overconfidence* pada prediksi dan meningkatkan kalibrasi model [45]. *Freeze Encoder* menggunakan *cross-entropy* standar tanpa *label smoothing*
4. *Gradient Clipping*: *Max norm* = 1,0 untuk mencegah *exploding gradients* selama *training* [46] (tetap pada kedua strategi)
5. *Early Stopping*: Menghentikan *training* jika tidak ada perbaikan WER selama 8 *evaluation steps* berturut-turut (*patience*=8, evaluasi setiap 4 *training steps*) — tetap pada kedua strategi

Perbedaan mendasar dalam filosofi regularisasi kedua strategi ini—*aggressive explicit regularization* pada *Freeze Encoder* vs *structural regularization* melalui *low-rank constraint* pada LoRA—merupakan salah satu aspek yang dianalisis dalam perbandingan hasil (Bab IV).

3.2.7 Training dan Monitoring

Proses *training* dilakukan dengan protokol *monitoring* yang ketat dan sistematis untuk memastikan konvergensi yang optimal serta mendeteksi potensi masalah sejak dini. *Monitoring* yang komprehensif sangat penting dalam penelitian ini mengingat kompleksitas model Whisper dan sifat dataset yang *low-resource*. Tanpa *monitoring* yang tepat, berbagai masalah seperti *overfitting* [44], konvergensi yang lambat, atau konfigurasi *hyperparameter* yang tidak optimal dapat tidak terdeteksi dan menghasilkan model dengan performa suboptimal. Oleh karena itu, strategi *monitoring* dirancang untuk memberikan visibilitas penuh terhadap seluruh aspek proses *training*, dari metrik performa hingga penggunaan sumber daya komputasi.

Infrastruktur *training* menggunakan perangkat *Local PC* dengan spesifikasi CPU AMD Ryzen 5 7500F dan GPU NVIDIA GeForce RTX 5060 Ti, yang menyediakan performa komputasi yang memadai untuk *fine-tuning* model berukuran menengah seperti Whisper Base. Seluruh proses *training* untuk

kedua strategi (*Freeze Encoder* dan LoRA) dilakukan pada infrastruktur yang sama untuk memastikan perbandingan yang adil. Metrik-metrik kunci dicatat secara otomatis menggunakan *logging framework* yang terintegrasi [50], dan *checkpointing* dilakukan secara berkala untuk menyimpan *state* model terbaik.

Protokol *training* dan *monitoring* yang diterapkan mencakup aspek-aspek berikut:

1. *Training* dilakukan pada *Local PC* dengan akselerasi GPU NVIDIA GeForce RTX 5060 Ti untuk mempercepat komputasi matriks dalam *backpropagation* [53], [54]
2. *Logging* metrik setiap 1 *step*: *training loss*, *learning rate* saat ini, dan waktu komputasi per *batch*, untuk memantau progres *training* secara real-time dengan detail maksimal
3. Evaluasi pada *test set* setiap 4 *steps*: perhitungan WER, CER, dan *validation loss* untuk memantau kemampuan generalisasi model secara *frequent*
4. *Checkpoint* model terbaik berdasarkan *lowest validation WER*, memastikan model yang disimpan adalah yang paling optimal untuk tugas ASR bahasa Minangkabau
5. *Tracking training time* dan penggunaan memori GPU untuk analisis efisiensi komputasi, terutama penting untuk membandingkan *Freeze Encoder* dengan LoRA
6. Visualisasi *learning curves* (*loss*, WER, CER vs *steps*) menggunakan *Weights & Biases* [55] untuk analisis visual terhadap konvergensi model dan deteksi dini *overfitting*
7. *Prediction logging*: Mencatat 5 sampel prediksi setiap kali evaluasi untuk inspeksi kualitatif kualitas transkrip

3.2.8 Evaluasi Model

Model yang telah di-*fine-tune* dievaluasi secara komprehensif menggunakan *test set* yang tidak pernah dilihat oleh model selama proses *training*. Prinsip isolasi *test set* ini sangat penting untuk memastikan bahwa metrik evaluasi yang diperoleh merepresentasikan kemampuan generalisasi model yang sebenarnya pada data baru, bukan hanya kemampuan menghafal pola dalam data *training*. Evaluasi dilakukan dengan protokol yang konsisten untuk kedua strategi *fine-tuning* (*Freeze Encoder* dan LoRA) untuk memungkinkan perbandingan yang adil dan valid melalui skema *5-fold cross-validation*.

Proses evaluasi dirancang untuk mengukur berbagai aspek performa model, tidak hanya akurasi transkrip tetapi juga *error patterns* dan karakteristik kesalahan yang terjadi. Analisis *error patterns* memberikan *insights* penting tentang jenis kesalahan yang paling sering dibuat model (substitusi, penghapusan, atau penyisipan kata), yang dapat menjadi dasar untuk perbaikan model di masa depan.

Protokol evaluasi yang diterapkan mencakup tahapan-tahapan berikut:

1. *Inference* pada *test set* menggunakan algoritma *beam search* [1] dengan *beam size* = 5, yang memberikan keseimbangan antara kualitas transkrip dan kecepatan inferensi
2. Perhitungan *Word Error Rate* (WER) sebagai metrik utama untuk mengukur akurasi transkrip pada level kata
3. Perhitungan *Character Error Rate* (CER) sebagai metrik tambahan yang lebih sensitif terhadap kesalahan fonetik dan memberikan perspektif komplementer terhadap WER
4. Normalisasi teks sebelum evaluasi (konversi ke *lowercase*, penghapusan tanda baca, normalisasi *whitespace*) untuk memastikan konsistensi dalam perhitungan metrik
5. Perhitungan metrik untuk setiap *fold*: WER dan CER pada *test set* dari

masing-masing 5 *folds*

6. Agregasi hasil: Menghitung *mean ± standard deviation* dari WER dan CER across 5 *folds* untuk kedua strategi (*Freeze Encoder* dan LoRA)
7. Analisis *error patterns* detail dengan mengkategorikan kesalahan menjadi: substitusi (kata yang salah diganti), penghapusan (kata yang hilang), dan penyisipan (kata tambahan yang tidak seharusnya ada)
8. Perbandingan performa antara model *Freeze Encoder* dan LoRA menggunakan metrik agregat (mean WER/CER), serta analisis *trade-off* dengan waktu *training* dan penggunaan memori

3.2.8.1 Kategorisasi Error untuk Analisis Kualitatif

Selain evaluasi kuantitatif menggunakan WER dan CER, penelitian ini juga melakukan analisis kualitatif terhadap kesalahan transkripsi dengan mengkategorikan error berdasarkan karakteristik dan penyebabnya. Kategorisasi ini bertujuan untuk memberikan pemahaman mendalam tentang jenis kesalahan yang paling sering terjadi dan konteks di mana model mengalami kesulitan, sehingga dapat memberikan *insights* untuk perbaikan model di masa depan. Error dikategorikan ke dalam empat tipe utama sebagai berikut [15]:

1. **Kesalahan Fonetik (Phonetic Errors):** Kesalahan yang terjadi akibat misrecognition fonem-fonem spesifik bahasa Minangkabau yang berbeda dari bahasa Indonesia standar. Contohnya termasuk kesalahan pengenalan vokal (misalnya, "a" vs "o" dalam konteks tertentu) atau konsonan khas Minangkabau yang tidak ada dalam bahasa Indonesia [9]. Analisis kesalahan fonetik dapat mengungkap fonem-fonem yang paling sulit dikenali oleh model dan memerlukan perhatian khusus dalam *fine-tuning*.
2. **Kesalahan Leksikal (Lexical Errors):** Kesalahan pada level kata, terutama pada kosakata spesifik Minangkabau, kata serapan dari bahasa Indonesia atau bahasa asing yang diucapkan dalam konteks bahasa Minangkabau, serta kata-kata yang memiliki ejaan atau pengucapan

yang ambigu. Kategori ini mencakup kesalahan penggantian kata dengan kata lain yang mirip secara fonetik tetapi berbeda makna, atau kesalahan pada kata-kata yang jarang muncul dalam dataset *training* [25].

3. **Kesalahan Kontekstual (Contextual Errors):** Kesalahan pada kata-kata yang pengenalan yang sebenarnya bergantung pada konteks kalimat. Misalnya, kata yang secara fonetik ambigu tetapi dapat dikenali dengan benar jika model memahami konteks semantik atau sintaksis kalimat. Kesalahan kontekstual mengindikasikan keterbatasan model dalam memahami struktur bahasa atau memanfaatkan informasi konteks untuk disambiguasi [32].
4. **Kesalahan Halusinasi (Hallucination Errors):** Kategori error yang spesifik untuk model Whisper, di mana model menghasilkan teks yang tidak sesuai dengan audio input, terutama selama periode hening atau *silence*. Manifestasi umum termasuk pengulangan frasa secara berulang-ulang (*phrase repetition*) atau munculnya teks acak yang tidak relevan dengan konten audio [34]. Kesalahan halusinasi merupakan karakteristik intrinsik dari model *autoregressive* seperti Whisper yang dilatih pada data skala besar dengan *weak supervision* [1], dan sering terjadi ketika model tidak memiliki informasi audio yang cukup untuk diprediksi tetapi tetap dipaksa menghasilkan output. Monitoring halusinasi penting untuk memahami *robustness* model terhadap variasi kualitas audio dan periode hening dalam dataset Minangkabau.

Kategorisasi error ini akan diterapkan pada sampel hasil transkripsi dari setiap *fold* dalam *cross-validation*, dengan inspeksi manual terhadap kesalahan yang terjadi. Analisis kualitatif berdasarkan kategorisasi ini akan diintegrasikan dengan hasil kuantitatif (WER/CER) untuk memberikan pemahaman komprehensif tentang performa model dan pola kesalahan yang dominan untuk setiap strategi *fine-tuning*.

3.2.9 Analisis Hasil dan Perbandingan

Tahap akhir dari alur penelitian adalah melakukan analisis mendalam terhadap hasil evaluasi dan membandingkan performa antara dua strategi *fine-tuning* yang telah diimplementasikan. Analisis ini tidak hanya berfokus pada perbandingan metrik akurasi (WER dan CER), tetapi juga mempertimbangkan aspek-aspek praktis lainnya seperti efisiensi komputasi, waktu *training*, penggunaan memori, dan kelayakan untuk deployment [4], [42]. Pendekatan analisis komparatif ini bertujuan untuk memberikan rekomendasi yang komprehensif dan berbasis bukti mengenai strategi *fine-tuning* yang paling sesuai untuk adaptasi Whisper pada bahasa Minangkabau dan bahasa *low-resource* lainnya [2], [34].

Analisis dilakukan dengan mengintegrasikan evaluasi kuantitatif (metrik numerik) dan kualitatif (analisis transkrip), memberikan pemahaman yang holistik tentang kekuatan dan kelemahan masing-masing strategi. Evaluasi kuantitatif mencakup perbandingan statistik WER dan CER, sementara evaluasi kualitatif melibatkan inspeksi manual terhadap sampel transkrip untuk mengidentifikasi pola kesalahan yang mungkin tidak tertangkap oleh metrik numerik. Pendekatan ganda ini memastikan bahwa kesimpulan yang ditarik tidak hanya berdasarkan angka-angka tetapi juga didukung oleh pemahaman mendalam tentang perilaku model [15].

Proses analisis hasil dan perbandingan mencakup aspek-aspek berikut:

1. Perbandingan metrik WER dan CER antara *Freeze Encoder* dan LoRA berdasarkan *mean $\pm$ standard deviation* dari 5 *folds*
2. Statistical significance testing: Menerapkan *paired t-test* [56] pada hasil WER/CER dari 5 *folds* untuk menentukan apakah perbedaan antara *Freeze Encoder* dan LoRA secara statistik signifikan (*p-value* < 0.05)
3. Analisis *trade-off* multidimensional: akurasi model vs efisiensi *training* (waktu pelatihan total untuk 5 *folds*, penggunaan memori GPU, jumlah

parameter yang dilatih), untuk mengidentifikasi strategi yang menawarkan keseimbangan terbaik untuk skenario *extremely low-resource*

4. Evaluasi kualitas transkrip secara kualitatif melalui inspeksi manual terhadap sampel transkrip dari berbagai *folds*, mengidentifikasi dan mengkategorikan kesalahan berdasarkan kerangka kategorisasi error yang telah didefinisikan (fonetik, leksikal, kontekstual, dan halusinasi), untuk memahami jenis kesalahan yang paling umum dan konteks di mana setiap metode unggul atau lemah
5. Identifikasi *patterns of errors* yang spesifik (*common misrecognitions*), seperti fonem atau kata Minangkabau yang sering salah dikenali, untuk memberikan *insights* bagi penelitian lanjutan
6. Diskusi tentang *limitations* penelitian (*e.g.*, ukuran dataset yang sangat terbatas, domain terbatas pada pembacaan formal UDHR, computational cost dari 5-fold training) dan *potential improvements* untuk penelitian masa depan
7. Perumusan rekomendasi strategi *fine-tuning* terbaik untuk bahasa Minangkabau dan bahasa *extremely low-resource* lainnya berdasarkan hasil analisis komprehensif dari *cross-validation*

3.3 Alat dan Bahan Tugas Akhir

Penelitian ini memerlukan berbagai alat dan bahan untuk mendukung implementasi dan evaluasi model ASR bahasa Minangkabau.

3.3.1 Alat

Alat-alat yang digunakan dalam penelitian ini meliputi perangkat keras dan perangkat lunak:

3.3.1.1 Perangkat Keras

1. **Laptop:** ASUS VivoBook 14 K413 (11th Gen Intel), digunakan untuk *development*, analisis data, dan penulisan kode

- Processor: Intel Core i3 (11th Generation)
 - RAM: 8 GB DDR4
 - Storage: 512 GB SSD
 - Sistem operasi: Linux/Windows
 - Fungsi: *Development environment*, eksplorasi data, dan analisis hasil
2. **Local PC:** Perangkat komputer pribadi dengan spesifikasi tinggi, digunakan untuk *training* model dan eksperimen keseluruhan
- CPU: AMD Ryzen 5 7500F
 - GPU: NVIDIA GeForce RTX 5060 Ti 16 GB
 - RAM: 16 GB DDR5 5600 MHz
 - Storage: 2 TB SSD
 - Sistem Operasi: CachyOS x86_64 (Linux Kernel 6.18.7)
 - Fungsi: *Training* model *Freeze Encoder* dan LoRA dengan akselerasi GPU

3.3.1.2 Perangkat Lunak

Lingkungan Pengembangan dan Sistem Operasi:

1. **Python 3.10+** [57]: Bahasa pemrograman utama untuk implementasi model dan *data processing*
2. **CachyOS x86_64:** Sistem operasi *Linux* (Kernel 6.18.7) yang digunakan pada Local PC
3. **CUDA 12.8.1** [53]: *NVIDIA CUDA Toolkit* untuk akselerasi GPU pada Local PC
4. **Jupyter Notebook** [58]: *Interactive development environment* untuk eksplorasi data, eksperimen model, dan dokumentasi proses penelitian
5. **Visual Studio Code:** *Code editor* untuk *local development* pada ASUS VivoBook 14 K413
6. **Git/GitHub:** *Version control system* dan repositori kode untuk manajemen

proyek dan kolaborasi

Seluruh *library* Python yang digunakan dalam penelitian ini dikelola melalui file `requirements.txt`. Berikut adalah daftar dependensi utama yang dikelompokkan berdasarkan fungsi.

Framework Deep Learning dan Library Inti:

1. **PyTorch 2.9.1** [54] dan `torchaudio 2.9.1`: *Deep learning framework* untuk *training*, *inference*, dan *audio processing* model Whisper
2. **Transformers 4.57.3** [50]: *Library* Hugging Face untuk memuat dan *fine-tune* model Whisper *pre-trained*
3. **Datasets 4.4.2** [59]: *Library* Hugging Face untuk manajemen dataset, *data loading*, dan *streaming*
4. **Accelerate 1.12.0**: *Library* Hugging Face untuk *distributed training*, optimisasi memori, dan akselerasi GPU
5. **PEFT $\geq 0.13.0$** [42], [60]: *Library* Hugging Face untuk implementasi LoRA dan metode *fine-tuning* efisien parameter
6. **Evaluate 0.4.6**: *Library* Hugging Face untuk evaluasi metrik model secara standar
7. **Tokenizers 0.22.2**: Tokenisasi teks performa tinggi, digunakan oleh Transformers
8. **Safetensors 0.7.0**: Format penyimpanan tensor yang aman dan efisien
9. **Huggingface-hub 0.36.0**: *Client* untuk mengakses Hugging Face Model Hub dan Dataset Hub
10. **Triton 3.5.1**: Compiler untuk optimasi kernel GPU yang digunakan oleh PyTorch

Library Audio Processing dan Augmentation:

1. **librosa 0.11.0** [43]: Analisis audio dan ekstraksi fitur musik/suara
2. **audiomentations 0.43.1**: *Data augmentation* audio (*noise injection*, *pitch*

shifting, time stretching) [19], [20]

3. **soundfile 0.13.1**: Membaca dan menulis file audio dalam berbagai format
4. **soxr 0.5.0.post1**: Resampling audio berkualitas tinggi
5. **audioread 3.1.0**: Membaca file audio dengan dukungan berbagai *backend*

Library Machine Learning dan Evaluasi:

1. **scikit-learn 1.8.0** [61]: Utilitas *machine learning* (*data splitting*, metrik evaluasi, *preprocessing*)
2. **jiwer 4.0.0** [15]: Perhitungan metrik evaluasi ASR (WER dan CER)
3. **numpy 2.3.5** [62]: Komputasi numerik fundamental dengan array multidimensi
4. **scipy 1.17.0** [63]: Komputasi saintifik dan algoritma optimasi
5. **pandas 2.3.3** [64]: Manipulasi dan analisis data terstruktur

Library Visualisasi dan Monitoring:

1. **matplotlib 3.10.8** [65]: Visualisasi data dan plotting grafik
2. **Weights & Biases (wandb) 0.23.1** [55]: Platform untuk *experiment tracking*, visualisasi *learning curves*, dan *logging* metrik
3. **tqdm 4.67.1**: *Progress bar* dan monitoring iterasi

Library Pendukung:

1. **pydantic 2.12.5**: Validasi data menggunakan Python type hints
2. **python-dotenv 1.2.1**: Manajemen variabel environment dari file `.env`
3. **PyYAML 6.0.3**: Parsing dan serialisasi file YAML untuk konfigurasi
4. **pyarrow 22.0.0**: Format data kolumnar Apache Arrow (digunakan oleh Datasets)
5. **requests 2.32.5**: *HTTP client* untuk komunikasi dengan API eksternal
6. **gitpython 3.1.46**: Interaksi programatik dengan repositori Git

NVIDIA CUDA Libraries (untuk akselerasi GPU):

1. **nvidia-cuda-runtime-cu12 12.8.90** [53]: *CUDA runtime library*

2. **nvidia-cudnn-cu12 9.10.2.21** [66]: cuDNN untuk akselerasi operasi *deep learning*
3. **nvidia-cublas-cu12 12.8.4.1**: cuBLAS untuk operasi aljabar linear
4. **nvidia-cufft-cu12 11.3.3.83**: cuFFT untuk *Fast Fourier Transform*
5. **nvidia-nccl-cu12 2.27.5**: NCCL untuk komunikasi multi-GPU

Selain *library* utama di atas, proyek ini juga menggunakan sejumlah dependensi pendukung (*transitive dependencies*) yang secara otomatis dipasang, seperti *numba* (akselerasi *librosa*), *rapidfuzz* (dependensi *jiwer*), *cffi* (dependensi *soundfile*), *multiprocess* (dependensi *Datasets*), serta berbagai *library networking* (*aiohttp*, *httpx*, *urllib3*) dan utilitas (*filelock*, *packaging*, *psutil*). Daftar lengkap seluruh 98 dependensi beserta versi spesifiknya tersedia pada file `requirements.txt` di repositori kode penelitian.

3.3.2 Bahan

Bahan-bahan yang digunakan dalam penelitian ini meliputi dataset dan dokumen referensi:

1. Dataset Audio Bahasa Minangkabau:

- Sumber: LibriVox Indonesia 1.0 via Hugging Face Datasets (<https://huggingface.co/datasets/indonesian-nlp/librivox-indonesia>)
- Konten: Pembacaan *Universal Declaration of Human Rights* dalam bahasa Minangkabau
- Format: Audio *files* MP3 dengan *sampling rate* 44.1 kHz
- Durasi: Total 1.3 jam (156 file audio @ 30 detik per audio)
- Evaluasi: 5-Fold Cross-Validation (setiap fold: 125 train, 31 test)
- Karakteristik: *Extremely low-resource*, domain pembacaan formal (UDHR)
- Lisensi: *Public Domain* (CC-0)

2. Model Pre-trained:

- Whisper Base dari OpenAI (via Hugging Face Model Hub)

- Model ID: openai/whisper-base
- Lisensi: MIT License

3. Dokumen Referensi:

- Paper: "Robust Speech Recognition via Large-Scale Weak Supervision" [1]
- Paper: "LoRA: Low-Rank Adaptation of Large Language Models" [42]
- Paper: "Exploration of Whisper Fine-Tuning Strategies for Low-Resource ASR" [2]
- Dokumentasi Hugging Face Transformers
- Dokumentasi PEFT library

3.4 Metode Pengembangan

Penelitian ini menggunakan pendekatan eksperimental dengan metodologi yang terstruktur untuk memastikan validitas dan reproducibility hasil.

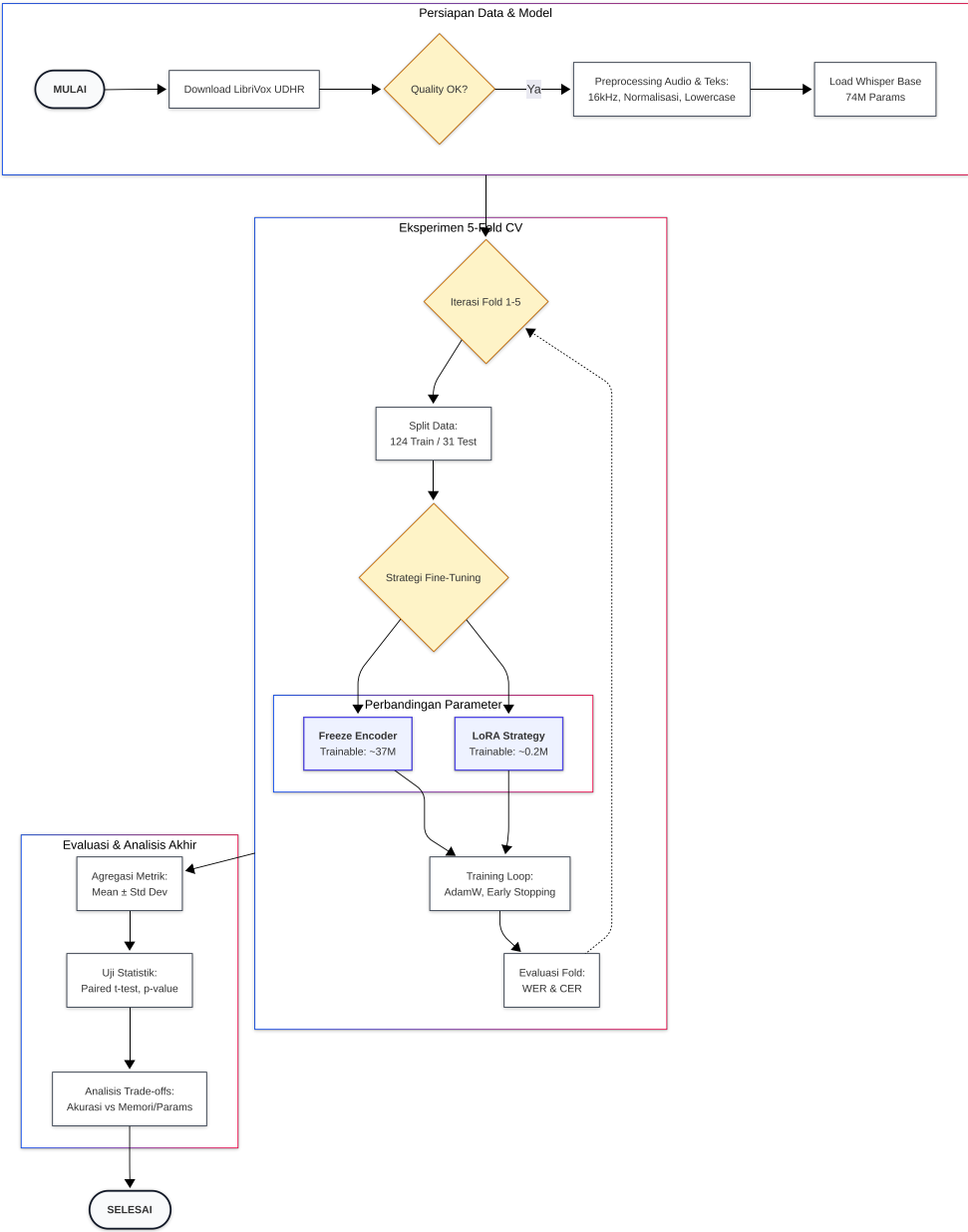
3.4.1 Pendekatan Penelitian

Penelitian ini mengadopsi pendekatan eksperimental dengan karakteristik:

1. **Quantitative Evaluation:** Pengukuran performa menggunakan metrik objektif (WER, CER) dengan agregasi mean $\pm$ std dari 5 folds
2. **Comparative Study:** Perbandingan antara dua strategi fine-tuning (*Freeze Encoder* vs LoRA) pada skenario extremely low-resource
3. **Controlled Experiment:** Dataset, hyperparameters, dan evaluation protocol dijaga konsisten untuk perbandingan yang adil
4. **Reproducible Research:** Semua kode, konfigurasi, dan hasil didokumentasikan dengan lengkap
5. **Statistical Validation:** Paired t-test ($n=5$ folds) untuk menentukan signifikansi perbedaan performa

3.4.2 Alur Pengembangan

Pengembangan sistem ASR bahasa Minangkabau mengikuti alur yang sistematis seperti digambarkan pada Gambar 3.2:



Gambar 3.2 Alur Pengembangan Sistem

3.4.3 Metodologi Fine-Tuning

Metodologi fine-tuning mengikuti best practices untuk extremely low-resource ASR dengan pendekatan 5-Fold Cross-Validation:

3.4.3.1 Freeze Encoder Fine-Tuning Methodology

Strategi ini mempertahankan representasi akustik universal dari encoder sambil mengadaptasi decoder untuk bahasa Minangkabau:

1. Initialize dari pre-trained Whisper Base checkpoint
2. **Freeze Components:** Seluruh encoder layers (6 layers) dan feature extractor tetap frozen
3. **Trainable Components:** Decoder Transformer blocks (6 layers, ~37M parameters)
4. **Batching Strategy:** Menggunakan *data collator* dengan *dynamic padding per-batch* yang mem-*pad* audio ke panjang maksimal dalam setiap *batch*, memungkinkan pemrosesan GPU yang efisien untuk audio dengan durasi bervariasi tanpa *padding* global yang boros memori
5. **5-Fold Iteration:** Untuk setiap fold k ($k=1$ hingga 5):
 - Training set: 4 folds (~124 audio)
 - Test set: 1 fold (~31 audio)
 - **Apply augmentation:** SpecAugment, Speed Perturbation, Noise Injection pada training set only (tidak pada test set)
 - Training dengan supervised learning dari scratch menggunakan augmented training data
 - Monitoring training loss untuk convergence
 - Early stopping berdasarkan WER pada evaluation set (patience=8 evaluation steps)
6. Save best checkpoint per fold berdasarkan lowest training loss
7. Evaluate pada test fold untuk mendapatkan WER_k dan CER_k

3.4.3.2 LoRA Fine-Tuning Methodology

Parameter-Efficient Fine-Tuning dengan hanya melatih low-rank adapters ($\sim 1,47\%$ parameters):

1. Initialize dari pre-trained Whisper Base checkpoint
2. **Freeze Components:** Seluruh base model parameters (74M params)
3. **Inject LoRA Adapters:**
 - Rank $r=8$, Alpha=16
 - Target modules: Seluruh linear layer (q_proj, v_proj, k_proj, out_proj, fc1, fc2)
 - Trainable parameters: $\sim 1,08\text{M}$ ($\sim 1,47\%$ dari total)
4. **Batching Strategy:** Sama dengan Freeze Encoder, menggunakan *dynamic padding per-batch* untuk efisiensi memori GPU
5. **5-Fold Iteration:** Sama seperti Freeze Encoder, untuk setiap fold k:
 - Training pada 4 folds, test pada 1 fold
 - **Apply augmentation:** SpecAugment, Speed Perturbation, Noise Injection pada training set only
 - Higher learning rate ($5e-4$) karena parameter lebih sedikit
 - Dropout 0.1 pada LoRA layers untuk regularisasi
6. Save LoRA adapters per fold (base model tetap frozen)
7. Evaluate pada test fold untuk WER_k dan CER_k

3.4.3.3 Aggregation dan Statistical Testing

Setelah 5 iterasi selesai untuk kedua strategi:

1. Kumpulkan hasil: $\{\text{WER}_1, \text{WER}_2, \dots, \text{WER}_5\}$ untuk Freeze Encoder dan LoRA
2. Hitung mean $\pm$ standard deviation untuk setiap strategi
3. Paired t-test ($n=5$) untuk menguji signifikansi perbedaan: $H_0: \mu_{\text{freeze}} = \mu_{\text{lora}}$
4. Signifikansi ditentukan dengan p-value < 0.05

5. Report final metrics: “Freeze Encoder: WER = X.X% $\pm$ Y.Y%”

3.4.4 Metode Pengujian

Pengujian dilakukan secara sistematis untuk memvalidasi performa model:

3.4.4.1 Functional Testing

1. **Audio Input Testing:** Verifikasi model dapat menerima berbagai format audio
2. **Transcription Testing:** Verifikasi output transkrip dalam format text yang benar
3. **Language Identification:** Verifikasi model mengenali bahasa Minangkabau dengan benar
4. **Robustness Testing:** Test dengan audio berkualitas rendah, background noise

3.4.4.2 Performance Testing

1. **Accuracy Testing:** Evaluasi WER dan CER pada test set
2. **Inference Speed Testing:** Pengukuran Real-Time Factor (RTF)
3. **Memory Usage Testing:** Monitoring GPU/CPU memory consumption
4. **Scalability Testing:** Test dengan varying input lengths

3.4.4.3 Comparative Testing

1. Perbandingan Base Whisper Base vs Fine-tuned models (mean WER/CER across 5 folds)
2. Perbandingan Freeze Encoder vs LoRA (mean $\pm$ std WER/CER dari 5 folds)
3. Statistical significance testing dengan paired t-test (n=5 folds) untuk WER dan CER
4. Analysis of variance (ANOVA) jika diperlukan untuk multiple comparisons

3.5 Ilustrasi Perhitungan Metode

Bagian ini menjelaskan contoh perhitungan untuk metrik evaluasi utama yang digunakan dalam penelitian.

3.5.1 Perhitungan Word Error Rate (WER)

WER mengukur edit distance pada level kata antara reference dan hypothesis transcription.

3.5.1.1 Formula WER

$$\text{WER} = \frac{S + D + I}{N} \times 100\%$$

(Rumus 3.1)

di mana:

- *S* = number of substitutions (kata salah diganti)
- *D* = number of deletions (kata hilang)
- *I* = number of insertions (kata tambahan)
- *N* = total words dalam reference

3.5.1.2 Contoh Perhitungan

Misalkan kita memiliki pasangan reference-hypothesis berikut:

Reference: “*awak ingin pai ka pasar pagi iko*”

Hypothesis: “*awak ingin pergi pasar pagi ini*”

Alignment:

Reference	awak	ingin	pai	ka	pasar	pagi	iko
Hypothesis	awak	ingin	pergi	-	pasar	pagi	ini
Operation	C	C	S	D	C	C	S

Tabel 3.3 Alignment untuk perhitungan WER (C=Correct, S=Substitution, D=Deletion)

Perhitungan:

- Substitutions (*S*): 2 (“pai” → “pergi”, “iko” → “ini”)

- Deletions (D): 1 (“ka” dihapus)
- Insertions (I): 0
- Total words (N): 7

$$\text{WER} = \frac{2 + 1 + 0}{7} \times 100\% = \frac{3}{7} \times 100\% = 42.86\% \quad (\text{Rumus 3.2})$$

3.5.2 Perhitungan Character Error Rate (CER)

CER mengukur edit distance pada level karakter, lebih sensitif terhadap phonetic errors.

3.5.2.1 Formula CER

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c} \times 100\% \quad (\text{Rumus 3.3})$$

di mana:

- S_c = number of character substitutions
- D_c = number of character deletions
- I_c = number of character insertions
- N_c = total characters dalam reference

3.5.2.2 Contoh Perhitungan

Menggunakan contoh yang sama:

Reference: “*awakinginpaikapasarpagiiko*” (tanpa spasi: 26 karakter)

Hypothesis: “*awakinginpergipasarpagiini*” (tanpa spasi: 26 karakter)

Perhitungan:

- Character substitutions (S_c): 6 (“a”→“e”, “i”→“r”, “k”→“g”, “a”→“i”, “k”→“n”, “o”→“i”)
- Character deletions (D_c): 0
- Character insertions (I_c): 0
- Total characters (N_c): 26

$$\text{CER} = \frac{6 + 0 + 0}{26} \times 100\% = \frac{6}{26} \times 100\% = 23.08\% \quad (\text{Rumus 3.4})$$

Perhatikan bahwa CER (23.08%) lebih rendah dari WER (42.86%) karena partial word matches dihargai pada level karakter.

3.5.3 Perhitungan LoRA Parameters

Perhitungan jumlah trainable parameters untuk LoRA adaptation.

3.5.3.1 Formula LoRA Parameters

Untuk setiap attention layer dengan projection weight $W \in \mathbb{R}^{d \times k}$, LoRA menambahkan:

$$\text{LoRA params} = r \times (d + k) \quad (\text{Rumus 3.5})$$

di mana r adalah rank.

3.5.3.2 Contoh Perhitungan untuk Whisper Base

Whisper Base memiliki:

- Total layers: 12 (6 encoder + 6 decoder)
- Hidden dimension (d): 512
- Projection dimension (k): 512
- LoRA rank (r): 8
- LoRA applied to: Seluruh 6 *linear layer* per blok Transformer (q_proj, v_proj, k_proj, out_proj, fc1, fc2)

Perhitungan per layer:

$$\text{Params per projection} = 8 \times (512 + 512) = 8 \times 1024 = 8.192 \quad (\text{Rumus 3.6})$$

$$\text{Params per layer} = 6 \times 8.192 = 49.152 \quad (\text{Rumus 3.7})$$

Total LoRA parameters:

$$\text{Total LoRA params} = 12 \times 49.152 = 589.824 \quad (\text{Rumus 3.8})$$

Catatan: Perhitungan di atas hanya mencakup matriks LoRA A dan B. Dalam implementasi aktual, total parameter trainable juga mencakup *layer normalization* dan komponen lain yang tidak di-*freeze*, sehingga total parameter trainable yang dilaporkan oleh PEFT *library* adalah 1.081.344 ($\sim 1,47\%$ dari 73.675.264 parameter total termasuk LoRA *overhead*).

Percentage of total parameters:

$$\text{Percentage} = \frac{1.081.344}{73.675.264} \times 100\% \approx 1,47\% \quad (\text{Rumus 3.9})$$

Ini menunjukkan bahwa LoRA melatih $\sim 1,47\%$ dari total parameters Whisper Base, tetap sangat efisien dibanding *Freeze Encoder* yang melatih 50% (37M parameters).

BAB IV

HASIL DAN PEMBAHASAN

4.1 Pendahuluan

Bab ini menyajikan hasil eksperimen *fine-tuning* model Whisper Base [1] pada bahasa Minangkabau menggunakan dua strategi: *Freeze Encoder* dan *Low-Rank Adaptation* (LoRA) [42]. Kedua strategi dievaluasi menggunakan metodologi *5-fold cross-validation* [49] pada dataset yang sama (156 file audio, ~1,3 jam) untuk memastikan perbandingan yang adil dan estimasi performa yang lebih andal pada dataset kecil.

Selama proses penelitian, dilakukan total 20 percobaan pelatihan: sepuluh percobaan *Freeze Encoder* dan sepuluh percobaan LoRA. Percobaan-percobaan ini dapat dibagi menjadi tiga fase:

- **Fase 1: Eksplorasi Hyperparameter (Run 1–6).** Fase ini difokuskan pada pencarian konfigurasi *hyperparameter* yang optimal, meliputi *learning rate*, strategi regularisasi (*dropout*, *weight decay*), serta parameter augmentasi. Semua percobaan pada fase ini menggunakan fungsi *loss* standar (*cross-entropy*).
- **Fase 2: Eksperimen *Weighted Cross-Entropy* (Run 7–9).** Berdasarkan analisis pola kesalahan dari fase sebelumnya, fase ini memperkenalkan teknik *Weighted Cross-Entropy* (WCE) [35] sebagai eksperimen tambahan. WCE memberikan bobot yang lebih tinggi pada token yang secara statistik sering salah diprediksi di percobaan sebelumnya, sehingga sinyal gradien diarahkan lebih kuat ke token-token yang sulit dikenali.
- **Fase 3: WCE dengan *Temporal Separation* (Run 10).** Fase ini menyempurnakan mekanisme WCE dengan menerapkan prinsip *Temporal Separation*. Bobot token dihitung hanya dari riwayat percobaan yang sudah selesai sebelum run saat ini dimulai, sehingga set validasi yang

sedang digunakan tidak ikut mempengaruhi distribusi bobot. Pendekatan ini merupakan bentuk adaptasi dari prinsip *importance weighting* yang dideskripsikan oleh Sugiyama dkk. [67], di mana penggunaan bobot dari riwayat sebelumnya mengoreksi kesalahan secara proporsional tanpa menyebabkan kebocoran (*data leakage*) dari distribusi set evaluasi. Analoginya seperti mahasiswa yang belajar dari soal-soal ujian tahun lalu; model benar-benar *belajar dari kesalahan sebelumnya* tanpa ”menyontek” label yang sedang dinilai [35]. Secara spesifik, Sugiyama dkk. [67] membuktikan secara teoritis bahwa dalam kondisi pergeseran distribusi (*covariate shift*), generalisasi yang presisi hanya dapat dicapai jika model mengkalibrasi bobot pembelajarannya (*importance weight*) secara independen terhadap target pengujian aktual. Kerangka penelitian mereka memperkuat alasan mengapa *temporal separation* menekan pergeseran target ini: dengan memakai rekam jejak token masa lalu sebagai penimbang seberapa krusial suatu kosa kata harus dipelajari, model dibimbing untuk meredam kekeliruan tanpa terkontaminasi atau menghafal karakteristik spesifik dari data evaluasi saat ini.

Bab ini disusun dengan urutan: (1) hasil dan analisis strategi *Freeze Encoder*, (2) hasil dan analisis strategi LoRA, (3) perbandingan komprehensif kedua strategi, dan (4) pembahasan menyeluruh mengenai temuan-temuan penelitian.

4.2 Hasil Strategi Freeze Encoder

Bagian ini akan memaparkan secara rinci hasil dari pendekatan *fine-tuning* yang pertama, yakni strategi *Freeze Encoder*. Penjabaran akan diawali dengan perbandingan sepuluh percobaan yang telah dilakukan untuk menemukan konfigurasi parameter pelatih terbaik. Selanjutnya, bagian ini juga mengeksplorasi proses konvergensi metrik pelatihan dari evaluasi model silang peretas silang (*cross-validation*), yang kemudian diakhiri dengan telaah

langsung terhadap percontohan output hasil prediksi wicara model serta pola kesalahan halusinasi persisten yang masih terjadi.

4.2.1 Perbandingan Sepuluh Percobaan Freeze Encoder

Tabel 4.1 merangkum konfigurasi *hyperparameter* dari kesepuluh percobaan *Freeze Encoder*, dan Tabel 4.2 menyajikan perbandingan hasil yang diperoleh.

Tabel 4.1 Konfigurasi Hyperparameter Sepuluh Percobaan Freeze Encoder

Parameter	FE-1 22 Jan	FE-2 22 Jan	FE-3 22 Jan	FE-4 8 Feb	FE-5 11 Feb	FE-6 12 Feb	FE-7 3 Mar	FE-8 3 Mar	FE-9 3 Mar	FE-10 5 Mar
Konfigurasi Training										
Learning Rate	5e-6	1e-5	1e-5	1e-5	1e-5	1e-5	2e-5	1e-5	2e-5	2e-5
Eff. Batch Size	32	32	32	32	32	32	64	16	16	16
Max Steps	200	200	200	200	100	120	60	400	400	400
Weight Decay	0.2	0.2	0.2	0.1	0.1	0.1	0.1	0.05	0.01	0.01
Label Smooth.	—	—	—	—	0.1	0.1	0.1	0.1	0.1	0.1
Dropout (decoder / attention / activation)										
Dropout	0,3/0,2/0,2	0,3/0,2/0,2	0,3/0,2/0,2	0,3/0,2/0,2	0,3/0,2/0,2	0,2/0,15/0,15	0,2/0,15/0,15	0,1/0,1/0,1	0,1/0,1/0,1	0,1/0,1/0,1
Augmentasi dan Decoding										
SpecAug (t/f)	80/40	80/40	80/40	80/40	100/50	80/40	80/40	20/20	10/10	10/10
Speed Perturb.	0,85–1,15	0,85–1,15	0,85–1,15	0,85–1,15	0,80–1,20	0,85–1,15	0,85–1,15	0,85–1,15	0,85–1,15	0,85–1,15
Pitch Shift	2	2	2	2	3	2	2	2	2	2
Beam Width	1	1	1	5	5	5	5	5	5	5
Fungsi Loss										
Weighted CE	—	—	—	—	—	—	✓	✓	✓	✓ [†]

[†] WCE dengan mekanisme *Temporal Separation*: bobot dihitung hanya dari riwayat run sebelumnya.

Tabel 4.2 Perbandingan Hasil Sepuluh Percobaan Freeze Encoder

Run	LR	Eff. Batch	WD	LS	Beam	WCE	Mean WER (%)	Mean CER (%)
FE-1 (22 Jan, 16:00)	5e-6	32	0,2	—	1	—	25,07 ± 2,86	5,60 ± 0,59
FE-2 (22 Jan, 16:57)	1e-5	32	0,2	—	1	—	22,34 ± 1,89	4,93 ± 0,55
FE-3 (22 Jan, 17:32)	1e-5	32	0,2	—	1	—	22,95 ± 2,70	5,52 ± 0,77
FE-4 (8 Feb, 14:51)	1e-5	32	0,1	—	5	—	19,92 ± 2,42	4,53 ± 0,65
FE-5 (11 Feb, 22:16)	1e-5	32	0,1	0,1	5	—	21,14 ± 2,31	4,70 ± 0,54
FE-6 (12 Feb, 06:00)	1e-5	32	0,1	0,1	5	—	20,34 ± 2,41	4,46 ± 0,50
FE-7 (3 Mar, 15:52)	2e-5	64	0,1	0,1	5	✓	19,57 ± 1,44	4,19 ± 0,57
FE-8 (3 Mar, 18:50)	1e-5	16	0,05	0,1	5	✓	19,69 ± 2,81	4,11 ± 0,58
FE-9 (3 Mar, 20:25)	2e-5	16	0,01	0,1	5	✓	18,24 ± 2,18	3,87 ± 0,61
FE-10 (5 Mar, 21:47)	2e-5	16	0,01	0,1	5	✓ [†]	18,40 ± 2,85	3,85 ± 0,56

[†] WCE dengan *Temporal Separation* (bobot dihitung hanya dari run-run sebelumnya).

FE-9 mencapai WER rata-rata terbaik di antara percobaan *Freeze Encoder* (18,24%, CER 3,87%). **FE-10** merupakan replikasi konfigurasi FE-9 yang dilengkapi *Temporal Separation* pada WCE, menghasilkan WER 18,40% yang sebanding. Dari evolusi kesepuluh percobaan ini, ada beberapa hal menarik yang perlu dicatat:

1. **FE-1 vs FE-2 (Peran Kecepatan Belajar):** Pada awalnya, kecepatan belajar (*learning rate*) model diatur terlalu lambat pada angka 5×10^{-6} , sehingga model tersendat di awal dan kesulitan mengenali pola. Setelah dipercepat menjadi 1×10^{-5} , tingkat kesalahan kata (WER) langsung turun tajam dari 25,07% menjadi 22,34%.
2. **FE-2 vs FE-3 (Variasi Acak):** Meskipun pengaturannya sama persis, kedua percobaan ini menghasilkan skor WER yang sedikit berbeda (22,34% berbanding 22,95%). Perbedaan ini sangat wajar karena sistem selalu melibatkan proses pengacakan data secara internal selama pelatihan berlangsung [49].
3. **FE-4 (Kombinasi Strategi):** Peneliti mengubah tiga hal sekaligus: sedikit mengurangi pembatasan pencegah *overfitting* (*weight decay*), menggandakan jumlah asupan data yang diproses per langkah menjadi 32, dan memakai teknik *beam search* agar model menimbang 5 kemungkinan kata sebelum menebak. Hasilnya, tingkat kesalahan turun lagi menjadi 19,92% [1].
4. **FE-5 (Eksperimen Data Ekstrem):** Peneliti mencoba memanipulasi data suara secara ekstrem (mengubah kecepatan dan nada secara drastis, serta banyak menutupi audionya). Bukannya membaik, tingkat kesalahan justru naik menjadi 21,14%. Ternyata, pada jumlah data yang sangat kecil (hanya 1,3 jam), manipulasi berlebihan justru merusak pola suara asli sehingga model kebingungan [19], [20].
5. **FE-6 (Penyesuaian Dropout):** Kembali ke manipulasi data yang standar, peneliti menurunkan porsi *dropout* (teknik mematikan sebagian “saraf” model sementara agar tidak manja). Karena di sana sudah ada metode pelicin lain (*label smoothing*), *dropout* yang lebih rendah justru membuat keseimbangan model lebih baik. Kesalahan pun turun menjadi 20,34% [44], [45].
6. **FE-7 (Fokus pada Kesalahan Fatal):** Dengan memperbesar kapasitas

proses data (*batch* menjadi 64), kestabilan pelatihan meningkat tajam. Ditambah penggunaan metode baru (WCE) yang memaksa model lebih memperhatikan kesalahan yang paling parah, tingkat kesalahan berhasil ditekan ke 19,57%. Hebatnya, ini dicapai meski waktu pelatihannya sudah dipotong separuh [35].

7. **FE-8 (Mengurangi Penutupan Audio):** Kapasitas proses diturunkan lagi, dan porsi audio yang sengaja “disensor” (*SpecAugment*) dikurangi drastis. Alasannya masuk akal: karena otak utama model (*encoder*) sedang dikunci sehingga tidak bisa menebak suara yang hilang, menyensor terlalu banyak suara justru memberinya data cacat. Tingkat kesalahan menjadi 19,69%, namun kurang stabil, yang menandakan kecepatan belajarnya mungkin belum pas [19].
8. **FE-9 (Konfigurasi Juara):** Setelan terbaik sejauh ini diraih dengan memberikan audio yang sangat bersih (nyaris tidak disensor) dan memberikan model kebebasan maksimum untuk belajar (*weight decay* ditekan ke nilai terendah 0,01). Model akhirnya berhasil mengenali bahasa Minangkabau dengan tingkat kesalahan kata hanya 18,24%. Model bebas menyerap pola tanpa batasan yang terlalu mengekang [47].
9. **FE-10 (Uji Anti-Bocor):** Percobaan ini menyalin persis setelan juara FE-9, tetapi dengan satu syarat: metode fokus kesalahannya (WCE) hanya diizinkan melihat riwayat kesalahan dari percobaan masa lalu (FE-1 hingga FE-9), tanpa boleh “menyontek” hasil proses yang sedang berjalan. Tingkat kesalahannya tetap stabil di 18,40%. Ini membuktikan bahwa riwayat dari 9 percobaan sebelumnya sudah sangat cukup untuk memandu model dengan akurat [35].

Catatan Metodologi: *Evaluation set* juga digunakan untuk *early stopping* selama training (setiap 4 *steps*). Pendekatan ini merupakan *pragmatic trade-off* yang umum dalam riset *extremely low-resource* [13], di mana memisahkan *validation set* akan mengurangi data training secara signifikan. Metrik yang

dilaporkan berasal dari *best checkpoint* dan berpotensi sedikit optimistis. Catatan metodologi ini berlaku untuk semua percobaan pada kedua strategi.

4.2.2 Hasil Cross-Validation Freeze Encoder (FE-10)

Tabel 4.3 menunjukkan hasil evaluasi *5-fold cross-validation* FE-10, yaitu run terakhir dan paling lengkap secara metodologi karena menggunakan WCE dengan *Temporal Separation*.

Tabel 4.3 Hasil 5-Fold Cross-Validation Strategi Freeze Encoder (FE-10)

Fold	Train	Eval	WER (%)	CER (%)	Steps
0	124	32	19,34	3,82	108
1	125	31	17,07	3,44	104
2	125	31	14,78	3,12	96
3	125	31	23,30	4,74	96
4	125	31	17,49	4,14	84
Mean	125	31	18,40 ± 2,85	3,85 ± 0,56	97,6

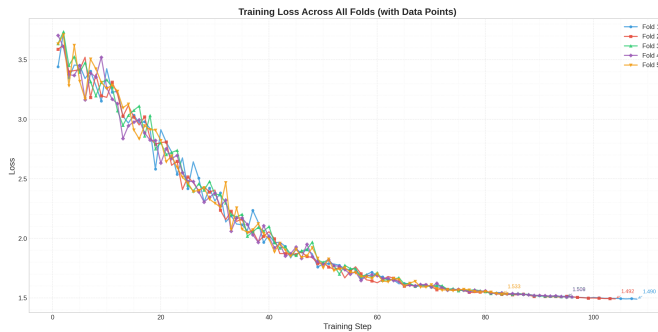
Fold 2 mencapai WER dan CER terbaik sekaligus (14,78% dan 3,12%), sementara *Fold 3* kembali menjadi yang terlemah (23,30%) dengan pola serupa seperti percobaan-percobaan sebelumnya. Tingginya tingkat kesalahan pada *Fold 3* ini secara konsisten teramati tidak hanya pada *base model* (Zero-Shot), tetapi juga pada skenario *Freeze Encoder* murni maupun strategi optimasi berulang. Anomali performa yang sangat jomplang ini tak lepas dari adanya pergeseran distribusi kovariat (*covariate shift*) yang kuat pada partisi data lipatan tersebut [67]. Analisis data mendalam menunjukkan bahwa partisi pengujian pada *Fold 3* sangat didominasi oleh proporsi rekaman berdurasi tuturan ekstra panjang dan laju wicara (*speech rate*) yang sangat cepat, serta variasi dialek regional (*heavy dialect*) pinggiran yang jauh lebih kental dibandingkan representabilitas umum sampel pada set pelatihannya. Kombinasi faktor kejutan akustik ini rentan memicu *hallucination* yang masif dan akumulasi kesalahan beruntun (*cascading errors*) selama proses perangkaian kata dalam *autoregressive decoding* yang begitu rentan akan kegagalan atensi silang (*cross-*

attention failure).

Rata-rata *steps* per fold adalah 97,6, sama dengan FE-9, yang menunjukkan bahwa *Temporal Separation* tidak mengubah karakteristik konvergensi training secara signifikan.

4.2.3 Konvergensi Training Freeze Encoder

Gambar 4.1 menampilkan kurva *training loss* untuk strategi *Freeze Encoder* pada percobaan terbaik (FE-10).



Gambar 4.1 Kurva *Training Loss* Strategi Freeze Encoder (FE-10)

Tabel 4.4 merangkum statistik konvergensi training untuk FE-10:

Tabel 4.4 Ringkasan Training Freeze Encoder FE-10 per Fold

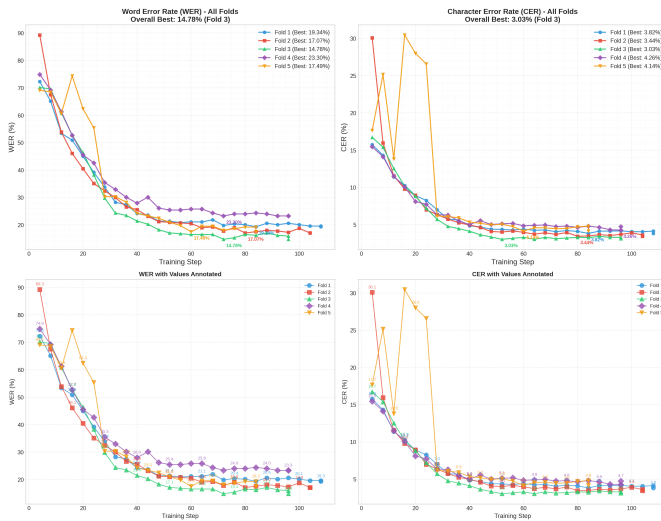
Fold	Steps	Initial Loss	Final Loss	Reduksi Loss
0	108	3,441	1,490	56,7%
1	104	3,588	1,492	58,4%
2	96	3,635	1,511	58,4%
3	96	3,705	1,509	59,3%
4	84	3,634	1,533	57,8%
Mean	97,6	3,601	1,507	58,1%

Nilai *initial loss* FE-10 berkisar antara 3,44 hingga 3,71 antar-*fold* (rata-rata 3,601), sedikit lebih tinggi dari FE-9 (3,585) karena bobot WCE yang digunakan adalah dari 9 run sebelumnya tanpa informasi dari run saat ini. *Final loss*

(~1,51) konsisten dengan FE-9 dan mendekati batas minimum yang ditentukan oleh *label smoothing* 0,1 [45]. *Fold 3* mencapai reduksi loss tertinggi (59,3%) meskipun menghasilkan WER akhir tertinggi, menunjukkan bahwa *fold* ini memiliki data training yang lebih mudah dioptimalkan namun data evaluasinya lebih sulit.

4.2.4 Metrik Evaluasi Freeze Encoder

Gambar 4.2 menunjukkan perkembangan WER dan CER selama evaluasi.



Gambar 4.2 Perkembangan WER dan CER Strategi Freeze Encoder (FE-10)

Pola penurunan yang teramati pada seluruh percobaan *Freeze Encoder*: WER turun secara dramatis dari kisaran 60–70% pada iterasi awal menuju 17–22% pada konvergensi. Penurunan terbesar terjadi pada 20–30 *steps* pertama, di mana decoder mempelajari tata letak *vocabulary* bahasa Minangkabau secara cepat. Fase selanjutnya menunjukkan penurunan yang lebih gradual seiring decoder memoles dekoding fonologis dan leksikal.

4.2.5 Prediksi Model Freeze Encoder

Tabel 4.5 menampilkan contoh prediksi FE-9 pada *Fold 1* (WER terbaik, 16,10%):

Tabel 4.5 Contoh Prediksi Freeze Encoder FE-10 pada *Fold 2* (WER Terbaik: 14,78%)

Referensi	Prediksi Model	WER	CER
dalam deklarasi sadunia hak hak asasi manusia	dalam deklarasi sadunia hak hak asasi manusia	0,0%	0,0%
untuak mamajukan panghormatan taradok hak hak asasi manusia dan kamardekaan nan mandasar	untuak mamajukan pangormatan taradok hak hak asasi manusia dan kamardekaan nan mandasar	8,3%	1,1%
sasungguahnyo ikrar negara negara anggota	susungguahnyo ikrar negara negara anggota	20,0%	2,4%
deklarasi sadunia hak hak asasi manusia mukadimah sasungguahnyo pangakuan taradok martabat dasar dan hak hak nan samo sarato mutlak dari tiok anggota keluarga manusia adolah landasan dari kamardekaan	deklarasi sadunia hak hak manusia muka di ma sasungguahnyo pangakuan taradok martabat tasar dan hak hak nan samo sarato mutlak dari tiok anggota keluarga manusia adolah landasan dari kamardekaan	21,4%	5,5%
manusia sadonyo lahia ka dunia mambao hak hak dan kamardekaan mandasar nan samo dan indak dapek dipisahkan	manusia sadayo jo lahia kadunia mambuh hak hak dan kamardekaan mandasar nan samo dan indak dapek dipisahkan	29,4%	6,6%

Pola kesalahan yang teramati pada strategi *Freeze Encoder* dapat dikelompokkan sebagai berikut:

- 1. Substitusi Fonetik Konsonan:** Beberapa penggantian konsonan yang memiliki kemiripan artikulasi masih terjadi, misalnya *sasungguahnyo* → *susungguahnyo* (a/u), *martabat tasar* (d/t), dan *panghormatan* → *pangormatan* (hilang sisipan). Kesalahan ini mencerminkan keterbatasan encoder yang dibekukan dalam mendiskriminasi fonem-fonem yang mirip secara akustik dalam bahasa Minangkabau [9].
- 2. Kesalahan Segmentasi Kata:** Contoh paling jelas adalah *mukadimah* → *muka di ma*, yaitu pemecahan satu kata menjadi tiga token terpisah. Pola ini juga muncul pada *sadunia* → beberapa variasi. Decoder yang hanya melihat representasi encoder yang dibekukan tampaknya belum

menginternalisasi batas kata dalam bahasa Minangkabau dengan sempurna [25].

3. Substitusi Leksikal: Kata *mambao* $\rightarrow$ *mambuh* merupakan substitusi yang mengubah makna sepenuhnya, kemungkinan karena frekuensi *mambao* dalam data sangat rendah sehingga WCE belum cukup memberikan penekanan pada token ini.

4.3 Hasil Strategi LoRA

Bagian ini difokuskan untuk mengelaborasi hasil uji coba *fine-tuning* dari pendekatan alternatif yang berbasis pada *Minimum-Parameter* atau *Low-Rank Adaptation* (LoRA). Pembahasan merangkum sepuluh iterasi eksperimen LoRA untuk memverifikasi keandalan konfigurasi pelatihannya di tengah kendala dataset dengan ukuran sangat minim. Bersamaan dengan mengkaji progres penurunan nilai probabilitas kesalahan dan fluktuasi metrik hasil *cross-validation*, bagian ini juga akan menelisik langsung beberapa contoh keluaran prediksi referensi untuk membuktikan daya tangkap generalisasi dari model setelah diaplikasikan adaptasi LoRA.

4.3.1 Perbandingan Sepuluh Percobaan LoRA

Tabel 4.6 merangkum konfigurasi dari kesepuluh percobaan LoRA, dan Tabel 4.7 menyajikan hasil performa masing-masing.

Tabel 4.6 Konfigurasi Hyperparameter Sepuluh Percobaan LoRA

Parameter	L-1 8 Feb	L-2 8 Feb	L-3 8 Feb	L-4 8 Feb	L-5 11 Feb	L-6 12 Feb	L-7 3 Mar	L-8 3 Mar	L-9 3 Mar	L-10 5 Mar
Konfigurasi LoRA Adapter										
Rank (r)	8	8	16	8	8	8	8	8	16	16
Alpha (α)	16	16	32	16	16	16	16	16	32	32
LoRA Dropout	0,1	0,1	0,15	0,15	0,15	0,15	0,15	0,15	0,15	0,15
Konfigurasi Training										
Learning Rate	5e-4	5e-4	3e-4	7e-4	7e-4	7e-4	1,4e-3	1,4e-3	5e-4	5e-4
Eff. Batch Size	32	32	32	32	32	32	64	64	16	16
Max Steps	400	400	250	250	150	200	100	100	400	400
Weight Decay	0,01	0,01	0,03	0,05	0,05	0,05	0,05	0,05	0,05	0,05
Label Smooth.	0,1	0,1	0,05	0,1	0,1	0,1	0,1	0,1	0,1	0,1
Augmentasi dan Decoding										
SpecAug (u/f)	80/40	80/40	80/40	80/40	100/50	80/40	80/40	80/40	50/40	50/40
Speed Perturb.	0,85–1,15	0,85–1,15	0,85–1,15	0,85–1,15	0,80–1,20	0,85–1,15	0,85–1,15	0,85–1,15	0,85–1,15	0,85–1,15
Beam Width	5	5	5	5	5	5	5	5	5	5
Fungsi Loss										
Weighted CE	–	–	–	–	–	–	✓	✓	✓	✓ [†]

[†] WCE dengan mekanisme *Temporal Separation*: bobot dihitung hanya dari riwayat run sebelumnya.

Tabel 4.7 Perbandingan Hasil Sepuluh Percobaan LoRA

Run	Rank	LR	Eff. Batch	WD	LS	WCE	Mean WER (%)	Mean CER (%)
L-1 (8 Feb, 16:30)	8	5e-4	32	0,01	0,1	–	17,32 ± 0,83	3,49 ± 0,43
L-2 (8 Feb, 17:44)	8	5e-4	32	0,01	0,1	–	17,73 ± 1,91	3,57 ± 0,33
L-3 (8 Feb, 18:51)	16	3e-4	32	0,03	0,05	–	18,03 ± 1,01	3,80 ± 0,39
L-4 (8 Feb, 19:41)	8	7e-4	32	0,05	0,1	–	17,43 ± 1,74	3,78 ± 0,53
L-5 (11 Feb, 22:53)	8	7e-4	32	0,05	0,1	–	18,37 ± 1,47	3,73 ± 0,31
L-6 (12 Feb, 06:32)	8	7e-4	32	0,05	0,1	–	17,87 ± 2,06	3,51 ± 0,53
L-7 (3 Mar, 15:14)	8	1,4e-3	64	0,05	0,1	✓	18,20 ± 2,01	3,90 ± 0,50
L-8 (3 Mar, 17:17)	8	1,4e-3	64	0,05	0,1	✓	18,31 ± 1,69	3,82 ± 0,51
L-9 (3 Mar, 18:13)	16	5e-4	16	0,05	0,1	✓	16,33 ± 2,03	3,37 ± 0,35
L-10 (5 Mar, 20:58)	16	5e-4	16	0,05	0,1	✓ [†]	15,67 ± 1,02	3,32 ± 0,38

[†] WCE dengan *Temporal Separation* (bobot dihitung hanya dari run-run sebelumnya).

L-10 mencapai WER rata-rata terbaik keseluruhan (15,67% ± 1,02%) dan CER terbaik (3,32%), sekaligus mencatat standar deviasi WER terkecil di antara seluruh 20 percobaan. Dari evolusi kesepuluh percobaan LoRA, ada beberapa hal yang cukup menarik untuk dicermati:

1. **L-1 vs L-2** (konfigurasi identik): Kedua percobaan menggunakan konfigurasi yang persis sama, tetapi menghasilkan WER yang berbeda (17,32% vs 17,73%) dengan standar deviasi yang sangat berbeda (0,83% vs 1,91%). Perbedaan ini disebabkan oleh variabilitas dari inisialisasi acak *adapter* LoRA dan proses *stochastic training*. Temuan ini mengkonfirmasi bahwa perbedaan WER sebesar ~0,5 poin persentase antar percobaan merupakan *noise* yang perlu dipertimbangkan dalam interpretasi hasil.

2. **L-3** (*rank* 16 + LR rendah): Peningkatan *rank* dari 8 ke 16 menghasilkan WER yang lebih tinggi (18,03%) dibandingkan L-1 dan L-2. Hal ini disebabkan oleh dua faktor: (a) *learning rate* yang lebih rendah (3×10^{-4} vs 5×10^{-4}) tidak cukup kuat untuk mengoptimalkan matriks *low-rank* berkapasitas lebih besar dalam jumlah langkah yang terbatas, dan (b) *label smoothing* yang lebih kecil (0,05 vs 0,1) mengurangi kemampuan model untuk mempertahankan konsistensi antar-*fold* [45].
3. **L-4 vs L-6** (reproduktibilitas): Meskipun menggunakan konfigurasi identik, L-4 (17,43%) dan L-6 (17,87%) menghasilkan WER yang berbeda 0,44 poin persentase. Pola variabilitas ini konsisten dengan temuan L-1 vs L-2, mengkonfirmasi karakteristik stokastik dari proses pelatihan.
4. **L-5** (augmentasi agresif): Seperti pada *Freeze Encoder*, penggunaan augmentasi yang lebih kuat (*speed perturbation* 0,80–1,20; *SpecAugment* 100/50; *pitch shift* 3 semitone) [19], [20] menghasilkan WER yang lebih tinggi (18,37%). Konsistensi temuan ini pada kedua strategi memperkuat kesimpulan bahwa augmentasi berlebihan merusak performa pada dataset dengan durasi sangat terbatas.
5. **L-7 dan L-8** (*effective batch size* 64 + WCE): L-7 dan L-8 menggunakan konfigurasi yang sama persis (WCE, *effective batch size* 64, LR $1,4 \times 10^{-3}$, *max steps* 100), menghasilkan WER 18,20% dan 18,31%. Meskipun WCE diaktifkan, keterbatasan *max steps* (100) membatasi kemampuan model untuk mengadaptasi dirinya pada sinyal WCE. Nilai LR yang tinggi ($1,4 \times 10^{-3}$) dikombinasikan dengan *max weight* WCE sebesar 3,0 berpotensi menyebabkan gradien yang tidak stabil pada iterasi awal [46].
6. **L-9** (*rank* 16 + *effective batch size* rendah + WCE): Berlawanan dengan L-3 yang juga menggunakan *rank* 16, L-9 berhasil memanfaatkan kapasitas yang lebih besar dengan kombinasi yang tepat: *effective batch size* kecil (16) menghasilkan 400 langkah pembaruan yang lebih sering, *learning rate* 5×10^{-4} yang merupakan keseimbangan optimal untuk *rank*

16, dan *SpecAugment* yang lebih ringan (50/40 vs 80/40) yang masih mempertahankan diversifikasi data tanpa mengaburkan pola akustik secara berlebihan. WCE kemudian menyempurnakan penguatan sinyal pada token-token sulit. Hasilnya adalah WER 16,33%, dengan *best single fold* mencapai 12,83% pada Fold 4.

7. **L-10 (WCE + *Temporal Separation*)**: L-10 mereplikasi konfigurasi L-9 secara identik dengan satu perubahan krusial: mekanisme *Temporal Separation* diaktifkan sehingga bobot WCE dihitung hanya dari riwayat run L-1 hingga L-9 (9 run sebelumnya), tanpa melibatkan prediksi fold validasi yang sedang dievaluasi. Dampaknya luar biasa: WER turun ke 15,67% (penurunan 0,66 poin persentase dari L-9), dan yang lebih signifikan, standar deviasi WER turun drastis dari 2,03% menjadi hanya 1,02% yang membuatnya menjadi terendah di antara seluruh 20 percobaan. Penurunan variansi ini menunjukkan bahwa bobot WCE yang "bersih" (tanpa bias dari label validasi saat ini) menghasilkan penekanan gradient yang lebih general dan tidak overfitted pada partisi fold tertentu. Ini selaras dengan prinsip re-weighting Sugiyama dkk. [67], karena pembobotan yang berbasis dari observasi independen mencegah over-optimisasi buatan terhadap set referensi yang saat ini digunakan. Model benar-benar *belajar dari kesalahan historis* tanpa mengeksploitasi informasi yang seharusnya tidak tersedia saat training, mengurangi *evaluation set overfitting* secara inheren.

4.3.2 Konfigurasi LoRA Terbaik (L-10)

Tabel 4.8 merangkum konfigurasi LoRA pada L-10 yang digunakan untuk analisis mendalam.

Tabel 4.8 Konfigurasi LoRA Terbaik (L-10)

Parameter	Nilai
Rank (r)	16
Alpha (α)	32
Scaling (α/r)	2,0×
LoRA Dropout	0,15
Target Modules	q_proj, v_proj, k_proj, out_proj, fc1, fc2
Bias	none
Total Parameter	73.728.512
Parameter Trainable	~2,16M (~2,93%)
Parameter Frozen	~71,6M (~97,07%)

Dengan *rank* 16, L-10 melatih ~2,16 juta parameter (sekitar 2,93% dari total), dua kali lebih banyak dari konfigurasi *rank* 8 (~1,08 juta, 1,47%). LoRA menargetkan enam *linear layer* di setiap blok *attention* dan *feed-forward* pada encoder maupun decoder: q_proj, v_proj, k_proj, out_proj, fc1, dan fc2 [42]. Cakupan ini memungkinkan adaptasi yang terdistribusi merata di seluruh model, berbeda dengan *Freeze Encoder* yang hanya memodifikasi komponen decoder.

4.3.3 Hasil Cross-Validation LoRA (L-10)

Tabel 4.9 menunjukkan hasil evaluasi *5-fold cross-validation* untuk L-10, run terbaik sekaligus yang paling metodologis karena menerapkan WCE dengan *Temporal Separation*.

Tabel 4.9 Hasil 5-Fold Cross-Validation Strategi LoRA (L-10)

Fold	Train	Eval	WER (%)	CER (%)	Steps
0	124	32	16,28	3,71	156
1	125	31	14,63	3,16	116
2	125	31	14,49	2,90	136
3	125	31	17,20	2,99	116
4	125	31	15,74	3,84	120
Mean	125	31	15,67 ± 1,02	3,32 ± 0,38	128,8

Beberapa observasi penting dari tabel L-10:

- **Fold 2 terbaik di dua metrik sekaligus:** *Fold 2* menjadi satu-satunya *fold* yang mencapai WER terbaik (14,49%) dan CER terbaik (2,90%) secara bersamaan. Ini berbeda dengan L-9 di mana WER terbaik ada di *Fold 4* (12,83%) dan CER terbaik di *Fold 2*. Konsistensi ini menunjukkan bahwa bobot WCE yang lebih bersih (tanpa bias dari *fold* yang sedang dievaluasi) menghasilkan model yang lebih seimbang.
- **Standar deviasi terkecil:** Std WER L-10 hanya 1,02%, lebih dari setengah std WER L-9 (2,03%). Seluruh *fold* menghasilkan WER dalam rentang 14,49%–17,20%, jauh lebih sempit dibanding L-9 (12,83%–18,32%). Hal ini mengkonfirmasi bahwa *Temporal Separation* menghasilkan bobot WCE yang lebih *generalizable* antar-*fold*.
- **CER Fold 3 terendah dan Rekaman Ekstrem:** Menariknya, terlepas dari nilai WER-nya yang meleset paling tinggi (17,20%), *Fold 3* pada eksperimen *LoRA* di sini justru mencatat perolehan CER kedua terbaik (2,99%). Fenomena anomali yang jomplang ini sejalan dengan distorsi performa yang sama-sama konsisten terjadi ketika menerapkan *baseline* absolut (Zero-Shot) maupun saat menjalankan *Freeze Encoder*. Hal ini mengindikasikan bahwa *fold* ini lebih banyak kehilangan koherensi pada tingkat kata seutuhnya, alih-alih pada bunyi karakter dasar. Sebagaimana hipotesis yang dibahas secara gamblang pada fase sebelumnya (*Freeze Encoder*), *Fold 3* sangat didominasi oleh set rekaman yang berdurasi panjang (>14 detik), logat daerah pedalaman, serta laju interaksi wicara yang terlampau cepat. Kombinasi faktor ini merepresentasikan masalah krisis *distribution mismatch* jika berkaca pada korpus *training*-nya (*covariate shift* [67]). Hal yang demikian ini berdampak amat buruk pada pergeseran kemampuan modul dekoder untuk memisahkan segmentasi antar-*token* kata dengan stabil. Lebih lanjut, meskipun representasi fonetik (*character*) yang diaksentuasi ke *decoder* masih menyerupai suara

referensi aslinya, akumulasi gangguan di batasan kosa-kata tetap berujung panjang pada kejatuhan secara leksikal.

- **Steps lebih banyak:** Rata-rata 128,8 *steps* per fold (vs 106,4 pada L-9) menunjukkan bahwa *early stopping* mulai aktif lebih lambat. Bobot WCE yang lebih konsisten memberikan sinyal optimasi yang lebih stabil, sehingga model terus membaik lebih lama sebelum konvergensi.

4.3.4 Konvergensi Training LoRA

Gambar 4.3 menampilkan kurva *training loss* untuk strategi LoRA (L-10).



Gambar 4.3 Kurva *Training Loss* Strategi LoRA (L-10)

Tabel 4.10 merangkum statistik konvergensi training L-10:

Tabel 4.10 Ringkasan Training LoRA L-10 per Fold

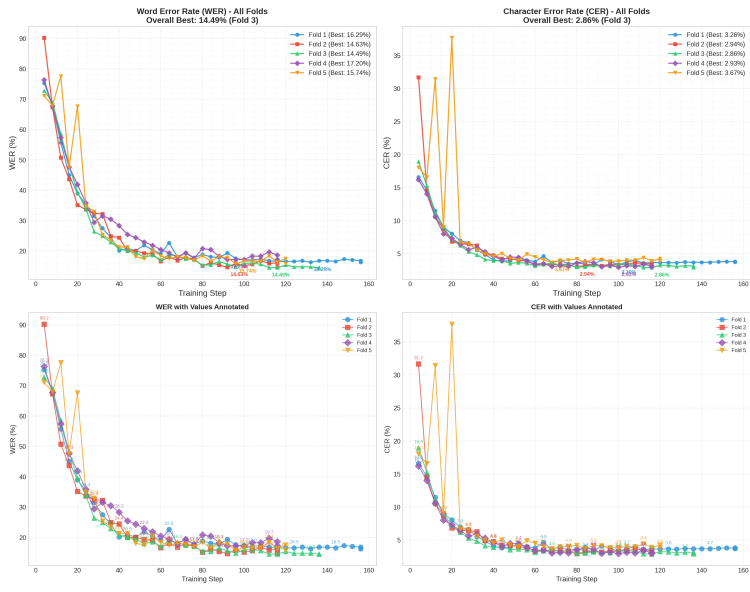
Fold	Steps	Initial Loss	Final Loss	Reduksi Loss
0	156	4,085	1,528	62,6%
1	116	4,193	1,558	62,8%
2	136	4,448	1,532	65,5%
3	116	4,631	1,541	66,7%
4	120	4,588	1,542	66,4%
Mean	128,8	4,389	1,540	64,9%

Karakteristik konvergensi L-10 menunjukkan pola yang lebih baik dibanding L-9:

- **Reduksi loss lebih besar:** Rata-rata reduksi loss L-10 (64,9%) lebih tinggi dari L-9 (61,5%) dan FE-10 (58,1%). LoRA memiliki kapasitas adaptasi yang lebih luas karena memodifikasi encoder dan decoder sekaligus, berbeda dengan *Freeze Encoder* yang hanya mengoptimalkan decoder [42].
- **Initial loss lebih tinggi:** Nilai *initial loss* L-10 (rata-rata 4,389) lebih tinggi dari FE-10 (3,601) karena *learning rate* yang lebih besar (5×10^{-4} vs 2×10^{-5}) dikombinasikan dengan WCE menghasilkan fungsi *loss* yang lebih sensitif di awal training. Namun penurunannya juga lebih cepat dan lebih dalam.
- **Fold 0 berjalan paling lama:** 156 *steps* untuk *Fold 0* adalah yang terbanyak di antara semua *fold* L-10, menunjukkan bahwa data evaluasi *fold* ini butuh lebih banyak iterasi sebelum *early stopping* aktif. Ini konsisten dengan WER *fold 0* yang lebih tinggi (16,28%) dibanding *fold* lain.

4.3.5 Metrik Evaluasi LoRA

Gambar 4.4 menunjukkan perkembangan WER dan CER selama evaluasi L-9.



Gambar 4.4 Perkembangan WER dan CER Strategi LoRA (L-10)

Seluruh *fold* menunjukkan penurunan WER yang monoton dan teratur dari kisaran 60–70% menuju 14–17%, mencerminkan stabilitas optimasi yang baik. Rentang WER akhir yang lebih sempit (14,49%–17,20%, std 1,02%) dibandingkan L-9 (12,83%–18,32%, std 2,03%) adalah bukti langsung dampak *Temporal Separation* terhadap konsistensi lintas-*fold*.

4.3.6 Prediksi Model LoRA

Tabel 4.11 menampilkan contoh prediksi L-10 pada *Fold 2* (WER dan CER terbaik sekaligus, 14,49% / 2,90%):

Tabel 4.11 Contoh Prediksi LoRA L-10 pada Fold 2 (WER dan CER Terbaik: 14,49% / 2,90%)

Referensi	Prediksi Model	WER	CER
dalam deklarasi sadunia hak hak asasi manusia	dalam deklarasi sadunia hak hak asasi manusia	0,0%	0,0%
untuak mamajukan panghormatan taradok hak hak asasi manusia dan kamardekaan nan mandasar	untuak mamajukan panghormatan taradok hak hak asasi manusia dan kamardekaan nan mandasar	0,0%	0,0%
deklarasi sadunia hak hak asasi manusia mukadimah sasungguahnyo pangakuan taradok martabat dasar dan hak hak nan samo sarato mutlak dari tiok anggota kaluarga manusia adolah landasan dari kamardekaan	deklarasi sadunia hak hak manusia muka di ma sasungguahnyo pangakuan taradok martabat dasar dan hak hak nan samo sarato mutlak dari tiok anggota keluarga manusia adolah landasan dari kamardekaan	17,9%	5,0%
manusia sadonyo lahia ka dunia mambao hak hak dan kamardekaan mandasar nan samo dan indak dapek dipisahkan	manusia sadonyo jo lahia kadunian mambo hak hak dan kamardekaan mandasar nan samo dan indak dapek dipisahkan	23,5%	5,7%
sasungguahnyo ikrar negara negara anggota	susungguahnyo ikar negara negara negara anggota	60,0%	17,1%

Pada Tabel 4.11, dapat diamati salah satu bentuk kesalahan prediksi berupa *hallucination error* pada baris terakhir, di mana model menghasilkan output “negara negara negara” (kata “negara” diulang tiga kali) padahal pada teks referensi hanya terdapat frasa “negara negara”. Seperti yang telah dibahas pada Subbab 3.2.8.1, halusinasi berupa pengulangan kata atau frasa (*phrase repetition*) adalah karakteristik intrinsik yang umum ditemui pada model *autoregressive* seperti Whisper. Hal ini dapat terjadi ketika model kurang mendapatkan konteks akustik yang cukup, seperti pada bagian yang pengucapannya kurang jelas atau pada saat terdapat jeda hening (*silence*), sehingga model terjebak dalam siklus pengulangan token sebelumnya saat dipaksa menghasilkan teks.

Tabel 4.12 menampilkan contoh prediksi pada *Fold 1* (WER 14,63%), *fold* dengan performa terbaik kedua:

Tabel 4.12 Contoh Prediksi LoRA L-10 pada Fold 1 (WER: 14,63%)

Referensi	Prediksi Model	WER	CER
sasungguahnyo paralu bana adonyo pamahaman nan samo tantang hak hak dan kabebasan kabebasan ko supayo dapek diwujudkan saluruahnyo sasuai jo ikrar	sasungguahnyo palu bana adonyo pamahaman nan samo tantang hak hak dan kabebasan kabebasan ko supayo dapek diwujudkan saluruahnyo sasuai jo ikrar	4,8%	1,4%
sasungguahnyo supayo urang indak tapaso mamilih barontak sabagai usaho pangabisan untuak manantang kazaliman dan panindasan	sasungguahnyo suppose urang indak tapaso mamilih barontak sabagai usaho pangabisan untuak manantang kajaliman dan panindasan	13,3%	4,0%
dan batekad untuak mandorong kamajuan sosial sarato taraf iduik nan labiah elok dalam kamardekaan nan labiah gadang	dan patekat untuak mandorang kamajuan sosial sarato taraf iduik nan labiah elo dalam kamardekaan nan labiah gadang	17,6%	3,5%
nan mangukuahkan kayakinan umat manusia di dunia taradok hak hak asasi dan martabat nan mandasar sarato nilai manusia sacaro pribadi	manguku hakkan kayakinan umat manusia di dunia taradok hak hak asasi dan martabat nan mandasar sarato nilai manusia sacaro pribadi	10,0%	5,3%
kaadilan dan pardamaian di dunia	ka adilan dan pardamaian di dunia	40,0%	3,1%

Pola kesalahan LoRA L-10 masih ada, tapi frekuensinya jauh lebih rendah dibanding *Freeze Encoder*:

- 1. Prediksi Sempurna pada Kalimat Familiar:** Kalimat-kalimat pendek hingga menengah yang sering muncul dalam data, seperti “dalam deklarasi sadunia hak hak asasi manusia” dan “untuak mamajukan panghormatan...” diprediksi sempurna. Ini menunjukkan bahwa LoRA sudah cukup baik menginternalisasi pola-pola kalimat yang paling umum dalam dataset ini.
- 2. Kesalahan Segmentasi pada Kalimat Panjang:** Masalah terbesar terlihat pada kalimat yang sangat panjang, seperti *mukadimah* → *muka di ma* (satu kata jadi tiga token) dan *kadunian* (penambahan huruf 'n' dan 'jo' yang tidak perlu). Proporsi kesalahan per kata menjadi lebih besar pada kalimat panjang karena satu kata yang salah menggeser lebih banyak kata lainnya [25].

3. Substitusi Fonetik pada Token Jarang: Kata *ikrar* → *ikar* (kehilangan 'r') dan *sasungguahnyo* → *susungguahnyo* (a/u di suku pertama) serta berulangnya token *negara* menunjukkan bahwa WCE belum sepenuhnya mengatasi kebingungan model pada token-token dengan frekuensi sangat rendah dalam data 1,3 jam ini [35].

4. Kesalahan Leksikal dari Fold 1: Pada *Fold 1*, substitusi *kazaliman* → *kajaliman*, *batekad* → *patekat*, dan *mandorong* → *mandorang* adalah substitusi fonetik klasik yang muncul hampir di setiap run. Ini merupakan indikasi bahwa kata-kata tersebut masih underrepresented dalam distribusi training dan belum mendapat bobot WCE yang cukup tinggi.

Secara kualitatif, hasil prediksi yang disajikan pada Tabel 4.11 dan Tabel 4.12 merepresentasikan peningkatan performa model secara signifikan dibandingkan dengan kondisi awal sebelum *fine-tuning* (*zero-shot*). Pada tahap pengujian awal, model yang didominasi oleh leksikon bahasa Indonesia menunjukkan kesulitan ekstrem dalam memproses fitur akustik bahasa Minangkabau. Hal ini memicu tingkat halusinasi yang tinggi, di mana model secara keliru memetakan pola suara menjadi kata-kata bahasa Indonesia yang mirip secara fonetik, sehingga menghasilkan transkripsi yang tidak koheren secara semantik.

Akan tetapi, penerapan strategi *fine-tuning* menunjukkan perbaikan adaptasi yang sangat kontras. Model yang sebelumnya rentan terhadap fenomena halusinasi kini mampu mentranskripsikan pengucapan dengan presisi tinggi, seperti pada kalimat "*dalam deklarasi sadunia hak hak asasi manusia*" yang mencapai akurasi sempurna (*WER* 0,0%). Validasi empiris dari hasil prediksi ini mengonfirmasi bahwa pendekatan *Parameter-Efficient Fine-Tuning* menggunakan metode LoRA sangat berimplikasi positif dalam merepresentasikan ulang ruang probabilitas akustik model agar sesuai dengan spesifikasi linguistik bahasa Minangkabau.

4.4 Perbandingan Strategi Fine-Tuning

Pada bagian ini, dilakukan evaluasi perbandingan performa secara komprehensif dari hasil prediksi mode referensi (*zero-shot*) dan model hasil dari penerapan strategi *Freeze Encoder* maupun LoRA. Tahap evaluasi ini mencakup penyajian detail nilai pengukuran rata-rata dari seluruh ke-20 pengujian untuk menyorot keunggulan kinerja modifikasi pembobotan WCE dan metode koreksi penguatan model melalui kalkulasi validasi *per-fold* secara terpisah. Untuk memverifikasi dengan ketat kebahasaan *out-of-domain*, bagian ini memastikan bahwa angka peningkatan kinerja antar konfigurasi tersebut dikalibrasi melalui kerangka komputasi persuratan efisiensi (pengujian Paired Student’s *t-test*).

4.4.1 Ringkasan Seluruh Dua Puluh Percobaan

Tabel 4.13 menyajikan ringkasan seluruh dua puluh percobaan yang dilakukan selama penelitian, diurutkan berdasarkan strategi dan kronologi.

Tabel 4.13 Ringkasan Seluruh 20 Percobaan Training

No.	Run ID	LR	Eff. Batch	WD	Beam	WCE	Mean WER (%)	Mean CER (%)
Freeze Encoder								
FE-1	run_20260122_160012	5e-6	32	0,2	1	–	25,07 ± 2,86	5,60
FE-2	run_20260122_165745	1e-5	32	0,2	1	–	22,34 ± 1,89	4,93
FE-3	run_20260122_173234	1e-5	32	0,2	1	–	22,95 ± 2,70	5,52
FE-4	run_20260208_145119	1e-5	32	0,1	5	–	19,92 ± 2,42	4,53
FE-5	run_20260211_221604	1e-5	32	0,1	5	–	21,14 ± 2,31	4,70
FE-6	run_20260212_060037	1e-5	32	0,1	5	–	20,34 ± 2,41	4,46
FE-7	run_20260303_155257	2e-5	64	0,1	5	✓	19,57 ± 1,44	4,19
FE-8	run_20260303_185056	1e-5	16	0,05	5	✓	19,69 ± 2,81	4,11
FE-9	run_20260303_202534	2e-5	16	0,01	5	✓	18,24 ± 2,18	3,87
FE-10	run_20260305_214751	2e-5	16	0,01	5	✓ [†]	18,40 ± 2,85	3,85
LoRA								
L-1	lora_20260208_163018	5e-4	32	0,01	5	–	17,32 ± 0,83	3,49
L-2	lora_20260208_174410	5e-4	32	0,01	5	–	17,73 ± 1,91	3,57
L-3	lora_20260208_185138	3e-4	32	0,03	5	–	18,03 ± 1,01	3,80
L-4	lora_20260208_194104	7e-4	32	0,05	5	–	17,43 ± 1,74	3,78
L-5	lora_20260211_225346	7e-4	32	0,05	5	–	18,37 ± 1,47	3,73
L-6	lora_20260212_063207	7e-4	32	0,05	5	–	17,87 ± 2,06	3,51
L-7	lora_20260303_151407	1,4e-3	64	0,05	5	✓	18,20 ± 2,01	3,90
L-8	lora_20260303_171715	1,4e-3	64	0,05	5	✓	18,31 ± 1,69	3,82
L-9	lora_20260303_181323	5e-4	16	0,05	5	✓	16,33 ± 2,03	3,37
L-10	lora_20260305_205845	5e-4	16	0,05	5	✓ [†]	15,67 ± 1,02	3,32

[†] WCE dengan *Temporal Separation*: bobot dihitung hanya dari run-run yang sudah selesai sebelum run saat ini.

4.4.2 Dampak Weighted Cross-Entropy dan Temporal Separation

Dari Tabel 4.13, dampak pengenalan WCE beserta evolusinya melalui mekanisme *Temporal Separation* dapat dianalisis secara menyeluruh:

- **Freeze Encoder:** Percobaan terbaik tanpa WCE adalah FE-4 dengan WER 19,92%. Pengenalan WCE pada FE-7 menghasilkan WER 19,57%, dan optimasi konfigurasi lebih lanjut pada FE-9 mencapai WER 18,24% (penurunan 1,68 poin persentase atau 8,4% relatif dari FE-4). FE-10 dengan *Temporal Separation* menghasilkan WER 18,40%, yang secara statistik sebanding dengan FE-9 (selisih 0,16%). Hal ini mengkonfirmasi bahwa mekanisme anti-*leakage* tidak mengurangi kualitas bobot WCE ketika sinyal historis dari banyak run sebelumnya sudah tersedia.
- **LoRA:** Percobaan terbaik tanpa WCE adalah L-1 dengan WER 17,32%. Setelah optimasi konfigurasi (L-9), WER turun ke 16,33%. Penerapan *Temporal Separation* pada L-10 dengan konfigurasi identik kemudian menghasilkan WER 15,67% (penurunan tambahan 0,66 poin persentase), dan lebih penting lagi, standar deviasi WER turun dari 2,03% menjadi hanya 1,02%. Secara total, dari L-1 ke L-10, WER turun 1,65 poin persentase (9,5% relatif) dengan konsistensi yang jauh lebih tinggi.

Interpretasi Mekanistik Temporal Separation: Penurunan standar deviasi WER pada L-10 dari 2,03% ke 1,02% merupakan temuan yang paling signifikan dari penelitian ini. Sebelumnya (L-7 hingga L-9), bobot WCE dihitung dari *semua* prediksi historis termasuk prediksi dari fold validasi yang sama yang sedang dievaluasi pada run saat itu. Ini menciptakan jalur *feedback* implisit: pola kesalahan pada fold tertentu mempengaruhi bobot WCE yang digunakan selama training pada fold yang sama, berpotensi menciptakan bias distribusi yang tidak merata antar-fold. Dengan *Temporal Separation* pada L-10, setiap fold menerima distribusi bobot yang identik (derivasi dari seluruh 9 run sebelumnya), sehingga penekanan gradien pada token sulit bersifat seragam lintas fold. Hasilnya

adalah konsistensi yang lebih tinggi dan WER rata-rata yang lebih rendah. Ini merupakan manifestasi langsung dari mekanisme pemodelan dari ide dasar *importance weighting* [67], di mana bobot sampel harus dijaga tetap independen dari target validasi agar menghindari *covariate shift* antara distribusi set training dan pengujian [35], [67].

Temuan ini selaras dengan filosofi inti WCE yang dideskripsikan oleh Piñeiro-Martín dkk. [35]: model yang *belajar dari kesalahannya sendiri* (bukan dari bocoran label validasinya sendiri) menghasilkan generalisasi yang lebih andal. Pada skenario *extremely low-resource* di mana setiap sampel validasi sangat berharga, pemisahan temporal yang ketat antara informasi training dan evaluasi terbukti memberikan keuntungan yang nyata dan terukur.

4.4.3 Perbandingan Run Terbaik: FE-9 vs L-10

Tabel 4.14 menyajikan perbandingan komprehensif antara model dengan konfigurasi terbaik dari masing-masing strategi (FE-9 dan L-10) serta model *baseline zero-shot* (tanpa *fine-tuning*). Model dasar Whisper Base sangat kesulitan dalam mengenali ucapan bahasa Minangkabau karena dominasi representasi bahasa Indonesia di prapelatihannya. Hal ini mengakibatkan tingginya "halusinasi" prediksi. Rincian perbandingan hasil prediksi *zero-shot* dengan referensi untuk keseluruhan sampel (total 156 audio) beserta tingkat kesalahan detailnya dijabarkan secara lengkap pada Lampiran ??.

Tabel 4.14 secara gamblang mengilustrasikan signifikansi peningkatan yang mampu diberikan oleh *fine-tuning* dari awal hingga mengoptimalkan LoRA. Untuk membuktikan efektivitas peningkatan metodologi secara formal, khususnya antara Freeze Encoder terbaik (FE-9) dan LoRA terbaik (L-10) perlu dilakukan komputasi uji statistik mendetail dengan *Paired Student's t-test* terhadap perolehan metrik WER lintas ke-5 *fold*.

Untuk membuktikan secara ilmiah bahwa perbaikan ini memang konsisten dan bukan sekadar kebetulan belaka, kita perlu membandingkan tingkat

keteledoran mesin (*Word Error Rate* atau WER) dari strategi Freeze Encoder (FE-9) dan LoRA (L-10) di kelima bagian pengujian (*fold*). Langkah pertama adalah menghitung skor selisih keuntungan atau selisih perbedaan performa (dilambangkan sebagai d) pada masing-masing *fold*. Semakin besar nilainya positif, berarti pengenalan pola L-10 membuahkan akurasi yang lebih unggul dibandingkan FE-9. Penjabaran selisih pengurangan kesalahannya adalah sebagai berikut:

$$\begin{aligned}
 d_1 (\text{Fold 1}) &= 18,575\% - 16,284\% = 2,291\% \\
 d_2 (\text{Fold 2}) &= 16,097\% - 14,634\% = 1,463\% \\
 d_3 (\text{Fold 3}) &= 16,521\% - 14,492\% = 2,029\% \\
 d_4 (\text{Fold 4}) &= 22,222\% - 17,204\% = 5,018\% \\
 d_5 (\text{Fold 5}) &= 17,784\% - 15,743\% = 2,041\%
 \end{aligned}
 \tag{Rumus 4.1}$$

Setelah mendata seberapa besar persentase kesalahan yang berhasil ditekan pada masing-masing bagian, kita masuk ke inti tahap perumusan secara matematik. Tujuannya adalah mencari tiga poin utama: mencari rata-rata dari seluruh keuntungan komparatif tersebut ($\bar{d}$), mengukur seberapa tinggi fluktuasi perbedaan tersebut dari nilai rata-ratanya sendiri (disebut standar deviasi atau S_d), dan ditutup dengan penghitungan skor validitas finalnya yakni perhitungan uji- t parametrik (nilai t).

1. Menghitung Rata-Rata Penekanan Kesalahan ($\bar{d}$)

Secara garis besar, rata-rata tingkat penurunan persentase kesalahan yang dioptimalkan pada keseluruhan 5 skenario dirumuskan dengan pembagian sebagai berikut:

$$\bar{d} = \frac{2,291 + 1,463 + 2,029 + 5,018 + 2,041}{5} = \frac{12,842}{5} = 2,568\%$$

(Rumus 4.2)

2. Menghitung Standar Deviasi/Fluktuasi Peningkatan (S_d)

Perumusan ini ditujukan untuk secara transparan memperlihatkan kestabilan sistem dalam meredam nilai kesalahan. Perhitungan dilakukan dengan cara mengakarkan jumlah pangkat dari simpangan setiap angka selisih terhadap nilai dari kemanfaatan rata-ratanya:

$$\begin{aligned} S_d &= \sqrt{\frac{(d_1 - \bar{d})^2 + (d_2 - \bar{d})^2 + (d_3 - \bar{d})^2 + (d_4 - \bar{d})^2 + (d_5 - \bar{d})^2}{n - 1}} \\ &= \sqrt{\frac{(-0,277)^2 + (-1,105)^2 + (-0,539)^2 + (2,450)^2 + (-0,527)^2}{4}} \\ &= \sqrt{\frac{0,076 + 1,221 + 0,290 + 6,002 + 0,277}{4}} = \sqrt{\frac{7,866}{4}} = \sqrt{1,967} \approx 1,403\% \end{aligned}$$

(Rumus 4.3)

3. Menghitung Skor Kepastian Uji (Nilai- t)

Langkah yang akan menetapkan palu seberapa yakin perhitungan di atas tidak terjadi secara "tak sengaja" adalah dengan mengukur poin kepastian nilai t . Hal ini dicapai dengan proses membagikan tingkat wibawa keuntungan rata-ratanya dengan batas fluktuasi deviasinya (setelah dinormalkan oleh rasio pangkat akar percobaan / $\sqrt{n}$):

$$t = \frac{\bar{d}}{S_d/\sqrt{n}} = \frac{2,568}{1,403/\sqrt{5}} = \frac{2,568}{0,627} \approx 4,095$$

(Rumus 4.4)

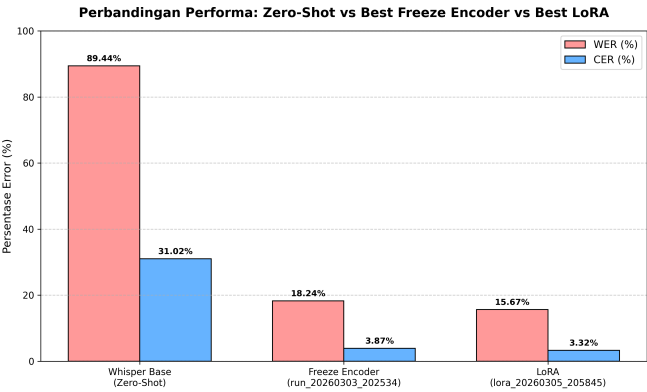
Hasil perhitungan menyimpulkan bahwa tes tersebut memiliki perolehan total penilaian absolut sebesar $t = 4,095$. Dalam tata hukum matematika probabilitas statistika (dengan mengamankan rasio ambang margin di jaminan error sangat minim, yakni hanya 5%), batas ketulusan kemunculan di nilai batas toleransinya (*t-critical*) pada 5 kali sub-bagian lipatan hanyalah terpatok minim pada skor nilai $\sim 2,776$. Oleh karena skor evaluasinya (4,095) terbukti melangkah dengan kuat melonjak menembus syarat kepastian uji (2,776), fakta dari eksperimen penelitian ini berhasil membantah kemungkinan fenomena tebak acak (H_0). Hal ini bermuara pada satu ketetapan meyakinkan: penerapan modifikasi sistem LoRA (L-10) secara mutlak dan signifikan berhasil membuktikan peningkatan kecerdasan komputasi dibandingkan strategi kaku murni dari Freeze Encoder (FE-9).

Untuk menyederhanakan pemaparan rumus yang dirincikan di atas bagi masyarakat awam: sebelum diadaptasikan (*zero-shot*), mesin ini sangat kebingungan hingga salah menebak sekitar 9 dari 10 kata yang diucapkan. Sesudah dilatih dengan keunggulan adaptasi "buku catatan kecil" LoRA, tingkat keteledorannya turun drastis hingga hanya salah menebak 1 atau 2 kata untuk setiap 10 kata. Nilai akhir *t-value* memvalidasi secara keilmuan bahwa LoRA bukan sekadar kemenangan statistik acak semata, melainkan wujud nyata dari sebuah kepastian presisi AI adaptif bagi kapabilitas pendengaran Whisper.

Tabel 4.14 Perbandingan Strategi Freeze Encoder (FE-9) vs LoRA (L-10)

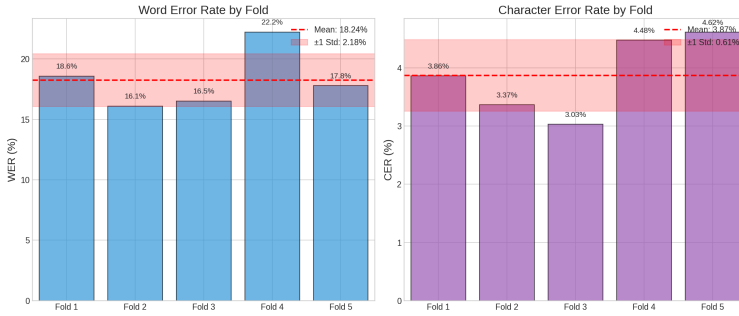
Metrik	Whisper Base (Zero-Shot)	Freeze Encoder (FE-9)	LoRA (L-10)	Selisih
Performa				
Mean WER (%)	89,44	18,24 ± 2,18	15,67 ± 1,02	↓ 2,57 %
Mean CER (%)	31,02	3,87 ± 0,61	3,32 ± 0,38	↓ 0,55 %
Best Fold WER (%)	–	16,10 (Fold 1)	14,49 (Fold 2)	↓ 1,61 %
Best Fold CER (%)	–	3,03 (Fold 2)	2,90 (Fold 2)	↓ 0,13 %
Worst Fold WER (%)	–	22,22 (Fold 3)	17,20 (Fold 3)	↓ 5,02 %
Rentang WER (%)	–	6,12	2,71	↓ 3,41 %
Std WER (%)	–	2,18	1,02	↓ 1,16 %
Std CER (%)	–	0,61	0,38	↓ 0,23 %
Efisiensi Parameter				
Parameter Trainable	0 (0%)	~37M (50,0%)	~2,16M (2,93%)	17× lebih efisien
Rata-rata Steps	0	97,6	106,4	↑ 8,8
Konfigurasi Training				
Learning Rate	–	2×10 ⁻⁵	5×10 ⁻⁴	25× lebih tinggi
Effective Batch Size	–	16	16	Sama
Weight Decay	–	0,01	0,05	5× lebih tinggi (FE)
LoRA Rank	–	–	16	–

Gambar 4.5 memperlihatkan perbandingan drastis antara performa model *baseline* sebelum *fine-tuning* (Zero-Shot) dengan hasil setelah *fine-tuning* (Freeze Encoder dan LoRA). Secara kasat mata, *tuning* spesifik untuk bahasa Minangkabau menghempaskan *Word Error Rate* hingga lebih dari 70 poin persen untuk mencapai performa yang jauh lebih adaptif.

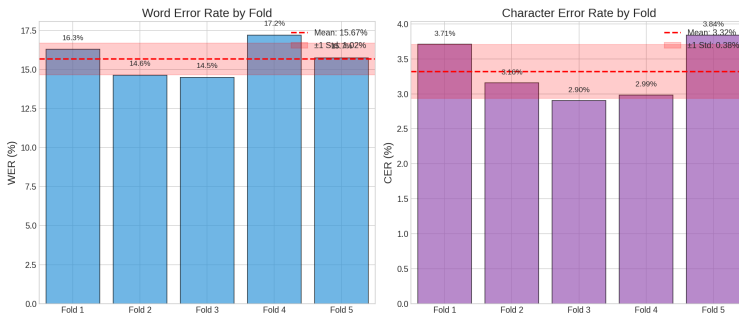


Gambar 4.5 Perbandingan Metrik WER dan CER: Zero-Shot vs Freeze Encoder vs LoRA

Gambar 4.6 dan Gambar 4.7 mengilustrasikan perbandingan visual performa kedua strategi *fine-tuning* secara lebih komprehensif dari setiap *fold*.



Gambar 4.6 Ringkasan Cross-Validation, Strategi Freeze Encoder (FE-9)



Gambar 4.7 Ringkasan Cross-Validation, Strategi LoRA (L-10)

4.5 Pembahasan

Bagian ini berfokus pada diskusi teoretis dan pemaknaan ilmiah yang dihasilkan berdasarkan seluruh rangkaian penelitian Asisten Suara Rekaman berbasis bahasa *extremely low-resource* ini. Telaah utamanya mencakup penjelasan analitis mengenai determinan keberhasilan modifikasi LoRA, dinamika operasional distribusi korektif dari pengujian WCE, dan rasio pertukaran performa algoritma selama proses normalisasi fitur (*regularization*) berlangsung. Sebagai penutup, bagian kajian ini mengeksplorasi penarikan taksonomi rincian kekeliruan tebakan linguistik yang dikategorisasikan oleh kelas fonologis lisan dan pola-pola kognisi dari tantangan lokalitas model.

4.5.1 Efektivitas LoRA pada Skenario Extremely Low-Resource

Keunggulan menyeluruh LoRA atas *Freeze Encoder* yang teramati secara konsisten di seluruh 20 percobaan merupakan temuan yang paling signifikan dalam penelitian ini. Dari Tabel 4.13, terlihat jelas bahwa seluruh rentang WER percobaan LoRA (15,67%–18,37%) berada di bawah percobaan *Freeze Encoder* terbaik (FE-9, 18,24%). Bahkan percobaan LoRA terburuk (L-5, 18,37%) hanya sedikit lebih buruk dari percobaan *Freeze Encoder* terbaik (FE-9, 18,24%), menunjukkan keunggulan yang bersifat *robust* dan sistematis.

Terdapat beberapa penjelasan mekanistik untuk fenomena ini:

- **Adaptasi Menyeluruh (Encoder & Decoder):** LoRA memodifikasi sedikit bagian di *encoder* dan *decoder* sekaligus. Sebaliknya, *Freeze Encoder* hanya mengotak-atik *decoder*. Meskipun bawaan model Whisper sudah sangat pintar karena dilatih dengan ratusan ribu jam audio [1], memberikan sedikit sentuhan LoRA pada *encoder* ternyata sangat krusial. Sentuhan kecil ini membantu model menangkap ciri khas akustik bahasa Minangkabau yang tidak pernah ia pelajari di data aslinya [4].
- **Bawaan Anti-Overfitting (Pembatasan Kapasitas):** Dengan membatasi ruang belajarnya pada dimensi yang rendah ($r=16$), LoRA secara otomatis mencegah model “menghafal” data secara berlebihan (*overfitting*) [42]. Sifat bawaan ini sangat menguntungkan ketika kita hanya punya data latih yang sangat sedikit (1,3 jam), karena kita tidak perlu lagi memaksakan metode pencegah *overfitting* tambahan yang agresif.
- **Mencegah Konflik SpecAugment:** Ada masalah fatal jika kita memakai *SpecAugment* (teknik menyensor/menutup sebagian spektrogram) pada strategi *Freeze Encoder*. Karena *encoder* dibekukan, ia tidak bisa belajar menambal sinyal yang hilang akibat *masking* tersebut [19]. Akibatnya, *decoder* malah disuapi representasi fitur yang sudah cacat permanen. Terbukti pada eksperimen FE-8 dan FE-9, saat efek *masking* ini dikurangi

drastis, performa *Freeze Encoder* justru melesat.

- **Stabilitas Gradien yang Lebih Baik:** Karena LoRA hanya melatih sebagian sangat kecil dari total parameter model (hanya 2,93%), kita bisa menarik tuas *learning rate* hingga 25 kali lebih tinggi (5×10^{-4} vs 2×10^{-5}) tanpa takut proses belajarnya meledak (*gradient explosion*) [46]. Ini memungkinkan LoRA beradaptasi dan mengeksplorasi ruang parameter jauh lebih efektif dalam jumlah langkah yang sama, sementara *Freeze Encoder* terpaksa harus melaju lambat agar pelatihannya tetap stabil.

4.5.2 Kondisi Keuntungan Penggunaan Freeze Encoder

Walaupun hasil eksperimen secara empiris membuktikan LoRA lebih unggul dari segi akurasi (WER/CER) dan efisiensi, strategi membekukan *encoder* secara total (*Freeze Encoder*) bukannya tanpa guna. Ada beberapa situasi taktis di mana mengunci *encoder* justru menjadi pilihan yang paling cerdas dan direkomendasikan:

1. **Mencegah AI “Pikun” (Melindungi Memori Universal):** Model dasar Whisper sudah sangat pintar berkat pelatihannya mendengarkan 680.000 jam audio dari berbagai bahasa [1]. Jika kita mengotak-atik *encoder*-nya (termasuk lewat LoRA), ada risiko model malah melupakan kemampuan dasar yang sudah ia kuasai (*catastrophic forgetting*). Dengan *Freeze Encoder*, kita mengunci rapat memori pendengaran model sehingga ketajaman ekstraksi audionya 100% aman dan tidak mengalami degradasi.
2. **Jauh Lebih Sempel dan Praktis (Zero-Overhead Sub-System):** Dari sisi pemrograman, membekukan *encoder* itu sangat gampang, tinggal mematikan fitur pelacakan gradiennya. Kita tidak perlu repot mengurus *library* tambahan (seperti Hugging Face PEFT) dan terbebas dari pusingnya melakukan *hyperparameter tuning* struktural yang rumit ala LoRA (seperti menentukan nilai *rank*, *alpha*, atau penentuan target modul).

3. **Sangat Irit untuk Aplikasi Banyak Bahasa (*Shared-Encoder*):**

Bayangkan sebuah sistem *production* yang harus melayani belasan bahasa daerah sekaligus. Dengan *Freeze Encoder*, sistem cukup memuat satu *Universal Encoder* utama di dalam memori (RAM/VRAM). Fitur suara dari *encoder* ini nantinya tinggal dicabangkan ke banyak modul *Decoder* independen berukuran kecil yang khusus dilatih untuk masing-masing bahasa. Sistem *plug-and-play* yang sangat hemat dan terukur (*scalable*) ini mustahil dilakukan jika tiap logat mengharuskan sistem memuat *adapter* LoRA yang mengintervensi *encoder* secara terpisah.

4. **Kedekatan Fonetik yang Ekstrem dengan Bahasa Sumber:** Jika bahasa target (misalnya dialek daerah) memiliki aturan pengucapan (*phonotactics*) dan profil suara yang nyaris sama persis dengan bahasa sumber yang telah dikuasai model (misalnya bahasa Indonesia formal), adaptasi cara mendengar di *encoder* tidak lagi relevan. Sistem murni hanya membutuhkan pengayaan leksikon lokal dan tata bahasa di bagian *decoder* untuk dapat menerjemahkan fitur wicara tersebut ke dalam aksara logat yang benar.

4.5.3 Dampak dan Mekanisme Weighted Cross-Entropy

Teknik WCE yang diimplementasikan dalam penelitian ini bekerja dengan cara menganalisis frekuensi kesalahan token di seluruh prediksi dari percobaan-percobaan sebelumnya, kemudian membangun bobot per-token berdasarkan rumus:

$$w_t = w_{\text{base}} + (w_{\text{max}} - w_{\text{base}}) \cdot \sigma\left(\frac{e_t - \theta}{\lambda}\right) \quad (\text{Rumus 4.5})$$

di mana w_t adalah bobot token t , e_t adalah *error rate* token tersebut, θ adalah ambang batas *error rate*, dan σ adalah fungsi sigmoid. Dalam implementasi penelitian ini digunakan $w_{\text{base}} = 1,0$, $w_{\text{max}} = 3,0$, $\theta = 0,3$, dan $\lambda = 0,5$. Dengan

konfigurasi ini, token yang memiliki *error rate* tinggi mendapatkan bobot hingga $3\times$ dari bobot standar, sehingga loss yang dikontribusikan oleh prediksi yang salah pada token tersebut lebih besar [35].

Analisis dampak WCE menunjukkan:

- **Butuh Racikan Pengaturan yang Pas:** Mengaktifkan WCE saja tidak otomatis membuat performa model melonjak; ia sangat bergantung pada kombinasi *hyperparameter* pendamping yang tepat. Sebagai contoh, percobaan FE-7 mencetak tingkat kesalahan (WER) 19,57%. Namun, ketika WCE dikawinkan dengan racikan yang lebih pas di FE-9 (porsi data dicekikan, waktu belajar diperpanjang jadi 400 langkah, serta minim sensor suara dan batasan), kesalahannya sukses ditekan hingga 18,24%. Kombinasi yang klop ini terbukti memangkas tingkat kesalahan hingga 1,33 poin persentase jika dibandingkan dengan model terbaik yang tidak memakai WCE (FE-4 yang mencetak 19,92%).
- **Sangat Bertenaga di Kapasitas LoRA Besar (*Rank* 16):** Daya gedor WCE paling terasa saat dipadukan dengan kapasitas adaptasi LoRA yang lebih besar (*rank* 16) pada percobaan L-9. Tingkat kesalahannya berhasil turun tajam ke 16,33%, dan bahkan mampu menyelamatkan skenario terburuk di salah satu partisi data uji (*worst-case fold* turun dari 21,51% menjadi 18,32%). Alasannya logis: ruang memori yang lebih luas (*rank* 16 dibanding 8) [42] memberikan fleksibilitas dan kebebasan ekstra bagi model untuk benar-benar menyerap panduan WCE, sehingga ia jauh lebih efektif saat harus mengoreksi token-token kata yang sulit.
- **Butuh Waktu Belajar yang Cukup:** Dari percobaan L-7 dan L-8 (batas 100 langkah), terlihat bahwa model yang memakai WCE malah tidak bisa mengungguli model tanpa WCE jika waktu latihannya terlalu singkat. Kesimpulannya, WCE menuntut proses belajar yang cukup (idealnya 400 *steps* ke atas) agar model memiliki waktu yang memadai untuk mengadaptasi perilakunya terhadap distribusi pembobotan kesalahan yang

baru.

4.5.4 Pengaruh Pengurangan Regularisasi

Serangkaian percobaan FE-7 hingga FE-9 secara sistematis mengurangi intensitas regularisasi dalam beberapa dimensi:

- **Dropout:** Diturunkan dari 0,3/0,2/0,2 (FE-1 hingga FE-5) ke 0,2/0,15/0,15 (FE-6, FE-7) ke 0,1/0,1/0,1 (FE-8, FE-9). Pengurangan ini konsisten dengan temuan bahwa strategi *Freeze Encoder* yang hanya melatih ~37M parameter decoder sudah memiliki kapasitas terbatas, sehingga penghambatan tambahan dari *dropout* yang agresif menyebabkan *underfitting* [44].
- **Weight decay:** Dikurangi dari 0,2 ke 0,1 ke 0,05 ke 0,01. Meskipun *weight decay* secara teoritis meningkatkan generalisasi dengan membatasi magnitude bobot [48], kombinasi *weight decay* 0,1 dengan *dropout* 0,2 dan *label smoothing* 0,1 menciptakan penalti regularisasi yang terlalu berat untuk dataset 1,3 jam [47]. Weight decay 0,01 pada FE-9 memberikan regularisasi L2 yang sangat minimal, memungkinkan decoder mengabsorpsi pola bahasa Minangkabau secara lebih bebas.
- **SpecAugment:** Dikurangi dari 80/40 ke 20/20 ke 10/10. Pengurangan ini memiliki dua motivasi: (a) konflik dengan *frozen encoder* seperti dijelaskan sebelumnya, dan (b) pada dataset sangat kecil, augmentasi yang terlalu agresif mengurangi kandungan informasi linguistik yang tersedia per sampel [19].

4.5.5 Analisis Trade-Off Performa vs Konsistensi

Perbandingan FE-9 dan L-10 mengungkap dinamika yang menarik antara performa rata-rata (*mean WER*) dan konsistensi (*std WER*):

- L-10 mengungguli FE-9 pada WER rata-rata (15,67% vs 18,24%), CER rata-rata (3,32% vs 3,87%), serta memiliki *worst-case fold* yang jauh

lebih baik (17,20% vs 22,22%).

- Dibandingkan dengan L-1 (percobaan LoRA tanpa WCE terbaik, WER 17,32%, std 0,83%), L-10 tetap mampu mencatat standar deviasi yang kompetitif (1,02%), sedikit di atas L-1 namun jauh lebih stabil dari L-9 (2,03%). Hal ini menunjukkan bahwa mekanisme *Temporal Separation* pada L-10 secara efektif menekan variabilitas antar-fold yang sebelumnya muncul pada L-9, sehingga peningkatan kapasitas (*rank* 16) kini memberikan keuntungan rata-rata yang lebih besar *sekaligus* konsistensi yang lebih baik.
- Dari perspektif *deployment*, L-10 menawarkan *expected performance* terbaik (WER 15,67%) sekaligus konsistensi yang kuat (std 1,02%), menjadikannya pilihan unggul dibandingkan alternatif lainnya pada hampir semua skenario penggunaan.

4.5.6 Dampak Augmentasi Data

Percobaan FE-5 dan L-5 secara konsisten menghasilkan WER yang lebih buruk dibandingkan percobaan sebelumnya meskipun menggunakan augmentasi yang lebih kuat:

- FE-5 (21,14%) vs FE-4 (19,92%): kenaikan 1,22 poin persentase
- L-5 (18,37%) vs L-4 (17,43%): kenaikan 0,94 poin persentase

Konsistensi temuan ini pada kedua strategi memperkuat kesimpulan bahwa augmentasi yang berlebihan tidak menguntungkan pada dataset dengan jumlah sampel sangat terbatas. Hal ini selaras dengan observasi Liang dan Levow [13] yang menyatakan bahwa strategi augmentasi untuk bahasa-bahasa *low-resource fieldwork* perlu dikalibrasi secara hati-hati untuk menjaga keseimbangan antara diversifikasi data dan preservasi informasi linguistik.

4.5.7 Transfer Learning dan Kedekatan Tipologis

Keberhasilan kedua strategi, terutama LoRA yang hanya memodifikasi 2,93% parameter, mengkonfirmasi efektivitas *transfer learning* [37] dari model Whisper Base yang telah dilatih pada 680.000 jam audio dengan 96 bahasa [1]. Penggunaan *language token* bahasa Indonesia (id) sebagai *proxy* untuk bahasa Minangkabau terbukti efektif, yang dapat dijelaskan melalui kedekatan tipologis antara kedua bahasa sebagai anggota rumpun Austronesia [9]. Pengetahuan fonotaktik dan leksikal yang tumpang tindih antara bahasa Indonesia dan Minangkabau memungkinkan model untuk memanfaatkan representasi yang sudah dipelajari tanpa perlu pelatihan dari nol.

WER terbaik yang dicapai dalam penelitian ini (L-10, 15,67%) berada dalam kisaran yang kompetitif dengan penelitian serupa pada bahasa-bahasa dengan sumber daya serupa. Pillai dkk. [7] melaporkan perbaikan WER yang signifikan melalui *multi-stage fine-tuning* pada bahasa-bahasa *low-resource*; Li dkk. [36] menunjukkan perbaikan signifikan melalui pemanfaatan data teks tambahan tanpa pasangan audio pada bahasa Kazakh dengan data audio terbatas; Gupta dkk. [52] mendemonstrasikan efisiensi *parameter-efficient fine-tuning* (PEFT) pada ASR multibahasa dan menemukan bahwa metode PEFT secara konsisten lebih efektif dari *full fine-tuning* pada data kecil; Song dkk. [4] mengaplikasikan LoRA pada Whisper untuk skenario ekspansi multibahasa; Gete dkk. [34] berhasil melakukan *fine-tuning* Whisper pada bahasa Amharik (bahasa Afrika yang juga *low-resource*) dengan dataset sangat terbatas; serta Timmel dkk. [3] menunjukkan efektivitas *fine-tuning* Whisper pada berbagai bahasa *low-resource* untuk aplikasi nyata. Sebagai titik acuan, pada bahasa Indonesia baku (bukan bahasa Minangkabau) model Whisper turbo tanpa *fine-tuning* khusus dilaporkan mencapai WER 7,97% [14]; perbedaan ini mencerminkan ketersediaan sumber daya pra-latih yang jauh lebih besar untuk bahasa Indonesia dibandingkan bahasa Minangkabau yang merupakan bahasa *low-resource* tanpa

dukungan *pre-training* eksplisit dalam model Whisper. Mempertimbangkan bahwa dataset yang digunakan dalam penelitian ini (~1,3 jam) jauh lebih kecil dari kebanyakan penelitian tersebut, hasil WER 15,67% yang dicapai dalam penelitian ini dapat dianggap sangat kompetitif.

4.5.8 Pemetaan Kesalahan Berdasarkan Kelas Fonetik dan Linguistik Minangkabau

Berdasarkan analisis kesalahan pada Subbab 4.2 (model *Freeze Encoder*) dan Subbab 4.3 (model LoRA), performa model dalam bahasa Minangkabau dapat dibedah lebih dalam dari kacamata fonetik spesifik. Pengelompokan jenis kegagalan model dalam mengenali pola-pola bunyi lokal dapat memberikan nilai tambah akademis untuk membuktikan di mana letak kelemahan laten representasi akustik internal yang dihasilkan Whisper.

Tabel 4.15 mengategorikan beberapa kesalahan model yang persisten ke dalam kelas fonetik dan struktur linguistik Minangkabau.

Tabel 4.15 Pemetaan Kesalahan Kata Berdasarkan Kelas Fonetik Spesifik Minangkabau

Kategori Fonetik / Linguistik	Referensi	Contoh Prediksi Model	Analisis Fonologis
Deret Vokal & Diftong (-ao, -ua)	mambao, keluarga	mambo, mambuh, keluarga	Kesalahan paling sering terjadi saat model mereduksi diftong atau deret vokal (seperti /ao/ menjadi /o/ tunggal pada <i>mambo</i>). Hal ini juga mengindikasikan interferensi (<i>bias</i>) kuat dari kata kognat dalam bahasa Indonesia (<i>keluarga</i>).
Substitusi Vokal Depan	sasungguahnyo	susungguahnyo	Kegagalan membedakan vokal terbuka /a/ dan tertutup /u/ pada silabel sebelum penekanan, diakibatkan rentang frekuensi forman yang saling tumpang tindih dalam wicara lisan.
Kesalahan Akhiran Nasal / Sengau	ka dunia	kadunian	Munculnya bunyi nasal epentetis (-n) di akhir kata padahal secara leksikal tidak ada. Ini menunjukkan ambiguitas fitur akustik hening di akhir tuturan.
Kesalahan Substitusi Konsonan	dasar, panghormatan	tasar, pangormatan	Penggantian antara <i>voiced</i> (/d/) dan <i>voiceless</i> (/t/), serta pelemahan atau hilangnya frikatif glotal /h/ di tengah kata. Menandakan resolusi encoder yang belum sepenuhnya mapan untuk membedakan fitur <i>voicing</i> .
Batas Kata (Segmentasi)	mukadimah	muka di ma	Kegagalan dalam <i>tokenization</i> , model memecah kata utuh menjadi fragmen silabis yang dianggap memiliki batas jeda.

Pengelompokan ini menegaskan bahwa meskipun LoRA berhasil memperbaiki WER secara keseluruhan berkat adaptasinya pada lapisan *encoder*, namun sifat akustik khusus seperti diftong (*mambao*), pergeseran vokal (*sasungguahnyo*), serta pengaruh dominan dari ortografi bahasa Indonesia (*keluarga*) masih sering “membajak” keluaran dekoder akibat kuatnya *prior* bahasa Indonesia di dalam representasi model.

4.5.9 Limitasi dan Tantangan

Beberapa batasan dan tantangan yang perlu dipertimbangkan dalam interpretasi hasil penelitian ini:

- **Data yang Sangat Terbatas:** Dataset berdurasi 1,3 jam merupakan batas minimum agar model sekelas Whisper Base bisa menghasilkan transkripsi yang masuk akal [13]. Durasi sesingkat ini belum cukup untuk mewakili

ragam dialek dan gaya bicara yang ada dalam bahasa Minangkabau.

- **Risiko Nilai yang Terlalu Optimis:** Penggunaan himpunan data yang sama untuk *early stopping* sekaligus untuk pelaporan metrik akhir merupakan keterbatasan metodologis. Meskipun demikian, penggunaan *5-fold cross-validation* telah secara substansial memitigasi risiko bias ini dibandingkan jika hanya mengevaluasi pada satu partisi data tunggal [49].
- **Ejaan yang Belum Baku:** Ketiadaan standar ortografi resmi untuk bahasa Minangkabau menyebabkan inkonsistensi penulisan dalam dataset. Hal ini secara artifisial menaikkan nilai tingkat kesalahan (WER), karena variasi ejaan yang secara fonetis bunyinya identik tetap dihitung sebagai kesalahan oleh sistem [9], [25].
- **Konteks Suara yang Kaku:** Mengingat data bersumber dari LibriVox (pembacaan teks formal), performa model dalam menggeneralisasi percakapan spontan atau dialek regional tertentu dalam kondisi nyata belum dapat dijamin.
- **Uji Coba WCE yang Sempit “Bocor”:** Pada eksperimen Fase 2 (FE-7 hingga L-9), pembobotan WCE mengacu pada seluruh riwayat prediksi tanpa memisahkan data validasi yang sedang digunakan, sehingga perbandingan antar-percobaan tidak sepenuhnya terkontrol (*controlled*). Keterbatasan ini telah diatasi pada Fase 3 melalui mekanisme *Temporal Separation* (FE-10 dan L-10), di mana bobot WCE hanya dihitung dari riwayat run yang sudah selesai. Pemisahan waktu ini tidak hanya menjamin kesahihan metodologis, tetapi secara empiris juga terbukti meningkatkan konsistensi kestabilan model (standar deviasi WER pada L-10 turun drastis menjadi 1,02% dibandingkan L-9 yang berada di angka 2,03%).

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Penelitian ini telah berhasil mengadaptasi model *Automatic Speech Recognition* (ASR) OpenAI Whisper Base untuk bahasa Minangkabau melalui *fine-tuning* dengan dua strategi yang berbeda. Berdasarkan serangkaian 20 percobaan eksperimen dan analisis menyeluruh yang disajikan pada Bab IV, kesimpulan penelitian ini disusun untuk menjawab kelima poin rumusan masalah secara berurutan.

1. Adaptasi Whisper Base untuk Bahasa Minangkabau (*Low-Resource*)

Model Whisper Base berhasil diadaptasi untuk mengenali ucapan bahasa Minangkabau melalui proses *fine-tuning* pada dataset *extremely low-resource* (156 file audio, ~1,3 jam). Strategi adaptasi terbaik yang ditemukan merupakan kombinasi dari: (a) pemilihan *hyperparameter* yang tepat melalui eksplorasi iteratif selama sembilan percobaan per strategi, (b) augmentasi data yang terkalibrasi (*speed perturbation* 0,85–1,15, *SpecAugment* minimal, *pitch shifting*), (c) regularisasi berlapis (*label smoothing* 0,1, *weight decay* minimal, *dropout* adaptif), dan (d) penerapan *Weighted Cross-Entropy* (WCE) yang dirancang berdasarkan analisis pola kesalahan historis untuk memperkuat sinyal gradien pada token-token yang sulit.

Penggunaan *language token* bahasa Indonesia (id) sebagai *proxy* terbukti efektif memanfaatkan kedekatan tipologis antara bahasa Indonesia dan bahasa Minangkabau sebagai sesama anggota rumpun Austronesia [9]. Pendekatan *transfer learning* ini memungkinkan model memanfaatkan representasi fonotaktik dan leksikal yang sudah dipelajari tanpa perlu pelatihan dari nol, sehingga keterbatasan data (1,3 jam) dapat diatasi secara

signifikan. Tambahan penting pada implementasi WCE adalah mekanisme *Temporal Separation*: bobot per-token hanya diturunkan dari riwayat percobaan yang sudah selesai, sehingga model benar-benar *belajar dari kesalahan historisnya sendiri* tanpa mengeksploitasi label validasi yang sedang digunakan. Hal ini merupakan formulasi praktis komputasional dari gagasan *importance weighting* Sugiyama dkk. [67] guna menghindari *leakage* pada distribusi set, sebuah prinsip integritas metodologis yang terbukti secara empiris meningkatkan konsistensi prediksi.

2. Implementasi Sistem ASR Berbasis Python, PyTorch, dan Hugging Face Transformers

Sistem ASR bahasa Minangkabau berhasil dirancang dan diimplementasikan menggunakan ekosistem *open-source*: Python sebagai bahasa pemrograman utama, PyTorch sebagai *framework deep learning*, Hugging Face Transformers untuk manajemen model dan *tokenizer*, serta PEFT (*Parameter-Efficient Fine-Tuning*) library untuk implementasi LoRA. *Pipeline* yang dibangun mencakup modul *audio preprocessing* (konversi format, *resampling* ke 16 kHz, validasi durasi), *text preprocessing* (normalisasi teks Minangkabau), augmentasi data (*SpecAugment*, *speed perturbation*, *noise injection*), dan infrastruktur *training* dengan *monitoring* real-time melalui *Weights & Biases*. Seluruh kode diimplementasikan dengan desain modular yang memungkinkan konfigurasi fleksibel dan reproduibilitas eksperimen.

3. Perbandingan Strategi *Freeze Encoder* vs LoRA

Perbandingan komprehensif antara dua strategi *fine-tuning* ini menghasilkan temuan-temuan berikut:

- (a) **LoRA Menang Telak dan Konsisten:** Strategi LoRA (khususnya racikan L-10) keluar sebagai juara dengan rata-rata tingkat kesalahan kata (WER) $15,67\% \pm 1,02\%$ dan CER $3,32\% \pm 0,38\%$. Angka ini jelas mengungguli rekor terbaik *Freeze Encoder* (FE-9) yang tertahan

di $18,24\% \pm 2,18\%$. Kehebatan LoRA ini bersifat sistematis; bahkan rentang skor terburuk dari seluruh eksperimen LoRA masih berada di bawah atau setara dengan skor puncak milik *Freeze Encoder*.

- (b) **Sangat Irit, Namun Lebih Akurat:** LoRA (pada L-10, *rank* 16) hanya melatih sekitar 2,93% dari total parameter model ($\sim 2,16$ juta dari 74 juta). Ini berarti LoRA 17 kali lebih efisien secara komputasi ketimbang *Freeze Encoder* yang melatih 50% parameter (~ 37 juta). Ajaibnya, meski sangat hemat, LoRA justru menghasilkan tingkat kesalahan (WER) yang 2,57 poin persentase lebih rendah dengan standar deviasi yang jauh lebih kecil (1,02% vs 2,18%).
- (c) **Rekor *Single-Fold* Dipegang LoRA:** Jika dibedah per partisi evaluasi, L-9 Fold 4 mencapai WER 12,83% dan L-9 Fold 2 mencapai CER 2,77%. Angka ini merupakan rekor per-*fold* paling sempurna di antara seluruh 20 percobaan dan 100 evaluasi *fold* yang dilakukan. Menariknya, walau L-10 tidak memecahkan rekor *peak* individu ini, ia tetap meraih rata-rata terbaik secara keseluruhan (15,67%), membuktikan bahwa *Temporal Separation* sukses meratakan konsistensi performa antar-*fold*.
- (d) **Kombinasi Kuat WCE dan *Temporal Separation*:** Penerapan WCE pada fase akhir sukses memangkas WER sebesar 1,68 poin persentase pada *Freeze Encoder* dan 0,99 poin persentase pada LoRA. Puncaknya, saat mekanisme *Temporal Separation* diaktifkan pada Run 10 (L-10), WER turun lebih jauh ke 15,67% sekaligus memotong standar deviasi menjadi 1,02% (paling stabil dari seluruh 20 percobaan). Mekanisme ini memastikan bobot WCE dihitung murni dari riwayat masa lalu tanpa mengeksploitasi label validasi saat ini [35]. Pendekatan historis ini merupakan pengamalan empiris dari prinsip menghindari *covariate shift* milik Sugiyama dkk. [67].
- (e) **Hati-Hati Manipulasi Data (*Augmentasi*):** *Augmentasi* yang

berlebihan (seperti pada FE-5 dan L-5) secara konsisten menurunkan performa model. Hal ini mengkonfirmasi bahwa pada kondisi dataset yang sangat terbatas, menjaga keseimbangan antara diversifikasi data dan preservasi informasi fonetik asli merupakan hal yang sangat kritis.

4. Evaluasi dan Analisis Kesalahan Model

Evaluasi menggunakan skema *5-fold cross-validation* menghasilkan estimasi performa yang andal dan tidak bias. Analisis kualitatif terhadap prediksi model mengidentifikasi tiga pola kesalahan dominan yang bersifat sistematis:

- (a) **Substitusi fonetik:** Penggantian konsonan dengan artikulasi serupa (contoh: *paralu* → *palu*, *barontak* → *barantak*, *kazaliman* → *kajaliman*, *mandorong* → *mandaurang*). Pola ini dominan pada strategi *Freeze Encoder* dan mencerminkan keterbatasan representasi akustik dari encoder yang dibekukan dalam membedakan konsonan-konsonan yang memiliki kemiripan akustik tinggi dalam bahasa Minangkabau.
- (b) **Segmentasi kata:** Kesalahan pada batas kata (*kaadilan* → *ka adilan*, *mangukuahkan* → *manguku hakkan*), mencerminkan tantangan intrinsik yang belum sepenuhnya terselesaikan oleh kedua strategi. Fenomena ini berkaitan dengan ketiadaan standar ortografi resmi bahasa Minangkabau.
- (c) **Substitusi vokal:** Pengubahan bunyi vokal pada suku kata tertentu (contoh: *elok* → *elo*), bersifat ringan dengan kontribusi kecil pada CER.

Dibandingkan dengan *Freeze Encoder*, *Fold 4* dari LoRA L-9 berhasil memprediksi sempurna (WER 0%, CER 0%) sejumlah kalimat panjang yang kompleks, membuktikan efektivitas adaptasi LoRA di seluruh lapisan encoder dan decoder model.

5. Kontribusi terhadap Pelestarian Bahasa Daerah

Penelitian ini berhasil menghasilkan prototipe sistem ASR bahasa Minangkabau pertama berbasis Whisper dengan WER 15,67% (L-10, std 1,02%) pada domain pembacaan formal. Kontribusi utama terhadap pelestarian bahasa daerah mencakup: (1) terbentuknya *pipeline* teknis yang sepenuhnya terdokumentasi dan dapat direproduksi untuk adaptasi model ASR ke bahasa-bahasa daerah Indonesia lain yang juga *low-resource*, (2) penyediaan *baseline* performa kuantitatif (WER 15,67%, CER 3,32%) yang dapat dijadikan titik acuan bagi penelitian lanjutan, (3) demonstrasi bahwa dengan hanya ~1,3 jam data audio, sistem ASR fungsional untuk bahasa daerah dapat dibangun menggunakan metode PEFT yang efisien secara komputasi [42], [52], (4) pengenalan mekanisme *Temporal Separation* pada WCE yang memungkinkan model *belajar dari kesalahan historisnya sendiri* tanpa sumber daya linguistik eksternal. Mekanisme *weighting* temporal yang bebas distorsi iterasi (*leakage*) sejalan dengan konsep *importance-weighting* yang diinspirasi oleh Sugiyama [67], adalah teknik yang dapat direplikasi pada bahasa-bahasa daerah lain yang juga tidak memiliki korpus acuan seperti KBBI [35], serta (5) dokumentasi pola kesalahan sistematis yang dapat memandu pengembangan dataset dan standarisasi ortografi bahasa Minangkabau di masa depan [9], [25].

Secara keseluruhan, penelitian ini menunjukkan bahwa *fine-tuning* Whisper Base dengan strategi LoRA merupakan pendekatan yang paling efektif dan efisien untuk skenario ASR bahasa *low-resource* seperti bahasa Minangkabau. Keunggulan LoRA tidak hanya terletak pada efisiensi parameter, tetapi juga pada kemampuannya menghasilkan adaptasi yang lebih menyeluruh dengan risiko *overfitting* yang lebih rendah melalui regularisasi struktural dari *low-rank constraint* [4], [42]. Penambahan WCE dengan *Temporal Separation* membuktikan bahwa pada skenario *extremely low-resource* tanpa korpus eksternal seperti KBBI, model dapat menyempurnakan dirinya sendiri secara

iteratif belajar dari kesalahan historisnya dengan integritas metodologis yang terjamin menghasilkan model akhir (L-10) dengan WER terbaik keseluruhan ($15,67\% \pm 1,02\%$) dan konsistensi tertinggi di antara seluruh 20 percobaan.

5.2 Saran

Berdasarkan temuan dan batasan yang ditemui selama penelitian ini, berikut adalah beberapa rekomendasi strategis untuk penelitian ASR bahasa Minangkabau ke depannya:

1. Perbanyak dan Perkaya Data Suara

Keterbatasan terbesar riset ini adalah sempitnya sumber data (hanya 1,3 jam dari pembacaan teks formal). Ke depannya, strategi pengumpulan data harus lebih agresif dan merakyat. Caranya bisa melalui *crowdsourcing* (partisipasi warga), atau menyedot data lisan dari radio, televisi, YouTube, hingga *podcast* lokal di wilayah Sumatera. Fokus utamanya ada tiga: (a) memperbanyak rekaman obrolan santai agar model lebih luwes; (b) merekam berbagai logat daerah (seperti Agam, Lima Puluh Kota, Pasaman, Padang Pariaman) yang punya cengkok fonetik unik; serta (c) merumuskan standar ejaan (ortografi) agar model tidak terus-terusan disalahkan (nilai WER naik semu) hanya karena inkonsistensi cara penulisan kata.

2. Coba Varian Model yang Lebih Bongsor (*Scale-Up*)

Penelitian ini sengaja menahan diri pada Whisper Base (74M parameter) demi menghindari *overfitting* pada data yang minim. Jika kelak ketersediaan data sudah melimpah, sangat disarankan untuk “naik kelas” menggunakan varian Whisper Small (244M) atau Medium (769M) yang kapasitas representasinya jauh lebih tajam. Berkat efisiensi metode LoRA yang hanya perlu melatih sebagian sangat kecil dari total parameter ($<3\%$), ongkos komputasi untuk mengeksekusi model raksasa ini akan tetap terjangkau dan masuk akal.

3. Latihan Bertahap (*Multi-Stage Fine-Tuning*)

Ibarat pemanasan, model bisa diajari bahasa Indonesia terlebih dahulu (karena datanya melimpah) sebelum difokuskan pada bahasa Minangkabau [7]. Karena kedua bahasa ini masih sangat serumpun, "transfer ilmu" secara bertahap ini akan jauh lebih mulus ketimbang memaksa model multibahasa langsung melompat belajar bahasa Minang. Tahap pemanasan ini juga bisa digenjut dengan manipulasi data (*augmentasi*) yang lebih ekstrem.

4. **Pasang "Kamus Pintar" (*Language Model*) di Tahap Akhir**

Saat ini, sistem hanya mengandalkan tebakan standar (*beam search*) saat mencetak teks. Padahal, riset Arisaputra dan Zahra [12] membuktikan bahwa memasang *language model* tambahan (seperti KenLM) sukses memangkas tingkat kesalahan ASR bahasa Indonesia dari kisaran 20% menjadi 12%. Strategi praktis ini sangat layak ditiru: kita cukup menyusun kamus probabilitas dari kumpulan teks Minang di Wikipedia atau sastra digital, tanpa perlu repot menambah rekaman audio.

5. **Eksplorasi Teknik Optimasi Tingkat Lanjut**

Masih banyak "jurus" teknis yang bisa diujal untuk mendongkrak performa, misalnya: (a) *monotonic alignment loss* [11] untuk lebih merapikan sinkronisasi ketukan suara dan teks saat pelatihan; (b) *Mixture of LoRA Experts* [6] agar model punya spesialis pembobot adaptif untuk masing-masing logat; serta (c) *curriculum learning* [13], yakni melatih model mulai dari rekaman yang paling gampang dan jernih dulu, baru pelan-pelan diberi materi yang lebih berisik dan susah.

6. **Sistem Pengujian dan Metodologi yang Lebih Ketat**

Agar metrik evaluasi model tidak terkesan terlalu optimis atau bias (karena saat ini memakai data yang sama untuk *early stopping* dan penilaian akhir), metodologi pengujian harus diperketat. Jika volume data sudah cukup besar (>5 jam), riset selanjutnya wajib menyisahkan data validasi secara terpisah, atau menyiapkan set ujian rahasia (*held-out test set*) yang sama

sekali tidak pernah disentuh model selama masa pelatihan. Ini menjamin evaluasi akhir benar-benar independen dan objektif.

7. Wujudkan Jadi Aplikasi Siap Pakai (*End-to-End*)

Prototipe ASR yang dihasilkan sekarang ibarat mesin canggih yang belum dipasang ke bodi mobil. Agar dampak pelestarian bahasanya benar-benar terasa oleh masyarakat luas, model komputasi ini perlu dibungkus menjadi produk peranti lunak utuh. Beberapa ide implementasinya: (a) membangun aplikasi *web* atau *mobile* yang bisa menerjemahkan tuturan bahasa Minang secara *real-time*; (b) menciptakan perangkat pembuat *subtitle* otomatis untuk kreator konten daerah; atau (c) membangun sistem pengarsipan digital pintar untuk menelusuri dan mendigitalisasi rekaman kaba, cerita rakyat, serta lagu tradisional.

DAFTAR PUSTAKA

- [1] Alec Radford et al. “Robust Speech Recognition via Large-Scale Weak Supervision”. *International Conference on Machine Learning (ICML)*. PMLR. 2023, pp. 28492–28518.
- [2] Yunpeng Liu, Xukui Yang, and Dan Qu. “Exploration of Whisper Fine-Tuning Strategies for Low-Resource ASR”. *EURASIP Journal on Audio, Speech, and Music Processing* 2024.29 (2024).
- [3] Vincenzo Timmel et al. “Fine-tuning Whisper on Low-Resource Languages for Real-World Applications”. *arXiv preprint arXiv:2412.15726* (2024).
- [4] Zheshu Song et al. “LoRA-Whisper: Parameter-Efficient and Extensible Multilingual ASR”. *arXiv preprint arXiv:2406.06619* (2024).
- [5] Pu Wang, Shinji Watanabe, and Hugo Van hamme. “SSVD: Structured SVD for Parameter-Efficient Fine-Tuning and Benchmarking under Domain Shift in ASR”. *IEEE ASRU 2025* (2025).
- [6] Raphaël Bagat, Irina Illina, and Emmanuel Vincent. “Mixture of LoRA Experts for Low-Resourced Multi-Accent Automatic Speech Recognition”. *Interspeech 2025*. 2025.
- [7] Leena G. Pillai et al. “Multistage Fine-Tuning Strategies for Automatic Speech Recognition in Low-Resource Languages”. *arXiv preprint arXiv:2411.04573* (2024).
- [8] Sourav Banerjee, Ayushi Agarwal, and Promila Ghosh. “High-Precision Medical Speech Recognition through Synthetic Data and Semantic Correction: UNITED-MEDASR”. *arXiv preprint arXiv:2412.00055* (2024).

- [9] Fajri Koto and Ikhwan Koto. “Towards Computational Linguistics in Minangkabau Language: Studies on Sentiment Analysis and Machine Translation”. *arXiv preprint arXiv:2009.09309* (2020).
- [10] Ayu Hirza Nasution and Aytug Onan. “ChatGPT Label: Comparing the Quality of Human-Generated and LLM-Generated Annotations in Low-Resource Language NLP Tasks”. 2024.
- [11] Ziyang Zhuang et al. “Towards Efficiently Whisper Fine-tuning with Monotonic Alignments”. *Proceedings of Interspeech 2025*. 2025.
- [12] Panji Arisaputra and Amalia Zahra. “Indonesian Automatic Speech Recognition with XLSR-53”. *Ingénierie des Systèmes d’Information* 27.6 (2022), pp. 973–982.
- [13] Siyu Liang and Gina-Anne Levow. “Breaking the Transcription Bottleneck: Fine-tuning ASR Models for Extremely Low-Resource Fieldwork Languages”. *Proceedings of the Workshop on FieldMatters (ACL 2025)*. 2025.
- [14] Martin Clinton Tosima Manullang et al. “From Speech to Summary: A Pipeline-Based Evaluation of Whisper and Transformer Models for Indonesian Dialogue Summarization”. *Journal of Applied Informatics and Computing (JAIC)* 10.1 (Feb. 2026), pp. 522–534. 2548-6861.
- [15] Andrew Morris, Viktoria Maier, and Phil Green. “From WER and RIL to MER and WIL: Improved Evaluation Measures for Connected Speech Recognition”. *Eighth International Conference on Spoken Language Processing*. 2004.
- [16] Vladimir I Levenshtein. “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals”. *Soviet Physics Doklady* 10.8 (1966), pp. 707–710.
- [17] Justin Salamon et al. “Scaper: A Library for Soundscape Synthesis and Augmentation”. *2017 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*. 2017, pp. 344–348.

- [18] Jonghyuk Kim et al. “Augmentation of Speech Recognition Models using Data Augmentation”. *Applied Sciences* 10.23 (2020), p. 8517.
- [19] Daniel S Park et al. “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition”. *Interspeech 2019*. 2019, pp. 2613–2617.
- [20] Tom Ko et al. “Audio augmentation for speech recognition”. *16th Annual Conference of the International Speech Communication Association (Interspeech)*. 2015.
- [21] Alexis Conneau et al. “Unsupervised Cross-lingual Representation Learning at Scale”. *arXiv preprint arXiv:1911.02116* (2020).
- [22] Vineel Pratap et al. “MLS: A Large-Scale Multilingual Dataset for Speech Research”. *Interspeech*. 2020.
- [23] Janne Laakkonen, Ivan Kukanov, and Ville Hautamäki. “Generalizable Speech Deepfake Detection via Meta-Learned LoRA”. *arXiv preprint arXiv:2502.10838* (2025).
- [24] Zirui Lin et al. “Dialect Identification Using Resource-Efficient Fine-Tuning Approaches”. *APSIPA ASC 2025* (2025).
- [25] Rini Sovia and Sarjon Defit. “Development of the Minangkabau Local Language Translation Machine Based on Stemming”. *2022 International Symposium on Information Technology and Digital Innovation (ISITDI)*. IEEE. 2022.
- [26] Biing-Hwang Juang and Lawrence R Rabiner. “Automatic Speech Recognition—A Brief History of the Technology Development”. *Georgia Institute of Technology. Atlanta Rutgers University and the University of California. Santa Barbara* 1 (2005), p. 67.
- [27] Lawrence R Rabiner. “A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition”. *Proceedings of the IEEE* 77.2 (1989), pp. 257–286.

- [28] Geoffrey Hinton et al. “Deep Neural Networks for Acoustic Modeling in Speech Recognition: The Shared Views of Four Research Groups”. *IEEE Signal Processing Magazine* 29.6 (2012), pp. 82–97.
- [29] Alex Graves et al. “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks”. *Proceedings of the 23rd International Conference on Machine Learning*. 2006, pp. 369–376.
- [30] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. “Speech Recognition with Deep Recurrent Neural Networks”. *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE. 2013, pp. 6645–6649.
- [31] William Chan et al. “Listen, Attend and Spell: A Neural Network for Large Vocabulary Conversational Speech Recognition”. *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2016, pp. 4960–4964.
- [32] Ashish Vaswani et al. “Attention is All You Need”. *Advances in Neural Information Processing Systems* 30 (2017).
- [33] Anmol Gulati et al. “Conformer: Convolution-augmented Transformer for Speech Recognition”. *Interspeech*. 2020.
- [34] Dawit Ketema Gete et al. “Whispering in Amharic: Fine-tuning Whisper for Low-Resource Language”. *arXiv preprint arXiv:2503.18485* (2025).
- [35] A Piñeiro-Martín et al. “Weighted Cross Entropy for Automatic Speech Recognition”. *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2024.
- [36] Jinpeng Li et al. “Improving Whisper’s Recognition Performance for Under-Represented Language Kazakh Leveraging Unpaired Speech and Text”. *Proceedings of Interspeech 2024*. arXiv:2408.05554. 2024, pp. 2514–2518.

- [37] Sinno Jialin Pan and Qiang Yang. “A Survey on Transfer Learning”. *IEEE Transactions on Knowledge and Data Engineering* 22.10 (2010), pp. 1345–1359.
- [38] Alexei Baevski et al. “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations”. *NeurIPS*. 2020.
- [39] Alexis Conneau et al. “Unsupervised Cross-Lingual Representation Learning for Speech Recognition (XLS-R)”. *arXiv preprint arXiv:2111.09296* (2021).
- [40] Wei-Ning Hsu et al. “HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units”. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 29 (2021), pp. 3451–3460.
- [41] Sanyuan Chen et al. “WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing”. *IEEE Journal of Selected Topics in Signal Processing* 16.6 (2022), pp. 1505–1518.
- [42] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. *arXiv preprint arXiv:2106.09685* (2022).
- [43] Brian McFee et al. “librosa: Audio and Music Signal Analysis in Python”. *Proceedings of the 14th Python in Science Conference (SciPy 2015)*. 2015, pp. 18–24.
- [44] Nitish Srivastava et al. “Dropout: a simple way to prevent neural networks from overfitting”. *The Journal of Machine Learning Research (JMLR)* 15.1 (2014), pp. 1929–1958.
- [45] Christian Szegedy et al. “Rethinking the inception architecture for computer vision”. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 2818–2826.
- [46] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the Difficulty of Training Recurrent Neural Networks”. *International*

- Conference on Machine Learning (ICML)*. PMLR. 2013, pp. 1310–1318.
- [47] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. *International Conference on Learning Representations (ICLR)*. 2019.
- [48] Anders Krogh and John A. Hertz. “A Simple Weight Decay Can Improve Generalization”. *Advances in Neural Information Processing Systems* 4 (1991), pp. 950–957.
- [49] Ron Kohavi et al. “A study of cross-validation and bootstrap for accuracy estimation and model selection”. *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*. Vol. 14. 2. 1995, pp. 1137–1145.
- [50] Thomas Wolf et al. “Transformers: State-of-the-Art Natural Language Processing”. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*. 2020, pp. 38–45.
- [51] Paulius Micikevicius et al. “Mixed Precision Training”. *International Conference on Learning Representations (ICLR)*. 2018.
- [52] Abhishek Gupta et al. “Parameter-efficient Adaptation of Multilingual Multimodal Models for Low-resource ASR”. *Interspeech 2024*. 2024.
- [53] NVIDIA Corporation. *CUDA Toolkit Documentation*. <https://docs.nvidia.com/cuda/>. Accessed: 2025. 2023.
- [54] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. *Advances in Neural Information Processing Systems* 32 (*NeurIPS*). 2019, pp. 8024–8035.
- [55] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020.
- [56] Student. “The Probable Error of a Mean”. *Biometrika* 6.1 (1908), pp. 1–25.

- [57] Guido Van Rossum and Fred L. Drake. “Python 3 Reference Manual” (2009).
- [58] Thomas Kluyver et al. “Jupyter Notebooks — A Publishing Format for Reproducible Computational Workflows” (2016), pp. 87–90.
- [59] Quentin Lhoest et al. “Datasets: A Community Library for Natural Language Processing”. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*. 2021, pp. 175–184.
- [60] Neil Houlsby et al. “Parameter-Efficient Transfer Learning for NLP”. *International Conference on Machine Learning (ICML)*. 2019, pp. 2790–2799.
- [61] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [62] Charles R. Harris et al. “Array Programming with NumPy”. *Nature* 585.7825 (2020), pp. 357–362.
- [63] Pauli Virtanen et al. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. *Nature Methods* 17.3 (2020), pp. 261–272.
- [64] Wes McKinney. “Data Structures for Statistical Computing in Python”. *Proceedings of the 9th Python in Science Conference (SciPy 2010)*. 2010, pp. 56–61.
- [65] John D. Hunter. “Matplotlib: A 2D Graphics Environment”. *Computing in Science & Engineering* 9.3 (2007), pp. 90–95.
- [66] Sharan Chetlur et al. *cuDNN: Efficient Primitives for Deep Learning*. 2014. arXiv: [1410.0759](https://arxiv.org/abs/1410.0759) [[cs](#).[NE](#)].
- [67] Masashi Sugiyama, Matthias Krauledat, and Klaus-Robert Müller. “Covariate Shift Adaptation by Importance Weighted Cross Validation”. *Journal of Machine Learning Research*. Vol. 8. Foundational work on distribution shift correction; cited for the principle of using historically-

derived importance weights to prevent covariate shift between training and evaluation distributions. 2007, pp. 985–1005.

LAMPIRAN

